

Università degli Studi di Milano Bicocca
Dipartimento di Informatica, Sistemistica e Comunicazione
Corso di Laurea Magistrale in Informatica

A language agnostic framework for Dependency Extraction

Supervisor: Prof. Francesca Arcelli Fontana
Co-Supervisor: Dr. Francesco Refolli

Master's Thesis by:
Christian Ferrario
Student ID 886230

Academic Year 2025-2026

Acknowledgements

I would like to express my deepest gratitude to everyone who contributed to the completion of this thesis and supported me throughout this challenging yet rewarding journey.

First and foremost, I wish to thank Prof. Arcelli and Dr. Refolli for their invaluable guidance, expertise and their great availability, being ready to answer my questions and provide insightful feedback at every stage of this research. Their mentorship has been instrumental in shaping the direction and quality of this work.

Above all, I dedicate this milestone to my family. To my mother and father, who I know are both immensely happy and proud of this achievement: thank you for your unconditional support and for believing in me every step of the way. I am truly fortunate to have you as my parents.

A very special thank you goes to my girlfriend: thank you for standing by my side every single day, for supporting me with love, and above all, for enduring me with boundless patience through every stressful moment. Your presence has been my greatest strength.

Finally, I would like to thank all the colleagues and friend who shared this journey with me, helping in making it lighter, enjoyable and memorable. You know who you are, and I wish the best to all of you in your future endeavors.

Contents

Acknowledgements **i**

Abstract **i**

1 Introduction **1**
1.1 Context and Motivation 1
1.2 Project Objectives 2
1.3 Methodological Approach 2
1.3.1 Type System and Intermediate Representation 3
1.3.2 Pipeline as Function Composition 3
1.3.3 Source Code Availability and Reproducibility 4
1.4 Organization of the Thesis 4

2 Background **6**
2.1 Software Dependencies 6
2.2 Technical Debt and Architectural Smells 7
2.3 Graph-Based Architectural Representation 8
2.3.1 Component and Dependency Graphs 8
2.3.2 Dependency Smells 9
2.4 Syntax Parsing and Tree-sitter 11
2.5 Name Resolution and Lexical Scoping 14

3 Related Work **17**
3.1 Multi-Language Analysis and Tooling 17
3.1.1 Compiler Front-Ends and the $M \times N$ Problem 18
3.1.2 Code Property Graph: Joern 19
3.1.3 Architectural Analyzers: Arcan and Depends 19
3.2 Language-Agnostic Syntax Representations 21
3.3 Graph Models for Reference Resolution 24
3.4 Direct Precursors 25
3.4.1 Stack-Graph Resolution: Skullian 25
3.4.2 Scope-Climbing Resolution: Entangle 25
3.5 Positioning of the Proposed Work 26

4 Intermediate Representation (IR) Model **28**
4.1 Introduction 28
4.2 The Type System 28
4.2.1 Identifiers and Paths (π) 28
4.2.2 Type References (τ) 29

4.3 Modeling Structural Constructs . . . 30
4.3.1 Members and Signatures . . . 31
4.3.2 Structured Types ($\mathcal{S}$) . . . 33
4.3.3 Type Alias ($\mathcal{A}$) . . . 33
4.3.4 Implementation Extensions ($\mathcal{X}$) . . . 34
4.4 Component Architecture . . . 34
4.4.1 Component ($\mathcal{C}$) . . . 35
4.4.2 Module ($\mathcal{M}$) . . . 35
4.5 Analyzer Configuration ($\mathcal{K}$) . . . 36

5 Analysis Pipeline Overview **40**
5.1 Phase 1: Syntactic Extraction (ε) . . . 40
5.2 Phase 2: Name Resolution (ρ) . . . 42
5.3 Phase 3: Graph Construction (γ) . . . 44
5.4 Pipeline and Architectural Guarantees . . . 45

6 Syntactic Extraction **47**
6.1 Overview . . . 47
6.2 The Entry Point: `extract_from_cst` . . . 47
6.3 CST Traversal: `walk_cst` . . . 48
6.3.1 Dispatch Priority Order . . . 50
6.3.2 Classification Heuristics . . . 50
6.4 Structural Parsers . . . 52
6.5 Low-Level Extractors . . . 56
6.6 Workspace Orchestration . . . 59
6.6.1 Module Grouping Strategies . . . 59
6.6.2 Out-of-Line Method Linking . . . 59

7 Name Resolution **61**
7.1 Overview . . . 61
7.2 The Query Algebra ($\mathcal{Q}$) . . . 63
7.3 Phase 2a: Query Building (ρ_{build}) . . . 64
7.3.1 The Symbol Stack (Δ) . . . 64
7.3.2 The Substitution Algorithm . . . 64
7.3.3 Variable and Parameter Binding . . . 65
7.3.4 Output of Phase 2a . . . 65
7.4 Phase 2b: Query Execution (ρ_{exec}) . . . 66
7.4.1 The Scope Tree ($\mathcal{E}$) . . . 66
7.4.2 The Execution Context (Γ) . . . 69
7.4.3 Lexical Scope Climbing (`Find`) . . . 69
7.4.4 Member Extraction and Inheritance (`Extract`) . . . 71
7.4.5 Call Query Evaluation (`Call`) . . . 73
7.5 Implementation Block Flattening . . . 79
7.6 The Primitive Registry ($\mathcal{P}$) . . . 80
7.7 Resolution Configuration ($\mathcal{K}$) . . . 81
7.8 Termination and Convergence Guarantees . . . 82

8 Graph Construction **84**
8.1 Overview . . . 84
8.2 Node Flattening . . . 84
8.2.1 Qualified Name Computation . . . 85
8.2.2 Entities Promoted to Vertices . . . 85
8.2.3 Dynamic Node Synthesis . . . 86
8.3 Edge Generation . . . 86
8.3.1 Type Reference Filtering and Referential Integrity . . . 87
8.3.2 Decomposition of Compound and Generic Targets . . . 87
8.3.3 Taxonomy of Dependency Edge Kinds . . . 88
8.3.4 Edge Deduplication . . . 91
8.4 Graph Representations and Visualization . . . 91

9 Evaluation and Results **96**
9.1 Research Questions . . . 96
9.2 Ground-Truth Validation . . . 98
9.3 Cross-Tool Structural Comparison . . . 99
9.3.1 Graph Harmonization Framework . . . 99
9.3.2 Comparative Graph Metrics . . . 102
9.3.3 Evaluation Dataset and Baseline Analyzers . . . 104
9.4 Runtime Performance Benchmarking . . . 105
9.5 Experimental Results . . . 106
9.5.1 RQ1: Ground-Truth Benchmarks Results . . . 106
9.5.2 RQ2: Cross-Tool Structural Alignment . . . 107
9.5.3 RQ3: Discrepancy Analysis Across Dependency Kinds . . . 112
9.5.4 RQ4: Runtime Scalability, Throughput, and Speedup . . . 114
9.6 Answers to Research Questions . . . 116

10 Discussion **120**
10.1 Empirical Findings Across Paradigms . . . 120
10.1.1 Object-Oriented Parity in Java . . . 121
10.1.2 Resolution Limits in Dynamic Languages . . . 121
10.1.3 Preprocessing and Headers in C and C++ . . . 123
10.2 Efficacy of the Language-Agnostic Approach . . . 126
10.2.1 Decoupling from Build Systems . . . 126
10.2.2 Precision Versus Scalability . . . 126
10.2.3 Polyglot Extensibility . . . 127
10.3 Practical Implications . . . 127
10.4 Threats to Validity . . . 128

11 Conclusions and Future Work **131**
11.1 Conclusions . . . 131
11.2 Future Work . . . 132

List of Figures **137**

List of Tables **139**

Bibliography and Sitography **140**

A Mathematical Notation and Glossary **144**
A.1 Notational Conventions 144
A.2 Glossary of Types and Domains 145
A.2.1 Base Types, Paths, and Identifiers 145
A.2.2 Structural Members and Signatures 145
A.2.3 Architectural Entities 146
A.2.4 Resolution Model and Scoping Contexts 146
A.2.5 Analysis Pipeline Phases 147

B Source Code and Reproducibility **148**
B.1 Repository Availability 148
B.2 Installation for End Users 148
B.3 Command Line Interface 149
B.4 Building from Source 150
B.5 Testing 150
B.6 Benchmarking Pipeline 150

Abstract

Static analysis of source code for dependency graph extraction traditionally requires implementations that are tightly coupled with the grammar of a specific programming language. This thesis project aims to define and formalize an analysis process capable of operating in a language-agnostic manner. Starting from Concrete Syntax Trees (CSTs) generated by generic parsers (such as Tree-sitter), this work formalizes the transformation of syntactic nodes into an abstract Component Tree and rigorously describes the rules for subsequent dependency resolution.

The correctness and consistency of the model are demonstrated by mapping the analysis functions to a formal Type System. Each phase of the process is modeled as a composition of strictly typed functions. Through the principles of functional semantics and polymorphism, the study demonstrates how extraction and resolution functions can handle constructs from different languages while ensuring Type Safety, termination, and the overall semantic coherence of the analysis pipeline.

Chapter 1

Introduction

1.1 Context and Motivation

Modern software development is an inherently complex and continuous process. Unlike traditional engineering disciplines, a software system is rarely ever "finished"; its lifecycle demands constant evolution to adapt to changing business requirements, new technologies, and expanding user needs. If a codebase cannot seamlessly accommodate change, it rapidly becomes obsolete [1]. However, this relentless pressure to evolve often leads to hasty implementation choices and poorly planned designs. Over time, these design flaws accumulate as **technical debt** [2],[3] eventually transforming the software into an unmaintainable monolith where the cost of adding new features or fixing bugs outweighs the cost of rewriting the system from scratch.

The root cause of such software decay is frequently architectural rather than localized. While bad programming practices at the method level (often referred to as *Code Smells* [4], [5]) are easily identifiable and correctable, poorly designed structural dependencies (*Architectural Smells* [6]) are much more insidious [7]. When components are highly coupled or burdened with cyclic dependencies, changing one module could break others in unpredictable ways. To prevent this architectural degradation, developers increasingly rely on automated static analysis tools. These systems scan the codebase to map out dependencies between functions, classes, and packages, allowing teams to evaluate structural quality, plan safe refactoring operations, and enforce architectural constraints before the damage becomes irreversible.

Despite the critical importance of these evaluations, a significant limitation plagues the current landscape of static analysis tools: they are typically **tightly bound to a single programming language** or a specific compiler ecosystem. Modern enterprise architectures, however, are increasingly polyglot. Building architectural analyzers for multiple languages currently forces engineers to rewrite the complex parsing and dependency extraction logic from scratch for every single language, resulting in immense resource consumption and fragmented, heavyweight, difficult to maintain tooling.

In recent years, the adoption of generic parsers based on unified formal grammars, such as Tree-sitter [8], has simplified the initial phase of the process. Tree-sitter can generate a Concrete Syntax Tree (CST) for almost any modern

language. However, the CST is a low-level data structure that models the exact syntax of the text (including parentheses, keywords, etc.) but lacks strong semantics. This work is based on the observation that, despite the syntactical differences between various programming languages, **the generated CSTs exhibit remarkable structural similarities**. It is therefore possible to exploit these commonalities to make the analysis process inherently language-agnostic.

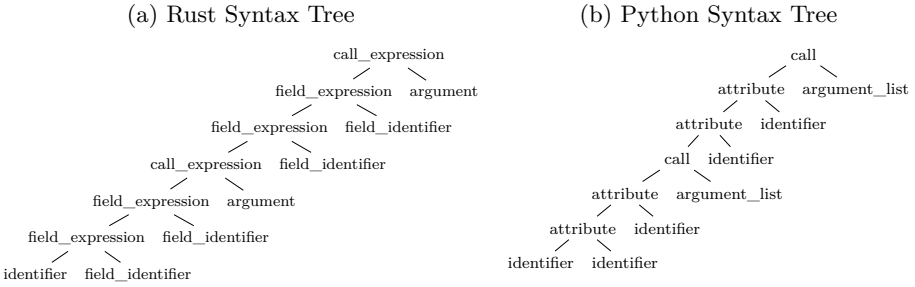


Figure 1.1: Rust and Python CST comparison from the code *a.b.c().d.e()*

1.2 Project Objectives

The primary objective of this thesis work is to design, formalize, and implement an architectural dependency analysis process that operates in an inherently *language-agnostic* manner.

Traditionally, developing cross-language analysis systems requires integrating disjointed tooling ecosystems, which significantly increases maintenance complexity. This project aims to bypass these limitations by processing the source code of heterogeneous languages through a single unified pipeline by transforming it to a standardized Intermediate Representation, to generate a the final **Dependency Graph**.

The nodes of this graph transcend simple text snippets, representing structured logical entities (e.g., Classes, Structs, Interfaces, Methods, and Modules). Concurrently, the edges represent the semantic relationships and interactions between them (e.g., Inheritance, Composition, Behavioral Invocations).

To achieve this goal without rebuilding a full compiler front-end, we deliberately scope out the task of performing logical validity checks (such as variable visibility, mutability, or strict type-checking). Instead, the system relies on the foundational **assumption that the input source code is already compiling and semantically valid** within its own ecosystem. This allows the analyzer to focus entirely on structural extraction rather than syntax validation.

1.3 Methodological Approach

To ensure that the system is extensible, structurally robust, and entirely independent of the syntactic peculiarities of individual languages, the project adopts an approach based on **algebraic data types** [9] and **functional composition**.

1.3.1 Type System and Intermediate Representation

The core of the system is the definition of a strongly typed Intermediate Representation (IR). The IR acts as an internal *universal language*. Every construct extracted from the code (whether a function, a class, or a simple parameter) is converted into an abstract IR component. This structural normalization makes it possible to entirely "forget" how an entity was represented by the specific programming language in the original file, allowing the subsequent phases to **operate** exclusively **on standardized architectural properties**.

1.3.2 Pipeline as Function Composition

The entire analysis process is modeled mathematically as a sequence of transformations, effectively a pipeline of strongly typed functions. Rather than relying on a monolithic, stateful procedure, the system is strictly decoupled into three distinct phases:

1. **Extraction**: Maps the raw CST nodes generated by Tree-sitter into the unified types of our IR. This is the only phase that "knows" the syntactic rules of individual languages, utilizing a heuristic approach to recognize common architectural patterns.

2. **Resolution**: A purely logical and semantic step that takes the extracted identifiers (e.g., the usage of a type in a method signature) and unambiguously connects them to their concrete definitions. This is achieved through a formal algebraic query engine and hierarchical lexical scoping rules.

3. **Graph Construction**: Transforms the resolved, object-oriented tree structure into the final directed dependency graph.

Figure 1.2 illustrates this three-stage architecture, highlighting how the structural properties evolve from raw source code to the final dependency graph.



Figure 1.2: The functional pipeline of the analyzer.

Modeling the system as a functional pipeline allows us to reason about its structural correctness: each phase receives a well-formed input and guarantees the production of an equally valid output (*Type Safety*). This separation of concerns ensures the overall semantic consistency of the architecture. In practice, adding support for a new language requires implementing only the extraction heuristics for Phase 1, without modifying the resolution engine, provided that the new language's semantic features (e.g., module strategies, self-parameter conventions, import semantics) are already covered by the existing configuration parameters. If the new language introduces novel semantic patterns not yet modeled by the configuration, limited extensions to the resolution engine may be necessary[10].

Scope

It is important to emphasize that this tool is designed exclusively for **static** architectural analysis. The extraction and resolution of dependencies are based solely on the source code as it is written. Consequently, the analyzer does not track or resolve dynamic behaviors that occur at runtime, such as reflection, advanced duck-typing, or dynamic dispatch mechanisms. The graph represents the architecture explicitly declared by the developer, offering a deterministic view of the codebase's structural dependencies.

1.3.3 Source Code Availability and Reproducibility

The *OmniDeps* framework and the *Graph-Synchron* benchmarking pipeline developed during this thesis are entirely open-source. A complete reproducibility package containing the source code of both tools, the snapshots of the 156 evaluated repositories, the extracted dependency graphs, and the comparative evaluation reports is permanently archived on Zenodo at `https://doi.org/10.5281/zenodo.23315197`. For instructions on how to access the repositories, download the standalone executables, build the project from source, or replicate the experimental benchmarks, please refer to Appendix B.

1.4 Organization of the Thesis

The remainder of this thesis is organized as follows:

- **Chapter 2** reviews the foundational concepts of software dependencies, technical debt, architectural smells, graph-based architectural modeling, parsing technologies, and lexical scoping.
- **Chapter 3** examines the multi-language analysis challenge, surveying traditional compiler front-ends and static analysis tooling, language-agnostic syntax models, reference resolution graphs, and direct precursors to this work.
- **Chapter 4** formally defines the language-agnostic Intermediate Representation (IR) that serves as the central data model throughout the pipeline.
- **Chapter 5** presents an overview of the three-phase analysis pipeline, describing the input, output, and key properties of each phase.
- **Chapter 6** details Phase 1 (Syntactic Extraction): the heuristic-based classification and parsing of Concrete Syntax Trees into the IR.
- **Chapter 7** details Phase 2 (Name Resolution): the query algebra, Scope Tree construction, and lexical scope climbing algorithm.
- **Chapter 8** details Phase 3 (Graph Construction): the flattening of the resolved IR into a dependency graph and its visualization.
- **Chapter 9** presents the formal evaluation methodology and empirical results across multiple benchmark projects and programming languages.

- **Chapter 10** discusses the empirical findings, trade-offs, architectural efficacy, practical implications, and threats to validity.
- **Chapter 11.1** draws conclusions and outlines potential directions for future work.

Additionally, two appendices are provided to assist the reader and ensure reproducibility:

- **Appendix A** provides a comprehensive glossary of the mathematical symbols and notation used to formalize the Intermediate Representation and the algebraic query engine.
- **Appendix B** details the open-source availability of the *OmniDeps* framework and its Zenodo reproducibility package (`https://doi.org/10.5281/zenodo.23315197`). It provides the repository links and exhaustive instructions for end-users and developers to download the executables, build the project from source, and replicate the experimental benchmarks.

Chapter 2

Background

This chapter introduces the fundamental concepts and technologies that form the basis of the architectural analysis process developed in this thesis. It begins by defining **software dependencies** and their formal representation through **graphs**, the accumulation of **technical debt** and **architectural smells**. It then details parsing mechanisms, syntax trees, and the **Tree-sitter** framework [8], and concludes with the formal principles of **lexical scoping** and **name resolution**.

2.1 Software Dependencies

In software engineering, a **dependency** exists when a component requires the presence or functionality of another component to perform its task or to compile correctly [11]. Managing these dependencies is arguably one of the most critical aspects of software design. It is directly related to the fundamental concepts of **coupling** (the degree of interdependence between software modules) and **cohesion** (the degree to which elements inside a module belong together) [12]. A well-architected system typically aims for *high cohesion* and *low coupling*, ensuring that components can be maintained, tested, and extended in isolation without triggering cascading changes across the entire codebase.

Dependencies in modern programming languages can manifest in several distinct forms, each carrying a different weight and architectural implication. They can be broadly categorized as shown in Table 2.1.

Furthermore, dependency tracking must distinguish between **static** and **dynamic** analysis. **Static analysis** examines the source code strictly at rest, extracting only the explicit relationships defined by the syntax and the type system. Conversely, **dynamic dependencies**, such as those introduced by reflection, dependency injection containers, late binding (polymorphism), or features like Python's `eval()`, manifest their true target only during program execution. Because static analysis cannot reliably predict the runtime state, it focuses exclusively on the declared, compile-time architecture. This thesis is strictly scoped to the extraction and modeling of *static* dependencies.

Category	Examples and Architectural Implications
Structural / Compositional	**Aggregation & Composition:** A type declares a field/property of another type (e.g., `Car` has an `Engine`). **Type Aliases:** Defining a new name for an existing type (`typedef`, `using`). **Generic Bounds:** Restricting a generic type parameter (e.g., `T extends B`). *Implication:* Alters the physical memory layout and fundamentally ties the data structures together.
Inheritance / Realization	**Class Extension:** A class derives from a base class. **Interface Implementation:** A class fulfills an interface contract. **Trait Mixins:** Including behavior from traits (e.g., in Rust or Scala). *Implication:* The strongest form of coupling (*IS-A* relationship). The child is inextricably tied to the parent's lifecycle and public contract.
Behavioral / Control Flow	**Method Invocations:** Calling a function or method. **Instantiations:** Creating a new object (`new B()`). **Field Access:** Reading or writing to another class's properties. **Signature Types:** Using a type as a formal parameter or return type. **Exception Handling:** Catching a specific exception type. *Implication:* Dictates runtime behavior and creates strict compile-time bonds, without altering structural memory layouts.

Table 2.1: Taxonomy of software dependencies extracted via static analysis.

2.2 Technical Debt and Architectural Smells

As software systems grow in size and complexity, continuous evolution, tight deadlines, and expedient engineering decisions inevitably introduce structural compromises. In software engineering, this accumulation of suboptimal design choices is formalized through the metaphor of **Technical Debt** [2], [3]. Much like financial debt, technical debt is a cause of ongoing "interest" in the form of increased maintenance overhead, decreased productivity, and a higher propensity for regressions during change operations. If not tackled, it leads to progressive *architectural decay* (or architectural erosion), wherein the actual implementation of the system significantly diverges from its intended architecture.

To identify symptoms of quality degradation, Martin Fowler popularized the concept of a "smell" [4]. A smell is not an explicit functional error or bug; rather, it represents an observable, surface indication of an underlying weakness in the system's design or maintainability. In the literature, smells are broadly differentiated by their scope and architectural granularity:

- **Code Smells:** These are localized anomalies confined to fine-grained programming constructs, such as an individual method, variable, or class [5]. Typical examples include *Long Method*, *Large Class*, or *Long Parameter*

List. Because their scope is purely local, code smells can usually be detected through direct syntactic analysis (or Abstract Syntax Tree inspection) and can be remedied via standard, localized refactorings without impacting the overall architecture of the system.

- **Architectural Smells:** Unlike code smells, architectural smells [6] represent macroscopic design flaws that affect the global structure of the software. They arise from poorly managed dependencies, suboptimal module organization, or cross-cutting violations of architectural boundaries. Empirical investigations have demonstrated that architectural smells are largely independent of code-level smells [13]: a codebase may feature clean, well-designed classes while simultaneously suffering from severe architectural interlacement. Correcting an architectural smell requires a comprehensive understanding of the system and demands complex multi-component refactoring operations.

Among the various categories of architectural smells identified in the literature **Dependency Smells** represent a particularly critical class. These flaws originate specifically from improper, excessive, or circular dependency patterns between software components (such as cyclic couplings or architectural bottlenecks). Because dependency smells reside in the relationships and interaction patterns between components rather than inside localized source code blocks, they cannot be discovered through simple line-by-line or syntactic analysis. Systematically identifying, tracking, and mitigating these smells requires modeling the components and relational dependencies of a codebase through formal graph-based representations, which can accommodate both densely coupled subsystems and disconnected components.

2.3 Graph-Based Architectural Representation

To systematically analyze, navigate, and reason about software architecture and its associated smells, the extracted dependencies are typically modeled using **Graph Theory**. The analysis process often relies on distinguishing between a *Component Graph* and a *Dependency Graph*.

2.3.1 Component and Dependency Graphs

A **Component Graph** represents the hierarchical containment and physical organization of the components defined within the source code. It is defined as a directed graph:

$$G_c = (V, E_c)$$

where V is the set of vertices representing software components (e.g., a package, a class, or a method), and $E_c \subseteq V \times V$ is the set of directed edges representing **direct containment** relationships. An edge $(u, v) \in E_c$ indicates that component u is the immediate hierarchical parent of component v (e.g., a method defined inside a class).

A **Dependency Graph** extends the component graph by modeling the semantic interactions between these components. It is defined as a multigraph:

$$G = (V, E) \quad \text{with} \quad E = E_c \cup E_d$$

The set E_d contains all the logical dependencies extracted from the code. A dependency fundamentally exists whenever a piece of code (Component A) cannot function, compile, or execute without the presence of another (Component B). [14]

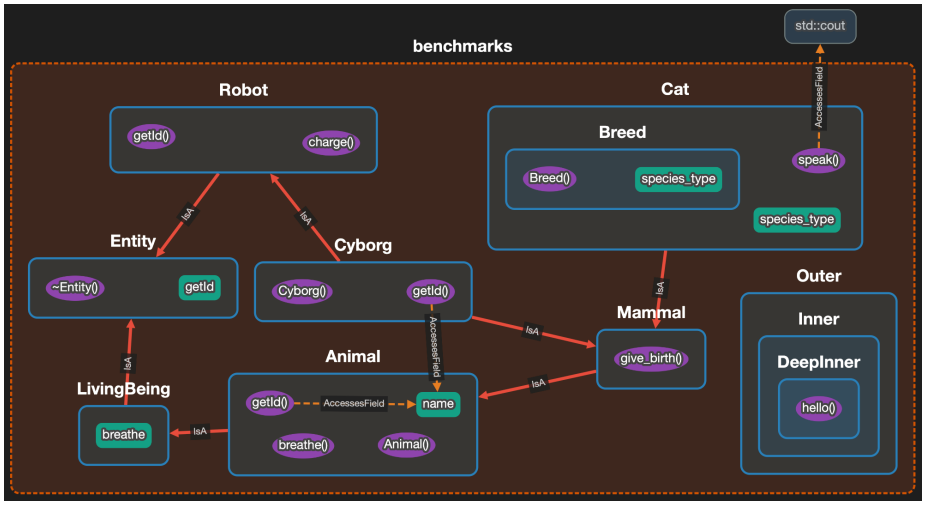


Figure 2.1: Small example of a Dependency Graph

In the context of source code, the **granularity** of these vertices can vary. They can represent high-level architectural boundaries, file-level modules, or fine-grained code constructs (such as classes, interfaces, or individual functions). The edges are intrinsically **directed**: if class `A` calls a method of class `B`, the edge points from `A` to `B`, establishing a dependency vector that indicates `A` is vulnerable to changes in `B`, but not necessarily vice versa. An illustrative example of a software dependency graph is provided in Figure 2.1.

2.3.2 Dependency Smells

Mapping source code to a graph structure provides a powerful mathematical foundation to evaluate the system's health and automatically detect specific Architectural Smells introduced in Section 2.2, commonly designated as **Dependency Smells** [7]. The presence of these configurations in the graph usually indicates a violation of core design principles (such as SOLID, high cohesion, or loose coupling). Empirical research has demonstrated that architectural smells are largely independent of code-level smells, thus requiring dedicated structural analysis [13]. Notable architectural dependency smells identified in the literature [6] and detected through dedicated architectural analysis tools such as Arcan [7] and DV8 [15] include:

- **Cyclic Dependency:** The most notorious anomaly occurs when two or more components are linked by circular dependency relationships, forming a cycle in the graph. This violates the Acyclic Dependencies Principle, rendering the involved components inseparable, impossible to test in isolation, and prone to infinite recursion. A healthy architectural dependency graph is generally expected to form a **Directed Acyclic Graph (DAG)**.

Consider the Java example in Listing 2.1, where three services (`ServiceA`, `ServiceB`, and `ServiceC`) depend on each other in a closed loop. This generates a cycle in the dependency graph, as visually represented in Figure 2.2.

```java
// In ServiceA.java
public class ServiceA {
    private ServiceB b = new ServiceB();
}

// In ServiceB.java
public class ServiceB {
    private ServiceC c = new ServiceC();
}

// In ServiceC.java
public class ServiceC {
    private ServiceA a = new ServiceA();
}
```

Listing 2.1: Example of a Cyclic Dependency involving three Java services.

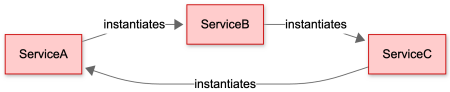


Figure 2.2: Dependency graph resulting from a Cyclic Dependency smell.

- **Unstable Dependency:** This occurs when a stable, core component depends on one or more highly unstable, frequently changing components. According to the metric formalization introduced by Martin [11], the instability $I \in [0, 1]$ of a component is quantified as the ratio of efferent coupling (C_e, outgoing dependencies) to total coupling:

$$I = \frac{C_e}{C_a + C_e}$$

  where C_a denotes afferent coupling (incoming dependencies). A component is considered *stable* when I approaches 0 (it is heavily depended upon and depends on few entities, making changes difficult and costly), whereas it is *unstable* when I approaches 1. Violating the Stable Dependencies Principle (SDP) by directing dependencies toward components with higher instability exposes critical architectural modules to frequent changes and cascading failures.

- **Hub-Like Dependency:** A component assumes a role of excessive centrality, exhibiting a very high number of both incoming and outgoing dependencies. Such "hub" components become critical bottlenecks, concentrating heterogeneous responsibilities and severely limiting the modularity and independent evolution of the system.

- **Deep Hierarchy:** An inheritance hierarchy that is excessively deep, where components at the bottom depend on a long chain of supertypes.

This increases system complexity and makes the behavior of child classes difficult to predict.

- **Wide Hierarchy:** An inheritance hierarchy that is excessively broad, with a single parent class having an unusually large number of direct subclasses. This often indicates poor intermediate abstraction or an incomplete decomposition of responsibilities.
- **Cyclic Hierarchy:** A critical structural error where an inheritance hierarchy forms a cycle (e.g., a supertype directly or indirectly inheriting from its own subtype). This breaks the fundamental *IS-A* relationship paradigm and can lead to severe compilation or runtime errors.

2.4 Syntax Parsing and Tree-sitter

Static analysis tools fundamentally rely on parsers to transform raw source text into structured syntactic models. In program analysis and compiler construction, these representations generally fall into two distinct paradigms [16], [17]:

- **Abstract Syntax Tree (AST):** Abstracts away non-semantic syntactic details, such as punctuation, delimiters, and whitespace, focusing exclusively on the computational logic (e.g., representing an addition as an operator node rather than preserving the literal `+` token). This structure is widely used in compiler back-ends and intermediate optimization passes.
- **Concrete Syntax Tree (CST):** Represents the exact, unaltered syntactic structure of the source text, maintaining a strict one-to-one mapping with the original file. It includes all tokens, whitespace, and formatting, making it ideal for tooling that requires exact code correspondence, such as linters and code formatters. From the perspective of multi-language architectural analysis, operating directly on CSTs provides a substantial engineering advantage: it eliminates the need to build, maintain, and synchronize custom language-specific translators from parse trees to intermediate ASTs, drastically reducing the complexity of supporting new programming languages.

To achieve high-performance, language-agnostic parsing across heterogeneous codebases, the analysis pipeline developed in this thesis utilizes **Tree-sitter** [8]. Tree-sitter is a modern, generic parser generator designed to build fast, robust CSTs. It generates parsers from formal grammars written in a Domain Specific Language (DSL) based on JavaScript.

One of the most defining characteristics of Tree-sitter is its vibrant, community-driven ecosystem. Rather than relying on a centralized team to build and maintain parsers, the ecosystem is heavily supported by the open-source community. Today, there are robust, production-ready parsers available for virtually all production grade programming languages.

A fascinating side effect of this community-driven development is a phenomenon of *structural convergence*. When developers write a Tree-sitter grammar for a new language, they frequently draw inspiration from existing, well-established grammars (such as those for C, Java, or Python). This cross-pollination naturally leads to a shared structural "DNA" across different languages' syntax trees. Naming conventions for AST nodes, hierarchical nesting

rules, and overall structural layouts end up being surprisingly similar, even for languages with vastly different paradigms. As will be extensively discussed in Chapter 6, this emerging structural similarity is the fundamental enabler of our framework. It allows us to abstract away syntactic peculiarities and identify overarching architectural components in a completely language-agnostic manner.

Under the hood, the generated parsers are implemented in C to guarantee maximum performance and portability. Because they adhere to the standard C Application Binary Interface (ABI), Tree-sitter exposes a uniform API and provides native bindings for numerous high-level languages, including Rust, Go, Java, and Python.

Parsing and Navigation

Using Tree-sitter within a Rust ecosystem requires initializing a `Parser` instance and binding it to a specific language grammar (such as `tree-sitter-rust` or `tree-sitter-java`). Once configured, the parser consumes the raw source code string and produces the Concrete Syntax Tree (CST).

Unlike traditional parsers that generate heavyweight, deeply nested object hierarchies in memory, Tree-sitter's CST is backed by an efficient C structure. To interact with it, static analyzers must navigate this tree. Doing so via standard recursive object visits could trigger stack overflows or incur severe heap allocation overhead on massive codebases. To solve this, Tree-sitter provides the **TreeCursor**.

The `TreeCursor` allows stateful, allocation-free traversal of the syntax tree using imperative movements. Because the cursor mutates its internal state *in place* rather than allocating new node objects at each step, it guarantees zero-allocation iteration over arbitrarily large trees. Table 2.2 summarizes the primary methods exposed by the cursor.

Cursor Method	Description	Returns
`goto_first_child()`	Moves the cursor down to the first child of the current node.	`bool`
`goto_next_sibling()`	Moves the cursor right to the next sibling of the current node.	`bool`
`goto_parent()`	Moves the cursor up to the parent of the current node.	`bool`
`node()`	Retrieves the `Node` object currently pointed to by the cursor.	`Node`
`field_name()`	Returns the field name associated with the current node, if any.	`Option<&str>`

Table 2.2: Key methods provided by the `TreeCursor` API for stateful CST navigation.

This uniform access mechanism is completely agnostic to the underlying programming language. In our framework, the extraction phase relies heavily on combining `goto_first_child()` and `goto_next_sibling()` within loops to systematically crawl through all components of a file. At any position, the cursor exposes the current `Node`, allowing the analyzer to extract structural metadata (via `node.kind()`) or the raw source string (via `node.utf8_text()`).

Listing 2.2 illustrates the complete pipeline: from parser initialization to

the stateful traversal of the resulting tree, mimicking the pattern used heavily during the analysis phase of our prototype.

```rust
use tree_sitter::Parser;

// 1. Initialize the parser and set the target language
let mut parser = Parser::new();
parser.set_language(tree_sitter_java::language())
      .expect("Error loading Java grammar");

// 2. Parse the source code into a Concrete Syntax Tree
let source_code = b"class Main { int x; }";
let tree = parser.parse(source_code, None).unwrap();

// 3. Initialize the TreeCursor at the root node
let mut cursor = tree.root_node().walk();

// 4. Navigate the tree statefully without allocating memory
if cursor.goto_first_child() {
    loop {
        let node = cursor.node();
        println!("Node kind: {}", node.kind());

        if let Ok(text) = node.utf8_text(source_code) {
            println!("Text: {}", text);
        }

        // Move to the next sibling, or break if there are no more
        if !cursor.goto_next_sibling() {
            break;
        }
    }
    // Return to the parent scope
    cursor.goto_parent();
}
```

Listing 2.2: Complete example of parsing source code and navigating the CST using Tree-sitter in Rust.

Advantages over Traditional Generators

Tree-sitter was selected over traditional parser generators (like ANTLR, Bison, or Yacc) [18] for several architectural reasons:

- **Performance and Algorithm:** Tree-sitter uses a **Generalized LR (GLR)** parsing algorithm. Being a bottom-up parser, it is generally faster and more efficient than top-down parsers like ANTLR's Adaptive LL(*). The time complexity for parsing is primarily bounded to $O(n)$, making it highly suitable for analyzing massive codebases.
- **Advanced Error Recovery:** When encountering a syntax error, Tree-sitter does not halt. Instead, it utilizes `ERROR` or `MISSING` nodes to bypass the anomaly and successfully parse the remainder of the file. This makes it incredibly robust for analyzing code under development.
- **Decoupling of Grammar and Code:** Tools like Bison and Yacc often require developers to mix executable C/C++ code directly into the grammar definitions (Syntax-Directed Translation). While powerful for

compiling, this makes the grammar practically impossible to port to other languages. Conversely, Tree-sitter enforces a strict separation: the grammar only defines syntax, while the application (in our case, the Rust analyzer) queries the resulting tree.

2.5 Name Resolution and Lexical Scoping

Constructing an accurate architectural dependency graph requires bridging the gap between raw syntactic identifiers and their concrete semantic declarations. This section examines the mechanisms of lexical scoping and the role of programming language type systems in static name resolution.

Lexical Scoping and Scope Climbing

Extracting the syntax tree is only the first step in constructing a dependency graph. The subsequent, and often significantly more complex challenge, is **name resolution** (or symbol resolution). Knowing syntactically that a class uses an identifier named `B` is insufficient; the analyzer must determine *exactly which concrete type* `B` refers to within the global project scope, distinguishing it from other types that might share the same name in different modules.

This resolution process depends heavily on **lexical scoping** rules, which dictate the visibility and lifetime of identifiers based on their physical placement within the source code blocks [17]. Scopes are typically hierarchical: an inner scope (like a method body) can access identifiers from its outer scope (like the class body or the global namespace). Complicating matters is the concept of **shadowing**, where a local variable might mask a global type with the same name.

To correctly resolve a reference, static analyzers typically simulate a bottom-up search (or "scope climbing"). As illustrated in the Java snippet in Listing 2.3 and its corresponding diagram in Figure 2.3, when the analyzer encounters the identifier `value` inside the `if` block, it first checks the immediate block scope. Not finding it, it climbs up to the method scope, and finally to the class scope, where the declaration is matched.

```java
public class Outer {
    int value; // Declaration matched at Class Scope
    void method() {
        if (true) {
            System.out.println(value); // Reference
        }
    }
}
```

Listing 2.3: A simple Java class illustrating nested scopes.

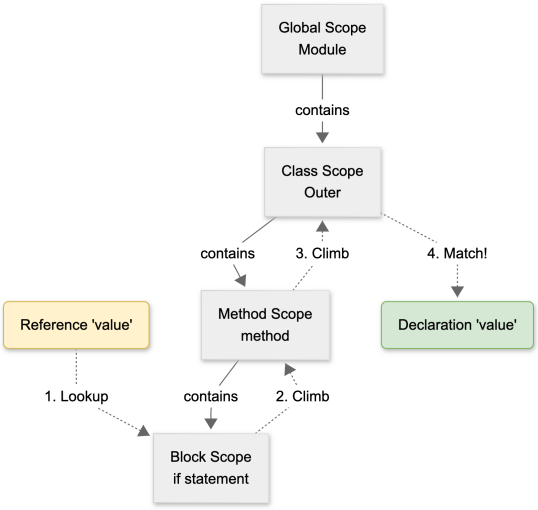


Figure 2.3: Visual representation of the scope climbing algorithm: the search starts from the inner-most block and climbs up the hierarchy until the identifier is found.

Furthermore, different programming languages handle namespaces, imports, and symbol resolution in drastically different ways. Because types can reference each other cyclically or be referenced before they are declared (forward declarations), resolving symbols in a **single linear pass** over the code is generally impossible.

Consequently, structural extraction must be strictly **decoupled** from semantic resolution. The syntactic phase (Extraction) must first traverse the CST to catalog all available declarations and record unresolved identifiers along with the exact context and scope in which they appear. Only afterward can a dedicated, **multi-pass resolution phase** traverse the hierarchical scope boundaries to semantically link these identifiers to their concrete definitions. This separation of concerns is a central architectural decision in designing a scalable, language-agnostic analyzer.

Type Systems and Type Hints

The complexity of name resolution is heavily influenced by the **Type System** of the underlying language [9]. In statically typed languages (such as Java, C++, and Rust), the type of every variable, field, and method return is explicitly declared (or rigorously inferred at compile time). This makes static resolution deterministic: if a variable `v` is declared as type `A`, any method call `v.foo()` can be reliably resolved by looking up the definition of `foo` within class `A`.

Conversely, dynamically typed languages (such as Python) utilize **duck typing**, where variables do not possess inherent types, and their types can change freely at runtime based on the assigned values. In such environments, determining the target of `v.foo()` using only static text analysis is fundamentally ambiguous without executing the code.

To bridge this gap and enable robust static analysis in dynamically typed environments, modern ecosystems have introduced optional static typing mech-

anisms, such as **Type Hints** (e.g., Python's PEP 484 [19]). Type hints allow developers to annotate variables and function signatures with expected types without altering the runtime behavior.

Consider the Python example in Listing 2.4. Without annotations, a static analyzer cannot determine the type of the `user` parameter, making it impossible to resolve the target of the `user.send()` method call. By providing type hints, the parameter is explicitly linked to the `User` class, allowing deterministic resolution.

```python
# Without type hints: 'user' is opaque to static analysis
def notify(user):
    user.send("Hello") # What is 'user'? Impossible to resolve statically.

# With type hints:
from models import User

def notify(user: User) -> bool:
    user.send("Hello") # Can be deterministically resolved to User.send
    return True
```

Listing 2.4: Comparison of a Python function without and with Type Hints (PEP 484).

For the purposes of this thesis, extracting architectural dependencies from dynamically typed languages relies on a crucial **assumption**: we assume that the source code is adequately decorated with Type Hints. When these annotations are present, the analyzer can treat them as pseudo-static declarations, mapping them to the internal IR and performing deterministic name resolution identically to statically typed languages. Without these hints, static resolution falls back to heuristics or yields unresolved edges, highlighting the inherent limits of analyzing dynamic languages statically.

Chapter 3

Related Work

This chapter provides a comprehensive analysis of previous research and state-of-the-art technologies addressing multi-language static code analysis, structural extraction, and name resolution. The analysis is structured to evaluate the most recent and effective techniques, critically examining their architectural choices, underlying mechanisms, and current limitations. Understanding these baseline approaches is essential for positioning the framework proposed in this thesis, highlighting how it overcomes existing bottlenecks in building language-agnostic dependency graphs.

The chapter is structured into five main sections: multi-language analysis and tooling, language-agnostic syntactic representations, graph models for reference resolution, direct precursor prototypes developed within our research laboratory, and finally, the architectural positioning of this thesis.

3.1 Multi-Language Analysis and Tooling

Extracting structural and architectural information from source code requires robust static analysis mechanisms. Traditionally, this process relies heavily on **compiler front-ends**, such as Clang for C/C++ or Roslyn for C# [20]. A compiler front-end typically performs:

1. **Lexical Analysis (Tokenization):** Converting text into a stream of tokens.
2. **Syntactic Analysis (Parsing):** Grouping tokens into grammatical structures (syntax trees).
3. **Semantic Analysis:** Type checking and symbol resolution.

Because these tools must generate executable code, they possess a mathematically sound and exhaustive understanding of the language's semantics. However, this deep integration is also their primary *limitation*: they are inherently **coupled to a single programming language**, its specific version, and its build ecosystem.

3.1.1 Compiler Front-Ends and the $M \times N$ Problem

Developing a cross-language analysis system using language-specific tools forces engineers to integrate various compiler front-ends, each with its own output format, terminology, and execution environment. If the system must support M source languages and N analysis tools (or back-ends), each tool must be individually integrated with each language's front-end, resulting in $M \times N$ separate integration efforts. As both M and N grow, this combinatorial complexity becomes unsustainable.

Modern compiler architectures, such as LLVM [20], solve this problem by introducing an **Intermediate Representation (IR)**: a standardized, language-agnostic data structure that sits between the front-end (which parses the source code) and the back-end (which performs analysis or generates machine code). Each language requires only one front-end that translates source code into the IR, and each tool requires only one back-end that consumes the IR. The total number of components is thus reduced from $M \times N$ to $M + N$. This architectural transition is visually illustrated in Figure 3.1.

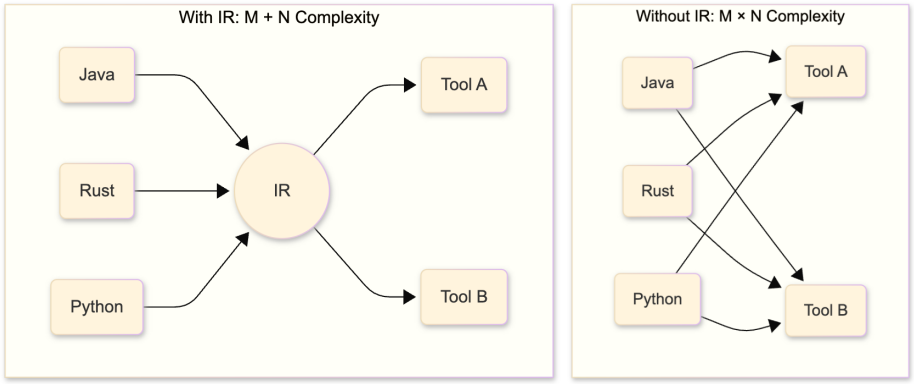


Figure 3.1: Comparison between the $M \times N$ combinatorial explosion and the linear $M + N$ complexity achieved by introducing an IR.

While LLVM uses a low-level IR focused on executable instructions, architectural analysis requires a high-level, structural representation. The same $M \times N$ decoupling principle applies to this domain in two concrete ways:

- **Internal decoupling:** Even within a single analysis tool, the pipeline contains multiple language-agnostic phases (name resolution, graph construction, export, visualization). Without an IR, each of these phases would need to handle the syntactic details of every supported language. With a shared IR, only the extraction phase is language-specific; all subsequent phases operate on a single, unified data model.
- **Ecosystem reuse:** A well-defined architectural IR can be consumed by multiple independent tools (e.g., architectural smell detectors, refactoring assistants, dependency auditors) without each tool reimplementing its own extraction logic for each language.

Designing a universal IR that captures the architectural semantics of heterogeneous languages, without descending into low-level memory layouts, is the

core theoretical foundation required to build a truly scalable, language-agnostic dependency extractor.

Language Server Protocol (LSP)

In recent years, protocols like the **Language Server Protocol (LSP)** have attempted to standardize how development environments interact with language-specific analyzers [21]. By standardizing the communication JSON-RPC interface, an IDE can interact with any language server to request symbol definitions or references. However, LSP is fundamentally designed for **interactive, query-based use cases** (e.g., hovering over a variable to see its type, or requesting "find references" for a specific symbol within an open file). Constructing a global dependency graph of an entire repository requires a **batch-processing approach**. Attempting to use LSP for this purpose would involve artificially spoofing thousands of "find references" queries across every symbol in every file, a process that is highly stateful, incredibly slow, and simply suboptimal for full-scale architectural extraction.

3.1.2 Code Property Graph: Joern

Beyond interactive protocols like LSP, practical multi-language static analysis often relies on frameworks that transform entire codebases into queryable graph representations. In vulnerability discovery and security research, this is typically achieved through fine-grained semantic graphs that model instruction-level control and data flows across different programming languages.

A well-known open-source platform in this domain is **Joern** [22], originally developed by Yamaguchi et al. for multi-language static analysis and vulnerability discovery [23].

How it works: Joern generates a *Code Property Graph (CPG)*, a data structure that unifies three representations: the AST, the Control Flow Graph (CFG, which models statement execution order and conditional branching), and the Program Dependence Graph (PDG, which tracks data and control dependencies between operations) into a single, queryable graph database. Analysts can query this graph using a specialized Scala-based Domain Specific Language (Ocular/Joern dialect) to track data flow and identify security flaws.

Weaknesses and Limitations: Constructing a full Code Property Graph incurs substantial computational overhead, as extracting CFGs and PDGs requires modeling fine-grained runtime execution semantics. While this instruction-level granularity is essential for vulnerability discovery, it is unnecessarily heavy for high-level architectural analysis. Furthermore, Joern relies on separate, heavyweight compiler front-ends (such as Eclipse JDT for Java or Eclipse CDT for C/C++) rather than a unified parsing pipeline. As a result, supporting additional languages requires integrating dedicated compiler tools, reproducing the front-end coupling of the $M \times N$ problem.

3.1.3 Architectural Analyzers: Arcan and Depends

While platforms like Joern operate at the instruction level, architectural analysis works at a higher level, extracting component and dependency graphs to evaluate structural health, modularity, and technical debt. Two widely used

tools in this domain, which also serve as experimental baselines in this thesis, are **Arcan** and **Depends**.

Arcan [7] is a static analysis tool developed by Fontana et al. dedicated to the automatic detection of architectural smells and the monitoring of architectural decay across software releases [24]. Similar component and dependency graphs are used in industrial architectural suites (such as SonarQube or Structure101) to monitor modularity, but Arcan uniquely formalizes the automated detection of architectural smells and architectural technical debt.

How it works: Arcan analyzes software projects (primarily Java and C/C++) by extracting multi-layered dependency graphs that capture structural relations among packages, classes, and methods. On top of these graphs, Arcan executes graph-based algorithms and computes structural and topological metrics to detect critical dependency smells, such as Cyclic Dependency, Hub-Like Dependency, and Unstable Dependency. Furthermore, it tracks the introduction, persistence, and refactoring of these smells over commit histories and release cycles, providing quantifiable indicators of architectural technical debt.

Role and Relevance to this Work: Within our research laboratory, Arcan represents the primary reference baseline for architectural analysis; both precursor prototypes to this thesis, Skullian [25] and Entangle [26], evaluated their dependency extraction accuracy and graph similarity against Arcan's output. However, Arcan's extraction pipeline relies on language-specific frontends and bytecode analyzers (such as Spoon for Java or compiler-based parsers for C/C++). Adding support for emerging or polyglot software systems (e.g., in Rust, Python, or Go) requires engineering dedicated frontends tailored to each language's compiler ecosystem. This limitation directly motivates the research in this thesis: designing a lightweight, uniform, and language-agnostic extraction engine capable of producing faithful dependency graphs across heterogeneous languages, serving as an effective frontend for architectural analysis tools like Arcan.

Depends [27] is an open-source, multi-language static analysis framework specifically designed to extract fine-grained syntactical dependencies among code entities (files, types, and methods). Developed to support architecture recovery, code visualization, and modularity evaluation,

How it works: Depends relies on language-specific parsers generated via **ANTLR** grammars. For each supported language, an ANTLR-generated parser constructs an Abstract Syntax Tree (AST), which is subsequently visited to infer explicit syntactic relationships such as inheritance (`extend`, `implement`), behavioral interactions (`call`, `create`), type declarations (`parameter`, `return`), and imports. The extracted dependencies are then aggregated into a unified dependency matrix and exported into standard interchange formats, such as JSON or Graphviz DOT.

Role and Relevance to this Work: In this thesis, Depends represents a crucial state-of-the-art multi-language reference baseline. Alongside Arcan, Depends is directly utilized in Chapter 9 to benchmark and validate the accuracy, graph similarity, and extraction performance of Omnideps across heterogeneous, multi-language codebases.

Weaknesses and Limitations: Although Depends successfully supports multiple programming languages, its reliance on language-specific ANTLR grammars presents some practical limitations:

- **Grammar Maintenance Burden:** While both tools require language-

specific logic to recognize initial code constructs, Depends authors and maintains independent ANTLR grammars (`.g4`) for each supported language. In contrast, our pipeline leverages existing, widely adopted Tree-sitter grammars and strictly isolates language-specific parsing to Phase 1, converting the syntax tree into a shared intermediate model before any resolution begins.

- **Parsing Algorithm Characteristics:** ANTLR utilizes a top-down $LL(*)$ parsing strategy, which can incur non-trivial memory and execution overhead on large source files compared to bottom-up GLR parsing algorithms. Conversely, Tree-sitter utilizes a bottom-up GLR parser implemented in native C that is inherently error-tolerant, producing a usable syntax tree even from malformed or incomplete source files.
- **Lack of a Formal Unified IR and Scope Model:** Depends maps AST nodes directly to dependency records without an intermediate, formal algebraic representation. As a result, name resolution and expression handling are implemented through ad-hoc logic within each language's AST visitor, rather than through a single, unified resolution engine. In contrast, our pipeline evaluates reference queries through a formal query algebra over a language-agnostic Scope Tree, ensuring consistent and mathematically defined resolution semantics across all supported languages.

3.2 Language-Agnostic Syntax Representations

A major challenge in cross-language static analysis is the profound heterogeneity of the Abstract Syntax Trees (ASTs) generated by different parsers. Analyzing multi-language software with a single unified backend therefore requires a common level of abstraction. In the literature, prior research has largely pursued this goal at the syntactic level by normalizing source code from different programming languages into unified tree representations, such as intermediate XML markup or universal abstract syntax trees.

Syntactic Wrapping: srcML

srcML [28] is an infrastructure and XML-based format designed to represent source code by wrapping the original raw text with syntactic XML tags.

How it works: Instead of transforming code into an abstract data structure that discards formatting, srcML parses the code and produces an XML document that preserves the exact original text, including whitespace and comments. It overlays this text with syntactic boundaries (e.g., `<class>`, `<function>`, `<expr>`). Analysts can then use standard XML querying tools, such as XPath, to extract facts and relationships from the codebase.

For example, given the simple C++ snippet in Listing 3.1, srcML translates it into the XML format shown in Listing 3.2, preserving the original tokens while exposing the syntax tree structure [29].

```
1  #include "rotate.h"
2
3  // rotate three values
4  void rotate(int& n1, int& n2, int& n3) {
```

```
  // copy original values
  int tn1 = n1, tn2 = n2, tn3 = n3;

  // move
  n1 = tn3;
  n2 = tn1;
  n3 = tn2;
}
```

Listing 3.1: Example of a simple C++ function to be analyzed by srcML.

```
<cpp:include>#<cpp:directive>include</cpp:directive> <cpp:file>"rotate.h"</
    cpp:file></cpp:include>

<comment type="line">// rotate three values</comment>
<function><type><name>void</name></type> <name>rotate</name><parameter_list>(<
    parameter><decl><type><name>int</name><modifier>&amp;</modifier></type> <
    name>n1</name></decl></parameter>, <parameter><decl><type><name>int</name>
    <modifier>&amp;</modifier></type> <name>n2</name></decl></parameter>, <
    parameter><decl><type><name>int</name><modifier>&amp;</modifier></type> <
    name>n3</name></decl></parameter>)</parameter_list> <block>{<block_content
    >

  <comment type="line">// copy original values</comment>
  <decl_stmt><decl><type><name>int</name></type> <name>tn1</name> <init>= <expr
      ><name>n1</name></expr></init></decl>, <decl><type ref="prev" /><name>
      tn2</name> <init>= <expr><name>n2</name></expr></init></decl>, <decl><
      type ref="prev" /><name>tn3</name> <init>= <expr><name>n3</name></expr><
      /init></decl>;</decl_stmt>

  <comment type="line">// move</comment>
  <expr_stmt><expr><name>n1</name> <operator>=</operator> <name>tn3</name></
      expr>;</expr_stmt>
  <expr_stmt><expr><name>n2</name> <operator>=</operator> <name>tn1</name></
      expr>;</expr_stmt>
  <expr_stmt><expr><name>n3</name> <operator>=</operator> <name>tn2</name></
      expr>;</expr_stmt>
</block_content>}</block></function>
```

Listing 3.2: The resulting srcML representation, keeping formatting intact but injecting syntactic tags.

Weaknesses and Limitations: While srcML is highly robust for lightweight fact extraction and refactoring tasks, it is essentially a syntactic markup rather than a semantic model. It does not provide built-in name resolution or cross-file dependency linking; it simply unifies the syntax. Furthermore, its parser ecosystem is currently restricted to a limited set of languages (C, C++, C#, and Java), and it does not support the full range of available programming languages, limiting the future extensibility of the analysis framework.

Relevance: Relevance: While srcML cannot be adopted in this work due to its current language restrictions, its underlying philosophy remains highly relevant. By mapping syntactic constructs to a common vocabulary of structural annotations across all supported languages, it provides a unified baseline for core program elements. Adopting such a cross-language unified annotation scheme could represent an interesting alternative for the extraction layer, significantly reducing the need for complex heuristics to recognize corresponding constructs across different parsing trees.

Universal ASTs: Babelfish

A widely explored direction in multi-language analysis is transforming source code into a unified, language-agnostic tree representation. A pioneer in this domain was **Babelfish** [30], an open-source framework that introduced the concept of the *Universal Abstract Syntax Tree (UAST)*. Babelfish aimed to parse any programming language into a standardized syntax tree annotated with language-independent semantic roles. In a similar vein, recent research by Curtis [31] and Schiewe et al. [32] focuses on generating language-agnostic ASTs to analyze heterogeneous, cloud-native microservices. In distributed systems, microservices frequently employ different programming languages, making traditional single-language analyzers insufficient for tracing cross-service architectural dependencies (e.g., a Python service invoking a Java endpoint).

How it works: To bridge language differences, these approaches introduce an explicit multi-stage translation pipeline. Language-specific parsers or drivers (often wrapping Tree-sitter or native compiler frontends) first produce language-dependent ASTs. Next, dedicated transformation drivers map syntax nodes into a serialized, language-agnostic representation, either an annotated UAST with normalized node types or a standardized intermediate format (frequently serialized in JSON or protocol buffers). Finally, generic component analyzers traverse this normalized tree to extract architectural abstractions (such as endpoints, controllers, and data models) regardless of the source language.

Weaknesses and Limitations: Although effective for coarse-grained syntactic classification, relying on a Universal AST presents several major limitations for deep architectural analysis. First, maintaining dedicated transformation drivers is costly: every supported grammar requires continuous manual engineering to map evolving language syntax into the universal schema. Second, full-tree normalization incurs significant computational and memory overhead, as entire codebases must be translated into statement-level and expression-level universal syntax trees, capturing low-level details (such as arithmetic operations or loop constructs) that are irrelevant for architectural dependencies. Third, attempting to unify diverse language syntaxes into a single syntactic tree is inherently lossy: collapsing language-specific constructs into generic node kinds discards the distinct scoping and visibility semantics required for faithful name resolution.

Relevance: Our work directly addresses these drawbacks by avoiding an instruction-level Universal AST entirely. Instead of normalizing all source syntax into a single monolithic tree, we perform selective, heuristic-based extraction directly on Tree-sitter Concrete Syntax Trees (CSTs). We extract only high-level architectural constructs (modules, types, signatures, and reference queries) into a lightweight Intermediate Representation ($\mathcal{D}_{IR}$), discarding intra-procedural statement syntax immediately after extraction. Crucially, rather than forcing heterogeneous scoping rules into a generic syntactic schema, our pipeline preserves language-specific resolution semantics through the declarative configuration $\mathcal{K}$ evaluated during Phase 2.

3.3 Graph Models for Reference Resolution

Once the structural components of a codebase are identified, the subsequent challenge is resolving identifiers (e.g., variable usages, method calls, and type references) to their concrete definitions across complex, nested scopes. This section evaluates declarative and automaton-based graph models developed in the literature for reference resolution.

Path-Based Resolution: Scope Graphs

Scope Graphs [33], developed by researchers at TU Delft, provide a foundational, language-parametric theoretical framework for name binding and resolution.

How it works: The model abstracts away the AST entirely, representing the program's lexical structure as a graph. Lexical scopes are represented as nodes, and visibility or reachability rules (such as lexical nesting or module imports) are represented as directed edges. Resolving a name becomes a formal path-finding problem within this graph, governed by specific regular expressions that dictate valid resolution paths.

Weaknesses and Limitations: The main weakness of standard Scope Graphs is their lack of scalability. The model typically requires a whole-program analysis, meaning the entire project must be parsed and translated into a single, massive graph in memory before resolution can begin. They struggle with incremental updates, making them computationally prohibitive for very large codebases.

Pushdown Navigation: Stack Graphs

Stack Graphs [34] are the direct successor to Scope Graphs, developed and heavily utilized by GitHub to provide code navigation (e.g., "jump to definition") at an industrial scale.

How it works: They extend Scope Graphs by introducing a "Symbol Stack" and two new node types: *push* and *pop* nodes. This crucial addition allows the graph to be built incrementally, file by file. When a reference is queried, the algorithm walks the graph, pushing identifiers onto the stack at *push* nodes and popping them at *pop* nodes. A path is considered valid, and the reference resolved, if the symbol stack is completely empty when a definition node is reached.

Weaknesses and Limitations: While theoretically elegant and excellent for localized IDE queries, Stack Graphs possess significant practical limitations when repurposed for total dependency extraction. First, constructing a Stack Graph requires the creation of highly complex and verbose grammars (using the Tree-sitter Graph or TSG language) for every single language. Second, because resolution in Stack Graphs relies on a pushdown automaton traversing graph edges, finding all dependencies requires executing this graph-walking algorithm from every single reference. In highly interconnected codebases with deep import chains, this mechanism suffers from a *path-explosion* problem: the algorithm evaluates thousands of combinatorial paths simultaneously to verify stack emptiness, leading to severe memory and performance bottlenecks. Conversely, our approach avoids this combinatorial explosion by replacing unguided

graph traversal with deterministic algebraic lookups on a Scope Tree.

3.4 Direct Precursors

This thesis represents the culmination and third iteration of a continuous research effort initiated by Francesco Refolli and later advanced by Luca Teruzzi, building upon two previous prototypes which laid the critical foundation for the current solution.

3.4.1 Stack-Graph Resolution: Skullian

Skullian [25] was the initial prototype of this research line, developed by Dr. Francesco Refolli (who currently serves as co-advisor for this thesis). It attempted purely language-agnostic dependency extraction by combining Tree-sitter for parsing with Stack Graphs for name resolution.

How it works: It parsed the source code with Tree-sitter and used custom TSG rules to map the generated CST into a Stack Graph. It then used the built-in Stack Graph path-finding algorithm to resolve all references and build a dependency graph.

Weaknesses and Limitations: The evaluation of Skullian exposed practical and scalability limits of Stack Graphs for architectural-level dependency extraction.

- **Grammar Complexity**: Writing the language-specific TSG rules were highly verbose and complex, requiring thousands of lines of configuration for a single language like Java. This made the system difficult to maintain and extend.
- **Path Explosion and Cycle Detection Issues**: The Stack Graph size grew very big even for modest projects, causing reference resolution times to scale exponentially. To avoid infinite loops, the system relied on a cycle-detection heuristic. However, in real-world software, this heuristic frequently cut off valid resolution paths passing through the root node, leading to missed dependencies. Relaxing the path-similarity constraints (MAX_SIMILAR_PATH_COUNT) inevitably resulted in system stalls or Out-Of-Memory (OOM) crashes, making large-scale projects unanalyzable.

3.4.2 Scope-Climbing Resolution: Entangle

Entangle [26] represents the second iteration of this research line, developed by Luca Teruzzi with the explicit goal of overcoming the critical performance and memory scalability barriers encountered by Skullian.

How it works: To achieve this, Entangle completely abandoned Stack Graphs, introducing a custom, highly efficient two-phase name resolution model.

- In the first phase, it performs a static traversal of the source code's CST (generated via Tree-sitter) to reconstruct a hierarchical **Component Graph** representing packages, classes, and methods, while storing unresolved references as nested, unresolved dependency queries (modeled

via recursive QueryData structures). A stack-based symbol table is maintained during this phase to map local variables to their respective types.

- **Phase Two: Scope Climbing Resolution**: In the second phase, the engine resolves these queries directly on the component graph using a custom `Lookup and Lookdown` algorithm. The lookup step ascends the component hierarchy to resolve the root of the query (handling imports as direct architectural bridges), while the lookdown step descends through the target component's children and inheritance chains to resolve nested members (e.g., method calls and field accesses).

To remain strictly language-agnostic, Entangle relies on a configuration **dispatch table** for each supported language. Users explicitly map abstract architectural concepts (e.g. Class, Method) to the concrete CST node identifiers defined by that language's Tree-sitter grammar (e.g., class_declaration or function_item), isolating language-specific differences in a declarative configuration layer

Weaknesses and Limitations: Despite drastically improving performance, outperforming Arcan [7] by completing analyses in approximately half the execution time on average, Entangle still exhibits structural and functional limitations:

- **Manual Dispatch Table Configuration**: The dispatch table requires manual configuration for each language, demanding deep knowledge of the specific Tree-sitter grammar and its node identifiers.
- **Lack of Formalization**: Entangle's internal model lacks a rigorous mathematical formalization, which can lead to inconsistencies in name resolution and dependency extraction.
- **No Support for Generics**: The prototype completely lacks the ability to resolve and propagate parameterized types (Generics), resulting in a notable accuracy loss on template-heavy codebases.
- **Monolithic Module Resolution**: The framework is currently unable to partition distinct package modules (such as multiple Rust crates) within a single analyzed directory, causing them to collapse into a single dependency tree and compromising namespace isolation.

3.5 Positioning of the Proposed Work

The framework presented in this thesis directly addresses the limitations of both the broad state-of-the-art technologies and the specific precursor prototypes.

By relying strictly on Tree-sitter and its cursor navigation, we maintain a lightweight, single-frontend extraction process, avoiding the heavy, language-specific pipelines and CFG computational overhead of tools like Joern. Unlike Arcan, which requires dedicated compiler-based frontends (such as Spoon) for each supported language, and unlike Depends, which demands maintaining independent ANTLR grammars and visitor passes, Omnideps overcomes the $M \times N$ problem through cross-grammar structural convergence on Concrete Syntax Trees. Instead of facing the severe scalability issues of Stack Graphs

(as evidenced by Skullian), or resorting to lossy Universal AST translations (as seen in Curtis and Schiewe), we have formalized a robust Intermediate Representation ($\mathcal{D}_{IR}$) based on algebraic data types.

Crucially, compared to Entangle, we have completely eliminated the need for a manual "dispatch table". Instead, our framework implements a **Heuristic-based extraction**. Because Tree-sitter parsers use highly predictable naming conventions across different grammars (e.g., node names consistently containing `struct`, `class`, `function`, or `module`), we extract constructs by applying heuristics directly on the original CST. Coupled with a feature-based configuration to manage broad behavioral differences (like identifying if a language uses directory-based packages or inline modules), this makes the extraction phase natively language-agnostic.

The formal IR natively models complex, multi-layered module hierarchies (solving the namespace collision issues). The resolution engine has been rigorously expanded to support generic types, evaluated access paths, and accurate type bounds. Ultimately, this thesis elevates the experimental concepts of previous iterations into a mathematically sound, provably decoupled language-agnostic architectural analyzer.

Chapter 4

Intermediate Representation (IR) Model

4.1 Introduction

The analysis pipeline described in Chapter 5 is structured as a composition of typed transformations. Given a workspace $\mathcal{W}$ containing source files $w \in \mathcal{W}$ across supported programming languages, Tree-sitter produces Concrete Syntax Trees (CSTs), which are then transformed by the extraction function ε (detailed in Chapter 6) into a hierarchical forest of architectural components:

$$\varepsilon : \mathcal{W} \to \mathcal{D}$$

The goal of this chapter is to formally define the domain $\mathcal{D}$ of the Intermediate Representation, i.e., the set of all valid component hierarchies rooted in top-level Module trees $\vec{\mathcal{M}}$ that can be produced by the extraction function. The IR is designed as a *conservative extension* [16] of the high-level syntactic constructs shared across languages: it preserves the essential architectural semantics (types, functions, fields, inheritance, modules) while abstracting away language-specific syntactic details.

The subsequent phases of the pipeline, Name Resolution (ρ, Chapter 7) and Graph Construction (γ, Chapter 8), operate exclusively on $\mathcal{D}$, which guarantees that their logic is completely independent of the source language syntax.

4.2 The Type System

To ensure language agnosticism, we define the type system as an algebra of sum and product types [9].

4.2.1 Identifiers and Paths (π)

Let $\mathcal{I}$ be the set of valid strings as identifiers. We define the path domain Π and the type **Qualified Name** (π) as the set of finite non-empty sequences of identifiers:

$$\Pi = \mathcal{I}^+ = \{\langle \iota_1, \iota_2, \ldots, \iota_n \rangle \mid \iota_j \in \mathcal{I}, n \geq 1\}, \qquad \pi \in \Pi$$

This type provides a uniform hierarchical identifier for modules, types, and symbols, normalizing language-specific delimiters such as `.` in Java and `::` in C++.

4.2.2 Type References (τ)

A type reference τ in the IR models the lifecycle of symbol discovery and resolution. To guarantee *Type Safety* between the extraction phases and the graph generation, we define τ as a sum type:

$$\begin{aligned}\tau ::= &\ \text{Primitive}(\iota_p) \quad (\iota_p \in \mathcal{P}) \\ &| \ \text{Unresolved}(\pi_{raw}) \\ &| \ \text{ResolutionQuery}(\mathcal{Q}) \\ &| \ \text{Resolved}(\pi_{abs}) \\ &| \ \text{External}(\pi_{ext}) \\ &| \ \text{Generic}(\tau_{base}, \vec{\tau}_{args}) \\ &| \ \text{TypeVar}(\iota, \vec{\tau}_{bounds}) \\ &| \ \text{Union}(\vec{\tau}) \\ &| \ \text{EvaluatedAccess}(\tau_{path}, \tau_{type}) \\ &| \ \text{Failed}(\pi_{raw})\end{aligned}$$

Where:

- **Primitive**: Represents fundamental primitive types ($\iota_p \in \mathcal{P}$). Since primitive keywords vary drastically across languages, the set $\mathcal{P}$ is not static but provided by a *Primitive Registry* populated at load time. Being defined at the language level, they do not require resolution.
- **Unresolved**: A raw text path extracted by the parser before any substitution.
- **ResolutionQuery($\mathcal{Q}$)**: Represents an algebraic expression composed of `Find`, `Extract`, and `Call` operations. It is the partial product that defers resolution to Phase 2.
- **Resolved**: Represents a user-defined type (internal to the project) whose declaration or definition has been successfully localized and associated within the absolute namespace of the project. The path π_{abs} is relative to the root of the module tree.
- **External**: Represents a type that is recognized and spatially localized, but belongs to a module or ecosystem outside the provided source code (e.g., standard libraries, `java.util.*`, external crates).
- **Generic($\tau_{base}, \vec{\tau}_{args}$)**: Represents a parameterized or template type application (such as `List<String>` or `Map<String, Integer>`). It couples a base generic type reference τ_{base} with a sequence of argument type references $\vec{\tau}_{args}$.

- **TypeVar**$(\iota, \vec{\tau}_{bounds})$: Represents a generic type parameter or placeholder variable declared on a structured type or method (such as `T` in `class Box<T extends Number>`). It is identified by $\iota \in \mathcal{I}$ and constrained by upper bounds $\vec{\tau}_{bounds}$.
- **Union**$(\vec{\tau})$: Captures a disjunction of types (e.g., `Union[A, B]` in Python or `A | B` in TypeScript). It maintains a vector of references that must be individually processed.
- **EvaluatedAccess**$(\tau_{path}, \tau_{type})$: A composite reference used for member and property accesses (such as `customer.address`). It pairs the type of the accessed object τ_{path} (e.g., `Customer`) with the type of the accessed field or return value τ_{type} (e.g., `Address`). If the intermediate representation only recorded the type of the field (`Address`), the analyzer would lose the architectural dependency on the object being accessed (`Customer`). Preserving both references ensures that dependency edges are created toward both the containing entity and the field type.
- **Failed**(π_{raw}): Indicates that the static resolution phase has failed. It keeps track of the target path to support fallback analysis and calculate statistical metrics on *Recall* (coverage and confidence of the analyzer).

Figure 4.1 illustrates the complete state machine of a type reference during the analysis pipeline. This rigorous separation allows the type system to enforce that graph generation does not occur from `Unresolved` or `Failed` references.

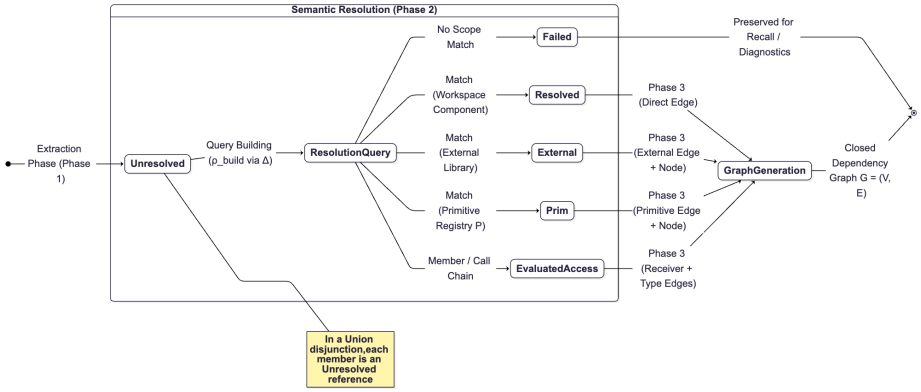


Figure 4.1: State machine diagram illustrating the lifecycle of a Type Reference (τ) from syntactic extraction to semantic resolution.

4.3 Modeling Structural Constructs

Structured Types represent the core of the architectural analysis. They are modeled as product types whose components define the state and behavior of the entity.

4.3.1 Members and Signatures

We define the **Field** type (f), representing an attribute or data field. A field is modeled as the tuple:

$$f = \langle \iota, \tau, \vec{\tau}_{ann} \rangle$$

where:

- $\iota \in \mathcal{I}$: the identifier or member name.
- τ: the declared or inferred type reference of the field.
- $\vec{\tau}_{ann}$: the ordered vector of type references representing annotations, attributes, or decorators applied directly to the field (e.g., `@Autowired`, `@Column`, or language-specific compiler directives).

We define the **Parameter** type (p) for function parameters:

$$p = \langle \iota^{?}, \tau, \mathbb{B}_{var} \rangle$$

where:

- $\iota^{?}$: the optional parameter identifier (omitted in anonymous parameters or lambda expressions).
- τ: the declared or inferred type reference of the parameter.
- $\mathbb{B}_{var} \in \{\top, \bot\}$: a boolean flag indicating whether the parameter is variadic (e.g., `...args` or `*args`).

To abstract the concept of a function's signature from the component that implements it, we introduce the **Signature** type (σ):

$$\sigma = \langle \vec{p}, \tau_{ret} \rangle$$

where:

- $\vec{p}$: the ordered vector of formal parameters.
- τ_{ret}: the return type reference.

We define the **Type Parameter** construct (θ) to model generic type variables and template parameters declared on structured types or functions:

$$\theta = \langle \iota, \vec{\tau}_{bounds} \rangle$$

where:

- $\iota \in \mathcal{I}$: the parameter identifier or name.
- $\vec{\tau}_{bounds}$: the ordered vector of type references specifying constraint bounds (such as interface implementations, trait bounds, or base class restrictions).

Building on this, to model the hierarchical body of a function, we introduce the **Block** construct ($\mathcal{B}$). A lexical block (typically delimited by braces `{}`) isolates an internal scope where local variables can be declared. Leveraging the same structure as data fields ($f = \langle \iota, \tau, \vec{\tau}_{ann} \rangle$), local variables are formalized as a vector of f. The block tracks its behavioral dependencies and can recursively nest further blocks (as in `if` or `while` statements):

$$\mathcal{B} = \langle \vec{f}, \vec{\tau}_{calls}, \vec{\tau}_{inst}, \vec{\tau}_{acc}, \vec{\tau}_{casts}, \vec{\mathcal{B}}_{sub} \rangle$$

where:

- $\vec{f}$: the vector of local variable declarations scoped to the block.
- $\vec{\tau}_{calls}$: the set of type references whose functions or methods are explicitly invoked within the block.
- $\vec{\tau}_{inst}$: the set of structured types explicitly instantiated (e.g., via `new`).
- $\vec{\tau}_{acc}$: the set of type references accessed directly through reads or writes to member states.
- $\vec{\tau}_{casts}$: the set of type references involved in explicit type casts or conversions (e.g., `as` expressions or C-style casts).
- $\vec{\mathcal{B}}_{sub}$: the vector of nested sub-blocks representing nested scopes (such as conditional branches and loops).

Finally, we define the **Function** type ($\mathcal{F}$), which models both class/struct methods and standalone free functions under a single formal representation:

$$\mathcal{F} = \langle \pi, \sigma, \vec{\theta}, \mathcal{B}^{?}, \mathbb{B}_{ctor}, \vec{\tau}_{ann} \rangle$$

where:

- $\pi \in \Pi$: the qualified path uniquely identifying the function within its declaring module or enclosing type.
- σ: the function signature $\langle \vec{p}, \tau_{ret} \rangle$, encapsulating the ordered parameters and the return type reference.
- $\vec{\theta}$: the ordered vector of generic type parameters declared on the function (e.g., `<T>` in generic methods).
- $\mathcal{B}^{?}$: the optional root lexical block representing the function body, present for concrete implementations and omitted for abstract methods or interface declarations.
- $\mathbb{B}_{ctor} \in \{\top, \bot\}$: a boolean flag indicating whether the function is a constructor or initializer.
- $\vec{\tau}_{ann}$: the ordered vector of annotation or decorator type references applied to the function.

The architectural role of an $\mathcal{F}$ (whether it acts as a method or a free function) is therefore not hardcoded in its type, but is determined exclusively by the structural context in which it is placed (within a structured type or directly within a module).

Overloading is naturally handled within the Intermediate Representation by the set of functions of an entity: a structured type or module can contain multiple functions sharing the same base identifier ι (or qualified name π) but distinguished by distinct signatures σ. When the IR is projected into the architectural dependency graph (Chapter 8), overloaded variants sharing the same qualified path π coalesce into a single graph vertex, prioritizing component-level coupling and architectural dependencies over fine-grained signature differentiation.

4.3.2 Structured Types ($\mathcal{S}$)

Let Kind = {Class, Struct, Interface, Trait, Enum, EnumVariant} be the domain of structural type kinds. A structured type is defined by the tuple:

$$\mathcal{S} = \langle \pi, k \in \text{Kind}, \vec{\theta}, \vec{f}, \vec{\mathcal{F}}, \vec{\tau}_{sup}, \vec{\mathcal{S}}_{nest}, \vec{\tau}_{ann}, \vec{\delta}_{imp} \rangle$$

where:

- π: the qualified name identifying the entity within the system.
- $k \in$ Kind: the structural kind that preserves the original semantics of the construct.
- $\vec{\theta}$: the vector of generic type parameters declared on the entity.
- $\vec{f}$: the vector of fields that compose the state of the entity.
- $\vec{\mathcal{F}}$: the vector of functions (assuming the role of methods) that define the behavior of the entity.
- $\vec{\tau}_{sup}$: the vector of references to super-types, modeling inheritance or trait/interface implementation relationships.
- $\vec{\mathcal{S}}_{nest}$: the vector of nested types.
- $\vec{\tau}_{ann}$: the vector of annotation, attribute, or decorator type references applied to the entity.
- $\vec{\delta}_{imp}$: the vector of import declarations visible within the scope of the structured type (see the definition of δ_{imp} in Section 4.4.2).

4.3.3 Type Alias ($\mathcal{A}$)

A **Type Alias** allows defining a new nominal identifier for an existing type. It is modeled as the tuple:

$$\mathcal{A} = \langle \pi, \tau_{target} \rangle$$

where:

- $\pi \in \Pi$: the qualified path uniquely identifying the type alias within its declaring scope or module hierarchy.

- $\tau_{target} \in \mathcal{T}$: the type reference pointing to the underlying aliased type.

During graph construction, type aliases are maintained as independent nodes to accurately track the semantic use of domain-specific types and architectural coupling.

4.3.4 Implementation Extensions ($\mathcal{X}$)

To support languages that decouple the schema definition of a structured type from the definition of its behavior and trait implementations (most notably Rust via detached `impl` blocks), during extraction we introduce the temporary construct **ImplBlock** or **Implementation Extension**:

$$\mathcal{X} = \langle \pi, \tau_{target}, \tau_{trait}^{?}, \vec{\mathcal{F}}, \vec{\mathcal{S}}_{nest}, \vec{\mathcal{A}} \rangle$$

where:

- $\pi \in \Pi$: the qualified path identifying where the extension block is declared in the project's module hierarchy. An implementation can be located in a different file or module from the target type itself. For example, a struct `Point` declared in module `geometry` can have an extension block written in module `render`. Tracking π allows the analyzer to know which source file contains the block, ensuring that references inside its methods are resolved using that file's import declarations before the methods are merged into the target type.
- τ_{target}: the type reference denoting the target structured type that receives the implementation.
- $\tau_{trait}^{?}$: an optional type reference identifying the trait or interface being implemented (omitted for inherent implementation blocks, such as `impl Target`).
- $\vec{\mathcal{F}}$: the vector of functions (methods) defined within the extension block.
- $\vec{\mathcal{S}}_{nest}$: the vector of nested structured types declared within the scope of the block.
- $\vec{\mathcal{A}}$: the vector of associated type aliases defined within the block (e.g., associated types satisfying trait contracts).

As detailed in Chapter 7, implementation extensions act as "syntactic sugar": their methods, nested types, and traits are *flattened* into the target structured type's scope, effectively disappearing from the final architectural model.

4.4 Component Architecture

The software system is formalized as a hierarchical forest of **Components** ($\mathcal{C}$), organized at the root level by a containment hierarchy of **Module** instances ($\mathcal{M}$). Rather than treating modules as the sole hierarchical structure, the Intermediate Representation recognizes that containment is recursive across multiple component layers:

- Modules recursively aggregate nested sub-modules ($\vec{\mathcal{M}}_{nest}$), structured types ($\vec{\mathcal{S}}$), free functions ($\vec{\mathcal{F}}$), and top-level definitions;
- Structured types recursively nest inner classes, structs, or traits ($\vec{\mathcal{S}}_{nest}$), as well as member fields ($\vec{f}$) and methods ($\vec{\mathcal{F}}$);
- Functions encapsulate lexical execution blocks ($\mathcal{B}$), which in turn nest sub-blocks ($\vec{\mathcal{B}}_{sub}$) and local declarations.

This multi-level structural containment constitutes the domain $\mathcal{D}$ of the Intermediate Representation, representing a forest of component trees where top-level modules serve as roots.

4.4.1 Component ($\mathcal{C}$)

We define a **Component** ($\mathcal{C}$) as the disjoint union (sum type) of the definitive structural constructs of the system. Each component represents the logical unit that can appear as a vertex in the dependency graph:

$$\begin{aligned}\mathcal{C} ::= &\ \text{Module}(\mathcal{M}) \\ &\mid \text{StructuredType}(\mathcal{S}) \\ &\mid \text{TypeAlias}(\mathcal{A}) \\ &\mid \text{Function}(\mathcal{F}) \\ &\mid \text{Field}(\pi, \tau) \\ &\mid \text{Primitive}(str) \\ &\mid \text{External}(\pi)\end{aligned}$$

4.4.2 Module ($\mathcal{M}$)

At the root level of the component forest, the primary namespace and encapsulation units are captured by the **Module** type ($\mathcal{M}$), defined recursively to reflect the hierarchy of the file system, packages, or namespaces:

$$\mathcal{M} = \langle \pi, str^{?}_{lang}, str^{?}_{path}, \vec{\delta}_{imp}, \vec{\mathcal{M}}_{nest}, \vec{\mathcal{S}}, \vec{\mathcal{A}}, \vec{\mathcal{F}}, \vec{f}_{free}, \vec{\mathcal{X}} \rangle$$

where:

- π: the qualified name of the module.
- $str^{?}_{lang}$: an optional string indicating the source language (e.g., `"rust"`, `"java"`).
- $str^{?}_{path}$: an optional string representing the file system path of the source file.
- $\vec{\delta}_{imp}$: the vector of **Import** declarations. Each import δ_{imp} is a product type:
  $$\delta_{imp} = \langle \pi_{path}, \iota^{?}_{alias}, \mathbb{B}_{wildcard} \rangle$$
  where π_{path} is the import path, $\iota^{?}_{alias}$ is an optional alias name (e.g., `import numpy as np` $\rightarrow \iota_{alias} =$ `np`), and $\mathbb{B}_{wildcard}$ is a boolean flag indicating whether the import is a wildcard inclusion (e.g., `from module import *`).

- $\vec{\mathcal{M}}_{nest}$: the vector of nested sub-modules.
- $\vec{\mathcal{S}}$: the vector of structured types defined in this module.
- $\vec{\mathcal{A}}$: the vector of type aliases.
- $\vec{\mathcal{F}}$: the vector of free functions (not belonging to any structured type).
- $\vec{f}_{free}$: the vector of free variables (global constants, static items).
- $\vec{\mathcal{X}}$: the vector of implementation extensions (to be flattened during name resolution).

A complete overview of the entities described so far, their composition, and their semantic references is illustrated in Figure 4.2 via a UML class diagram.

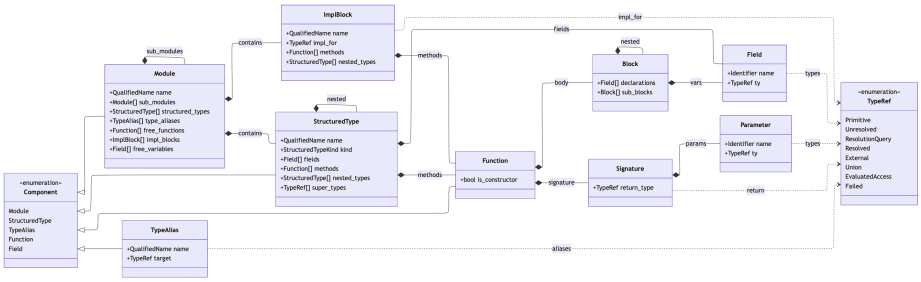


Figure 4.2: UML class diagram representing the complete Intermediate Representation (IR) domain model, highlighting hierarchical composition and semantic references via `TypeRef`.

4.5 Analyzer Configuration ($\mathcal{K}$)

A fundamental challenge in building a language-agnostic analyzer is reconciling the semantic differences between languages without introducing language-specific code branches. The approach adopted in this work draws from the concept of **Language Product Lines** [10]: rather than treating each language as a monolithic entity, the analyzer decomposes languages into a set of orthogonal **language features**. Each feature represents a specific semantic capability (e.g., "supports implicit self parameter", "uses transitive imports") that may or may not be present in any given language. The analyzer's behavior is then parameterized by these features, selecting the appropriate processing strategies at each pipeline phase.

By modeling semantic capabilities as orthogonal configuration flags, the analyzer decouples the core analysis logic from language-specific syntactic and semantic details. Rather than implementing M independent, hard-coded analysis pipelines, supporting a new language becomes a declarative task: identifying which semantic features the language exhibits and specifying its configuration profile.

Formally, the analyzer configuration is modeled as a product type:

$$\begin{aligned}\mathcal{K} = \langle\ & \text{ModuleConfig},\\ & \text{TransitiveImports},\\ & \text{AssignmentTypeAliases},\\ & \text{SelfKeyword},\\ & \text{SelfTypeKeyword},\\ & \text{ImplicitFirstParamAsSelf},\\ & \text{ExtractDynamicFields}\rangle\end{aligned}$$

This feature-based configuration model is the mechanism that makes the resolution engine and graph construction phases truly language-agnostic: they do not contain conditional branches on the source language, but instead query the configuration flags to determine the appropriate behavior.

The parameters are organized into three orthogonal dimensions: module topology, declaration linking, and instance semantics.

Module Organization Strategies

The sub-configuration **ModuleConfig** models how source files and directories are grouped into hierarchical module namespaces:

$$\begin{aligned}\text{ModuleConfig} = \langle\ & \text{FileBased},\\ & \text{DirectoryBased},\\ & \text{PackageDeclBased},\\ & \text{NamespaceBased},\\ & \text{InlineModBased}\rangle\end{aligned}$$

Each boolean flag controls whether a specific grouping mechanism is active during extraction:

- **File-Level Package Declarations (`PackageDeclBased`)**: In languages such as Java, the compilation unit declares its enclosing namespace with a single statement at the top of the file (e.g., `package com.xmpl.service`). This statement assigns the entire file to the designated logical package without wrapping declarations in syntactic braces.
- **Filesystem-Inferred Hierarchy (`FileBased`, `DirectoryBased`)**: Languages like Python and Rust infer module namespaces directly from the filesystem layout. Directories represent parent packages or modules, while individual source files correspond to leaf modules.
- **Open Namespace Blocks (`NamespaceBased`)**: In C++, namespaces are explicit structural containers delimited by braces (e.g., `namespace core { ... }`). Crucially, C++ namespaces are open and reopenable across distinct files and header units, bearing no intrinsic relationship to the directory layout.
- **Encapsulated Inline Modules (`InlineModBased`)**: In Rust, the module system exhibits a dual nature. The keyword `mod` can appear as a

file-level declaration without a body (`mod foo;`), which delegates to an external file or directory, or as an inline structural block (`mod foo { ... }`). Unlike C++ namespaces, Rust inline modules are closed and non-reopenable scopes that nest within the enclosing filesystem module.

- **Absence of Modularization (Flat Root Module)**: The C language features neither package statements nor namespace containers. When all configuration flags evaluate to false, all extracted components are collected into a single, flat root module, matching the global symbol model of the C linker.

Table 4.1 summarizes these strategies across the supported languages.

Language	FileBased	DirBased	PkgDecl	Namespace	InlineMod
Java	False	False	True	False	False
Rust	True	True	False	False	True
Python	True	True	False	False	False
C++	False	False	False	True	False
C	False	False	False	False	False

Table 4.1: Module organization strategies across supported languages.

Declaration Linking and Visibility

Languages differ fundamentally in how definitions are exposed across boundaries and linked across compilation units:

Transitive Imports (`TransitiveImports`) In languages such as Python, packages idiomatically re-export members of sub-modules through `__init__.py` files, allowing consumers to access them transitively. Conversely, in Java and Rust, imports are file-local by default and require explicit re-export declarations (e.g., `pub use`).

Assignment Type Aliases (`AssignmentTypeAliases`) Dynamically typed or scripting languages such as Python introduce type variables and type aliases through assignment expressions at module or class scope (for instance, `T = TypeVar('T', bound=Entity)` or `Vector = list[float]`). Enabling this flag activates parser heuristics that recognize type variable instantiations and subscripted type annotations on assignment targets, promoting them into formal type aliases ($\mathcal{A}$) or type parameters (θ) instead of discarding them as runtime variable assignments.

Table 4.2 summarizes the declaration and linking profiles.

Instance Semantics and Receiver Resolution

Resolving member accesses and invocations requires understanding how an instance is referenced and how its members are declared within local scopes:

Language	TransitiveImports	AssignmentAliases
Java	False	False
Rust	False	False
Python	True	True
C++	False	False
C	False	False

Table 4.2: Declaration linking and visibility configuration profiles.

Receiver Binding (`SelfKeyword`, `ImplicitFirstParamAsSelf`) In C++ and Java, `this` is a language-level keyword providing an implicit reference to the calling instance. In Rust, instance methods explicitly declare `self` as their first parameter. In Python, `self` is not a keyword, but a naming convention; Python instance methods implicitly bind the calling instance to their first formal parameter (`ImplicitFirstParamAsSelf` = True).

Dynamic Attribute Discovery (`ExtractDynamicFields`) Unlike statically typed languages where fields are declared explicitly in the type body, dynamic languages like Python frequently introduce instance fields dynamically within constructor bodies via assignments of the form `self.field = value`. Enabling this flag instructs the extractor to inspect constructor ASTs and harvest these dynamic attributes.

Type Self-Reference (`SelfTypeKeyword`) Rust introduces the `Self` keyword to reference the implementing type within an `impl` block, injected as a type alias into the local scope.

Structural Deref Delegation In languages featuring smart pointer dereferencing through traits (such as Rust's `std::ops::Deref`), method and member accesses on a container implicitly delegate to an inner target type. In OmniDeps, when an implementation block implements a trait named `Deref` and defines an associated type alias `type Target = T`, the target type T is appended directly to the implementing type's super-types in the Scope Tree. This structural rule integrates seamlessly with the standard Method Resolution Order without requiring an ad-hoc configuration parameter in $\mathcal{K}$.

Table 4.3 outlines the instance semantics and receiver resolution features.

Language	SelfKeyword	ImplicitSelf	DynamicFields	SelfType
Java	`this`	False	False	—
Rust	`self`	False	False	`Self`
Python	—	True	True	—
C++	`this`	False	False	—
C	—	False	False	—

Table 4.3: Instance model, receiver resolution, and type anchor features.

Chapter 5

Analysis Pipeline Overview

The analysis system is modeled as a composition of sequential transformations, each operating on well-defined input and output domains. Let $\mathcal{W}$ be the domain of a Workspace (a finite collection of source files $w \in \mathcal{W}$), and $\mathcal{D}$ the domain of the Intermediate Representation defined in Chapter 4. The complete analysis function ϕ is defined as:

$$\phi : \mathcal{W} \to \mathcal{G} \qquad \phi = \gamma \circ \rho \circ \varepsilon = \gamma \circ (\rho_{\text{exec}} \circ \rho_{\text{build}}) \circ \varepsilon$$

The pipeline is illustrated in Figure 5.1. Each phase receives the output of its predecessor and transforms it into a higher-level, semantically refined representation of the codebase.

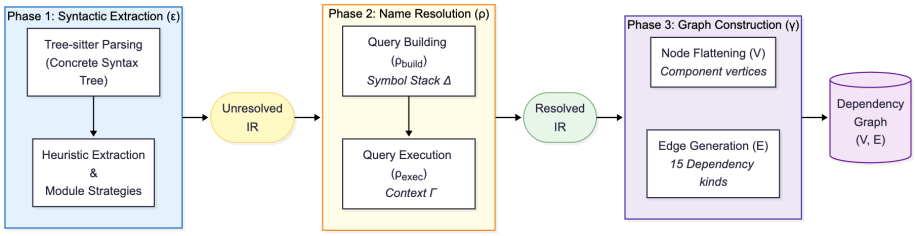


Figure 5.1: Architectural overview of the three-phase analysis pipeline, from source code to dependency graph.

A fundamental design property of this decomposition is the classical separation between concrete syntax and semantic analysis, mirroring standard compiler pipeline architectures [16]. Phase 1 is the only phase that interacts with language-specific syntax and parse trees. Across the pipeline, the declarative language configuration $\mathcal{K}$, inspired by modular language product lines [10], parameterizes language-specific behaviors governing module grouping strategies in Phase 1 and semantic scoping rules in Phase 2. Finally, Phase 3 is purely structural and completely language-agnostic.

5.1 Phase 1: Syntactic Extraction (ε)

$$\varepsilon : \mathcal{W} \to \mathcal{D}$$

The extraction phase transforms source code into the Intermediate Representation. For each source file in the workspace, a Tree-sitter parser [8] generates a Concrete Syntax Tree (CST), which is then traversed by a set of heuristic classifiers and parsers to identify and extract the relevant software components (modules, structured types, functions, fields, imports, and implementation blocks).

Input

The input is a **Workspace** $\mathcal{W}$: a directory tree containing source files in one or more of the supported languages (C, C++, Java, Python, Rust). Each file is independently parsed and extracted.

Process

The extraction operates through four logical layers:

1. **Parsing**: Tree-sitter generates a CST for each source file, preserving every token of the original code.
2. **Classification**: Heuristic predicates inspect CST node kinds (such as `struct_item`, `function_definition`, `class_declaration`) using string-based pattern matching to determine the role of each node. This approach exploits the structural similarity of Tree-sitter grammars across languages to avoid maintaining per-language rule sets.
3. **Structural Parsing**: Classified nodes are transformed into IR components by extracting their constituent parts (names, fields, methods, parameters, return types, super-types, annotations).
4. **Body Extraction**: Function bodies are analyzed to extract syntactic invocations, field accesses, and reference queries alongside local variable declarations, organized into a recursive `Block` structure that mirrors the lexical scope hierarchy of the source code.

After individual file extraction, a **Module Grouping Strategy** organizes the extracted modules into a hierarchical tree that reflects the project's namespace structure. The grouping mechanism is driven by declarative configuration flags in $\mathcal{K}$ rather than hardcoded language branches, supporting directory-based hierarchies (e.g., Python, Rust), explicit package declarations (e.g., Java), syntax-level namespace blocks (e.g., C++), or a flat root fallback (e.g., C).

Output

The output is a hierarchical forest of **component trees rooted in module instances** $\vec{\mathcal{M}} \in \mathcal{D}$, unified across all files in the workspace. Each module contains its structured types, free functions, imports, implementation blocks, type aliases, and free variables. All type references (τ) at this stage are in their initial extraction states: raw symbolic identifiers $\text{Unresolved}(\pi_{\text{raw}})$, parameterized type applications $\text{Generic}(\tau_{base}, \vec{\tau}_{args})$, type parameter variables $\text{TypeVar}(\iota, \vec{\tau}_{bounds})$, or composite disjunctions $\text{Union}(\vec{\tau})$ for dynamic typing (e.g., Python type unions). No cross-component linking or primitive matching has taken place yet.

Key Properties

- This is the **only language-dependent** phase. The heuristic classifiers and extractors are the sole components that interact with language-specific CST node kinds.
- Each file is extracted **independently**. Cross-file dependencies are not resolved in this phase.
- The output IR is **structurally complete** but **semantically unresolved**: the topology of the codebase (what components exist and what they contain) is fully captured, but the targets of type references are not yet known.

The detailed implementation of this phase, including the heuristic classification system, the extraction algorithms, and the module grouping strategies, is presented in Chapter 6.

5.2 Phase 2: Name Resolution (ρ)

$$\rho : \mathcal{D} \to \mathcal{D}_{\text{res}} \qquad \rho = \rho_{\text{exec}} \circ \rho_{\text{build}}$$

The name resolution phase transforms the symbolic type references produced by extraction into resolved semantic pointers. This is the phase that binds isolated structural components into a semantically connected architectural model by determining, for each type reference, which component in the workspace (or external library) it refers to.

Input

The input is the forest of component trees $\vec{\mathcal{M}} \in \mathcal{D}$ produced by Phase 1, accompanied by the declarative Language Configuration $\mathcal{K}$ and the Primitive Registry $\mathcal{P}$.

Process

The resolution is decomposed into two sequential steps:

$$\rho_{\text{build}} : \mathcal{D} \xrightarrow{\Delta} \mathcal{D}_{\mathcal{Q}} \qquad \rho_{\text{exec}} : \mathcal{D}_{\mathcal{Q}} \xrightarrow{\Gamma} \mathcal{D}_{\text{res}}$$

This two-step decomposition separates local intra-procedural context from global workspace resolution. References inside function bodies frequently depend on local variables, parameters, and block-level shadowing. Resolving these references in a single step would require maintaining complex local state alongside global symbol lookups. Instead, dividing the process into query building and query execution cleanly isolates these concerns:

1. **Query Building** (ρ_{build}): The component hierarchy is traversed with an ephemeral *Symbol Stack* (Δ) that tracks local variable and parameter bindings across lexical blocks. Each `Unresolved` type reference is replaced by an algebraic *Resolution Query* ($\mathcal{Q}$) composed of three primitives:
    - `Find(`s`)`: Ascending scope search for identifier s.

- `Extract(`q`, `s`)`: Structural member access on the result of sub-query q.
- `Call(`q`)`: Return-type extraction after evaluating sub-query q as a callable.

The Symbol Stack enables local variable substitution: when an expression such as `x.method()` is encountered, the stack substitutes `x` with its declared type, transforming the intra-procedural reference into a self-contained algebraic query that can be resolved globally. Once traversal completes, all local variable state is eliminated.

2. **Query Execution** (ρ_{exec}): A global *Scope Tree* ($\mathcal{E}$) is constructed to index all declared modules, types, and members across the workspace. The engine evaluates each query expression against this tree, navigating the corresponding lexical and structural scopes to bind each reference to its definitive target declaration or identify it as an external symbol.

The resolution is parameterized by a per-language **configuration** ($\mathcal{K}$) that controls behaviors such as transitive import visibility, implicit receiver parameter binding and instance keyword recognition.

Output

The output of Phase 2 is the resolved component forest $\vec{\mathcal{M}}_{\text{res}} \in \mathcal{D}_{\text{res}}$. Structurally, it maintains the exact topology and component hierarchy extracted in Phase 1. Semantically, every symbolic identifier and query expression has been evaluated to determine the target component it refers to.

In this resolved model, references targeting internal components within the analyzed workspace are bound to their fully qualified paths, while references to standard library APIs or third-party packages are classified as external symbols or language primitives. For parameterized and composite constructs, all constituent type arguments are recursively resolved. Finally, any symbol that cannot be located in visible scopes is recorded as a failed lookup, preserving diagnostic context for downstream evaluation. At this stage, the codebase is represented as a fully linked architectural model ready for graph generation.

Key Properties

- The resolution engine is **language-agnostic**: the core resolution algorithm, Scope Tree traversal, and query evaluation operate exclusively on the shared IR, while language-specific semantic rules (such as import visibility, `self` bindings, and inheritance conventions) are supplied declaratively through the configuration $\mathcal{K}$.
- The resolution is **workspace-global**: because the Scope Tree is built from all files simultaneously, cross-file dependencies are resolved transparently.
- The Execution Context ($\Gamma = \langle \mathcal{E}, \mathcal{P}, \mathcal{K} \rangle$) is **immutable** during query evaluation, guaranteeing deterministic results independent of evaluation order.

The detailed implementation of this phase, including the query algebra, the Scope Tree construction, the lexical climbing algorithm, and the inheritance resolution mechanism, is presented in Chapter 7.

5.3 Phase 3: Graph Construction (γ)

$$\gamma : \mathcal{D}_{\text{res}} \to \mathcal{G}$$

The graph construction phase projects the resolved IR into a flat directed dependency graph $\mathcal{G} = (V, E)$ suitable for architectural analysis and visualization.

Input

The input is the resolved component forest $\vec{\mathcal{M}}_{\text{res}} \in \mathcal{D}_{\text{res}}$ produced by Phase 2, in which all type references have been resolved to their corresponding internal declarations, external dependencies, or language primitives.

Process

The construction proceeds in two steps:

1. **Node Flattening**: The hierarchical component forest is recursively traversed, promoting every declared component (modules, structured types, functions, fields, and type aliases) into a flat set of vertices V. Each vertex receives a globally unique qualified name that reflects its full enclosing namespace path. Additionally, implicit nodes are synthesized in V for primitive types and external dependencies that are referenced by edges but not declared in the workspace, ensuring the graph is self-contained.

2. **Edge Generation**: The component forest is traversed a second time, inspecting every resolved reference across declarations, signatures, and blocks to emit directed dependency edges. Each edge connects the source component to its resolved target and is typed according to the architectural relationship it represents (such as inheritance, type usage in parameters or return types, member accesses, or function invocations). Unresolved references are omitted to avoid introducing noise, and redundant parallel edges of the same kind are deduplicated to prevent distorted coupling metrics.

Output

The output of Phase 3 is a unified **Dependency Graph** $\mathcal{G} = (V, E)$:

- V: A set of component vertices $v \in \mathcal{C}$, each identified by its canonical qualified name $\pi(v) \in \Pi$ and typed by its architectural kind (Module, Structured Type, Function, Field, Type Alias, Primitive, External).

- E: A set of directed dependency edges $E \subseteq V \times V \times \mathcal{K}_{\text{edge}}$, where each directed edge $e = \langle u, v, k \rangle$ specifies the source component u, the target component v, and the taxonomic dependency kind $k \in \mathcal{K}_{\text{edge}}$.

The graph is primarily serialized in a Cytoscape.js-compatible JSON format [35] where structural containment is encoded as compound node nesting rather than explicit edges.

Key Properties

- This phase is **completely language-independent**. The lowering algorithm operates uniformly over the resolved IR without language-specific branches.
- **Graph Closure Property**: The constructed graph is strictly closed: $\forall \langle u, v, k \rangle \in E \implies u \in V \land v \in V$. Whenever a resolved reference targets an external library or a primitive type not present among declared workspace entities, synthetic vertices (`Component::External` and `Component::Primitive`) are dynamically instantiated in V to guarantee structural integrity.

The detailed implementation of this phase, including the flattening algorithm, the complete edge taxonomy, and the Cytoscape export process, is presented in Chapter 8.

5.4 Pipeline and Architectural Guarantees

Table 5.1 synthesizes the pipeline transformations across phases, detailing their domain mappings, operational contexts, and execution scopes.

Phase	**Transformation**	**Context**	**Scope**
1. Extraction	$\varepsilon : \mathcal{W} \to \mathcal{D}$	Tree-sitter CST	Per-file
2a. Query Building	$\rho_{\text{build}} : \mathcal{D} \to \mathcal{D}_{\mathcal{Q}}$	Symbol Stack Δ	Local scope
2b. Query Execution	$\rho_{\text{exec}} : \mathcal{D}_{\mathcal{Q}} \to \mathcal{D}_{\text{res}}$	Execution Context $\Gamma = \langle \mathcal{E}, \mathcal{P}, \mathcal{K} \rangle$	Workspace
3. Graph Construction	$\gamma : \mathcal{D}_{\text{res}} \to \mathcal{G}$	-	Workspace

Table 5.1: Summary of pipeline transformations: operators, domain mappings, operational contexts, and execution scope.

This pipeline architecture establishes several key guarantees:

1. **Type-Safe Functional Composition**: Each stage is a pure deterministic function over well-defined domains [9]. The end-to-end transformation $\phi = \gamma \circ (\rho_{\text{exec}} \circ \rho_{\text{build}}) \circ \varepsilon$ is well-typed and preserves structural correctness across lowering steps.
2. **Determinism and Confluence**: Because the execution context $\Gamma = \langle \mathcal{E}, \mathcal{P}, \mathcal{K} \rangle$ remains immutable during query evaluation, query evaluation has no side effects on the environment. Consequently, evaluating type queries in any order yields identical results.
3. **Guaranteed Termination**: The pipeline terminates for any finite workspace: the CST traversal is bounded by the finite node depth of parse trees, lexical scope climbing is bounded by the finite height of the Scope Tree $\mathcal{E}$, and inheritance resolution employs cycle detection to prevent infinite loops.
4. **Graph Closure**: As established in Section 5.3, Phase 3 ensures that $\forall \langle u, v, k \rangle \in E \implies u \in V \land v \in V$. Synthetic vertices are dynamically

added to V for primitive types and external libraries, guaranteeing that the exported graph contains no dangling edges.

5. **Extensibility**: Adding support for a new language requires implementing only the syntactic CST extraction (Phase 1) and declaring its configuration $\mathcal{K}$ (Phase 2). The query algebra, scope climbing, and graph construction remain entirely reused.

Chapter 6

Syntactic Extraction

6.1 Overview

The extraction phase is the first stage of the analysis pipeline. It transforms source code files into the language-agnostic Intermediate Representation defined in Chapter 4. Formally, the extraction phase is defined as:

$$\varepsilon : \mathcal{W} \to \mathcal{D}$$

where the workspace $\mathcal{W}$ represents the collection of source files across the repository, and $\mathcal{D}$ denotes the domain of valid component hierarchies rooted in top-level module trees $\vec{\mathcal{M}}$. For each individual source file $w \in \mathcal{W}$, extraction parses the file using the corresponding language grammar and produces a local component tree rooted in a module $\mathcal{M} \in \mathcal{D}$. This is the **only phase that depends on the source language**. Since Tree-sitter CST nodes contain only positional metadata (byte offsets into the source buffer), every extraction function accesses the underlying source buffer of w to retrieve textual identifiers and expressions.

The extraction subsystem is organized into four logical layers, each depending only on the layers below it (depicted in Figure 6.1 and summarized in Table 6.1):

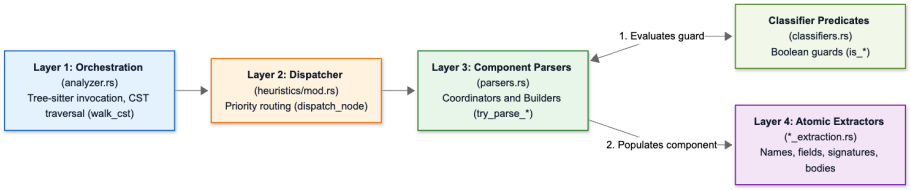


Figure 6.1: Layered architecture of the syntactic extraction subsystem.

6.2 The Entry Point: `extract_from_cst`

The top-level extraction function `extract_from_cst` receives a Tree-sitter language grammar and a source string, and produces a vector of IR modules. Its workflow is:

Layer	Module	Responsibility
1 (Top)	`extraction/mod.rs`	Orchestration: Tree-sitter parsing, CST traversal, workspace grouping
2	`heuristics/mod.rs`	Dispatcher (`dispatch_node`), priority routing
3	`heuristics/classifiers.rs`, `parsers.rs`	Classification predicates and IR component parsers
4 (Bottom)	`heuristics/*_extraction.rs`, `text_parsing.rs`	Low-level data extraction from CST nodes

Table 6.1: Logical layer architecture of the extraction subsystem.

1. A Tree-sitter parser is instantiated and configured with the language grammar.
2. The source code is parsed into a Concrete Syntax Tree (CST), producing a root node.
3. If the language uses package declarations (e.g. `package com.example`), the root children are scanned for a package declaration node and the package path is extracted.
4. A module vector and a pending-attributes buffer are initialized.
5. The recursive traversal function `walk_cst` is invoked on the root node, populating the module vector with extracted IR components.

The function returns both the extracted modules and the optional package path. A higher-level function, `parse_source`, then applies the appropriate Module Grouping Strategy (Section 6.6.1) to organize the components into their hierarchical module paths.

6.3 CST Traversal: `walk_cst`

The function `walk_cst` is the core of the extraction phase. It recursively visits every node of the CST and delegates classification to the dispatcher. Its traversal logic follows four rules:

1. For each CST node, invoke `dispatch_node`. If the dispatcher returns a recognized component, insert it into the current module (`modules[0]`) and **return immediately**, without descending into children. This "pruning" rule is fundamental: without it, the methods of a class would be extracted both as members of the class (by the structured type parser) and again as independent free functions (by a subsequent recursive call).
2. **Exception for Modules**: When a module is recognized (e.g., Rust `mod`, C++ `namespace`), a temporary module vector containing the newly parsed module is created, and the children of the CST module node are re-traversed in the context of this new module. After traversal, the populated module is attached as a sub-module of `modules[0]`. This mechanism preserves the hierarchical nesting of the source code in the IR.

3. **Exception for Functions**: When a free function is recognized, its children of type "body" or "block" are still traversed. This handles the case of struct or class definitions nested inside a function body, which is permitted in some languages.

4. If the dispatcher returns `None` (unrecognized node), recursively traverse all children of the current node.

Figure 6.2 summarizes this traversal workflow, highlighting the pruning branch that prevents redundant extraction of internal methods as free functions.

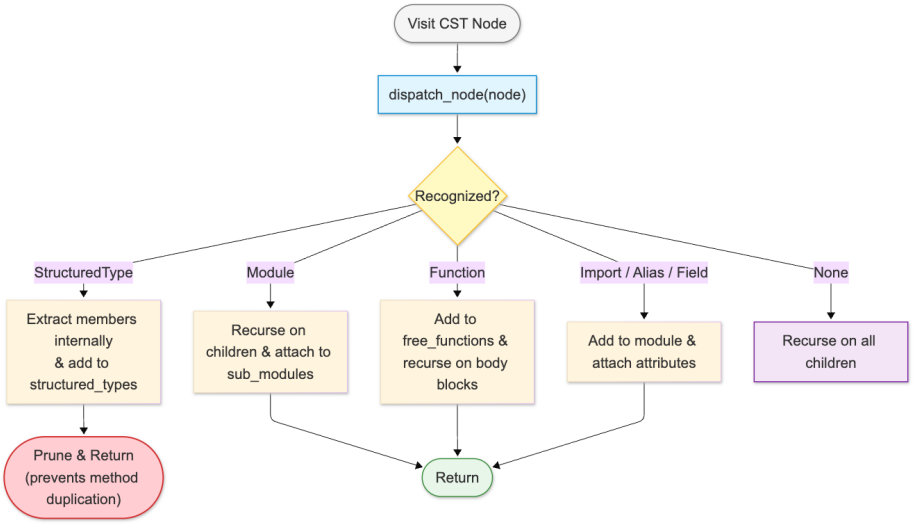


Figure 6.2: Flowchart of the `walk_cst` traversal logic and subtree pruning mechanism.

During single-file CST traversal, an ephemeral "root" module container is created lazily on demand: when the first component is encountered and the module vector is still empty, a temporary module named `"root"` is inserted to collect declarations before the Module Grouping Strategy organizes them into their final namespace paths.

Pending Attributes Mechanism

Annotations, decorators or attributes (e.g., Python's `@staticmethod`, Rust's `#[derive(Debug)]`, Java's `@Override`) are syntactically placed *before* the component they annotate. The CST represents them as sibling nodes, not as children of the annotated component. To handle this, `walk_cst` maintains a `pending_attributes` buffer: when the dispatcher recognizes an attribute node, the extracted annotation type references are appended to this buffer. When the next component (function, class, field, etc.) is recognized, the buffered attributes are attached to it as its `annotations` list, and the buffer is cleared.

6.3.1 Dispatch Priority Order

The dispatcher is the central decision point of the extraction phase. For each CST node, it invokes a chain of `try_parse_*` functions in a fixed priority order, returning the first successful match:

1. **Imports**: Recorded before processing contained declarations.
2. **Decorators**: Declarative metadata to subsequent declarations.
3. **Modules**: If module syntax is enabled by the language configuration.
4. **Structured Types**: Classes, structs, interfaces, traits, and enums.
5. **Functions and Methods**: Functions, constructors and methods.
6. **Detached implementations**: Such as Rust `impl` blocks.
7. **Type Aliases**: Explicit type synonyms and alias declarations.
8. **Free Variables**: Global variables and static constants.

The dispatch order prevents ambiguity: a module node must be captured as a module before any of its inner structs are individually matched, and a class node must be captured as a whole before its methods are matched as free functions. This ordering ensures that enclosing containers take precedence over their member declarations.

The dispatcher's return type is a `ParsedItem` enum with four variants: `Component` (for modules, types, functions, fields, type aliases), `ImplBlock`, `Imports` (which may return multiple imports from a single node), and `Attribute`.

6.3.2 Classification Heuristics

The classifiers are boolean predicates that identify the *role* of a CST node. They use string-based pattern matching on the node's `kind()` property. This approach is the key to language agnosticism: rather than maintaining separate grammar rules per language, the analyzer exploits the fact that Tree-sitter grammars across different languages use similar naming conventions for equivalent constructs [8]. In contrast to universal syntax tree frameworks such as Babelfish UAST [30], which attempt to map diverse languages into a single unified taxonomy, OmniDeps relies on lightweight nominal heuristics directly over native CST nodes.

Each classifier follows a pattern of **positive conditions** (the node kind must match or contain expected substrings, or satisfy structural sub-tree patterns) combined with **negative exclusions** (the node kind or syntactic context must *not* indicate a different semantic role).

The negative exclusions are necessary to prevent false positives. Some representative examples:

- `struct_expression` contains "struct" but represents a struct instantiation (e.g., `MyStruct { field: value }`), not a type definition. The exclusion of "expression" prevents this from being classified as a structured type.

Classifier	Positive Conditions	Negative Exclusions
`is_module`	`mod_item`, `module`, `namespace`	`identifier`
`is_package_declaration`	`package_declaration`, `package_clause`	None (exact match)
`is_structured_type`	`struct`, `class`, `interface`, `trait`, `enum`, `union`, `annotation_type`; or `type_definition` enclosing `struct`, `enum`, or `union`	`bound`, `clause`, `list`, `expression`, `argument`, `call`, `identifier`, `reference`, `body`, `mod`, `super`, `base`, `constructor`
`is_function`	`function`, `method`, `fn_item`, `func`, `constructor`, `destructor`, `decorated_definition`; or `declaration` with nested `function_declarator`	`class`
`is_closure`	`closure_expression`, `lambda_expression`, `lambda`, `arrow_function`, `function_expression`, `func_literal`, `lambda_literal`, `anonymous_method_-expression`	None (exact match)
`is_import`	`use_declaration`, `import_declaration`, `import_statement`, `import_from_statement`, `preproc_include`	None (exact match)
`is_impl_block`	`impl_item`, `impl_block`	None
`is_free_variable`	`static_item`, `const_item`, `global_variable_-declaration`; or top-level `declaration`, `variable_declaration`	Ancestor is function, method, body, or block scope; or contains nested `function_declarator`
`is_type_alias`	`type_alias_declaration`, `alias_declaration`, `type_item`, `type_alias_statement`; or non-struct `type_definition`	Excludes structured types

Table 6.2: Summary of classifier predicates and their heuristic matching rules.

- `constructor_declaration` contains "struct" (via "con**struct**or") and must be excluded.
- `class_body` contains "class" but is the body of a class, not the class declaration itself. The exclusion of "body" prevents this match.
- `trait_bound` and `base_class_clause` contain "trait" and "class" respectively, but represent constraint syntax, not type definitions.

The `is_import` classifier is the only one that uses exact matching (`matches!` in Rust) rather than substring containment, because import node names are highly specific and substring matching would cause excessive false positives.

The `is_free_variable` classifier is notable for its use of **parent chain inspection**: in C, both global and local variable declarations produce the same `declaration` node kind. To distinguish them, the classifier walks up the parent chain. If the declaration is nested inside a `compound_statement` or `function_definition`, it is classified as local (and ignored); if its parent is a `translation_unit` or `declaration_list`, it is classified as a global variable.

A specialized helper, `has_nested_function_declarator`, unrolls nested declarators to disambiguate function declarations from variable declarations that happen to contain function pointer syntax.

6.4 Structural Parsers

Rather than processing nodes in a rigid sequential pipeline, the extraction subsystem adopts a coordinator pattern. The heuristic dispatcher delegates candidate nodes to the structural parsers (`try_parse_*`). Each parser coordinates the extraction of a specific component kind: it first invokes the corresponding classifier predicate as a boolean guard to verify whether the CST node matches the target role. If the check succeeds, the parser coordinates the lower-level atomic extractors to retrieve identifiers, fields, signatures, and method bodies, assembling and returning the populated IR component. If the classifier returns false, the parser immediately returns `None`, prompting the dispatcher to evaluate the next candidate.

Structured Type Parsing

The `try_parse_structured_type` function extracts the full content of a class, struct, interface, trait, or enum definition:

1. The qualified name is extracted via `extract_qualified_name`.
2. The `StructuredTypeKind` is determined by inspecting the node kind and its textual content (`Class`, `Struct`, `Interface`, `Trait`, `Enum`, `EnumVariant`).
3. Fields, methods, super-types, nested types, and annotations are extracted using the respective extractors.

A special case handles C/C++ `type_definition` nodes that contain an inner `struct_specifier` or `enum_specifier`: the parser hoists the fields from the inner specifier and filters out redundant anonymous types.

The methods, fields, and nested types are extracted *internally* by the structured type parser, using the generic list combinator `extract_list_of`. This is why the CST traversal in `walk_cst` prunes the subtree after recognizing a structured type: all relevant information has already been extracted by the parser.

Dynamic Field Extraction

In Python, instance fields are typically not declared at the class level but are dynamically assigned in methods via `self.field = value` patterns. When the `extract_dynamic_fields` configuration flag is enabled, the field extractor enters method bodies and tracks the active `self` parameter name. Assignments to `self.field` are detected and the field name and inferred type are extracted. Field types are inferred from the right-hand side of the assignment using `infer_variable_type`.

To prevent duplicates when a field is assigned in multiple methods, the extractor deduplicates fields by name, favoring entries with more informative type references (resolved or inferred types over unresolved or failed ones).

Function Parsing

The `try_parse_function` function extracts:

1. **Identifier**: The simple or qualified function name, unwrapping wrapper nodes such as Python's `decorated_definition`.
2. **Parameters**: Formal parameters extracted via `extract_parameters`, including parameter names and type references.
3. **Return Type**: The declared return type, extracted via `extract_return_type`.
4. **Constructor Flag**: Marked as a constructor when the node kind indicates constructor semantics.
5. **Annotations**: Metadata decorators or compiler attributes attached to the declaration.
6. **Function Body**: If a body is present (located through the `body` field or compound block containers), its internal scope hierarchy is extracted as a structured `Block` ($\mathcal{B}$) via `extract_block` (Section 6.4). When no body is present (such as interface methods or C/C++ forward declarations), the body is recorded as `None`.

Structured Body Parsing

The body of a function is extracted into a structured `Block` ($\mathcal{B}$) that preserves the lexical scope hierarchy. The function `extract_block` classifies each child of a body node into three categories:

1. **Local Variable Declarations**: Nodes matching known declaration kinds (`let_declaration`, `local_variable_declaration`, `variable_declaration`, `lexical_declaration`, etc.) are extracted as `Field` entries in the

block's `declarations` vector. When a variable has no explicit type annotation, the function `infer_variable_type` inspects the initializer expression for patterns that reveal the type:

- `new Foo()` or `Foo {...}` → the type is inferred as `Foo`.
- `Bar::new()` → the type is inferred as the return type of `Bar::new`, encoded as a `ResolutionQuery(Call(Extract(Find(Bar), new)))`.
- In dynamically typed code lacking type annotations [19], untyped assignments like `x = Admin()` produce a `Query::Call` to resolve the callable's return type.

This mechanism highlights the modular decoupling between extraction (ε) and resolution (ρ): because per-file extraction operates strictly on local syntax without global symbol visibility, `infer_variable_type` does not attempt to evaluate return types eagerly. Instead, it synthesizes a deferred expression $q \in \mathcal{Q}$ within the Query Algebra (Chapter 4), delegating its semantic evaluation to the name resolution engine ρ_{exec} (Chapter 7) once the global workspace scope tree is constructed.

2. **Nested Blocks**: Children whose kind contains "body" or "block", or equals `compound_statement`, are recursively extracted as sub-blocks via a recursive call to `extract_block`. This creates a tree of `Block` structures that mirrors the control-flow nesting of the source code (`if`, `while`, `for`, `try/catch`, anonymous scopes).

3. **Behavioral Dependencies**: All remaining children (expression statements, return statements, assignments, etc.) are inspected recursively to discover behavioral dependencies across four semantic categories:

   - **Instantiations**: Constructor invocations (e.g., `new` expressions in Java and C++) and struct literal expressions (e.g., Rust struct initializations).
   - **Function and Method Calls**: Invocations across three structural forms:
     - *Scoped calls* (`Module::func()`) → the scope qualifier is captured to resolve namespaced or static calls.
     - *Method invocations* (`obj.method()`) → the receiver object expression is captured to track member dereferencing.
     - *Simple calls* (`func()`) → the bare callable identifier is captured.

     When callable identifiers or expressions are enclosed in protective parentheses (e.g., `(func)()` or macro expansions), the function `extract_call_path` recursively unwraps `parenthesized_expression` CST nodes. This strips literal punctuation tokens and yields clean identifier sequences without attempting semantic resolution during syntactic extraction.
   - **Field and Member Accesses**: Explicit property accesses, attribute dereferences, and qualified symbol paths.
   - **Type Casts**: Explicit type conversions and cast expressions, ensuring dependency tracking on target coercion types.

The behavioral dependency search **skips** nested block nodes to avoid double-counting: dependencies inside sub-blocks are captured by the recursive `extract_block` call, not by `find_behavioral_deps`. This guarantees that each dependency is recorded at the exact lexical scope level where it occurs.

Import Parsing

The `try_parse_imports` function handles import statements across all supported languages. It returns a vector of `Import` structures because some syntactic forms (e.g., Python's `from module import a, b, c`) produce multiple logical imports from a single CST node. The parser:

- Detects wildcard imports by checking for `*`, `.*`, `::*`, or `using namespace` patterns.
- Detects import aliases by searching for the `as` keyword in the import text.
- Extracts import paths using CST named fields where available, with textual fallback parsing. File extensions (e.g., `.h`, `.hpp`) are stripped from C/C++ `#include` paths.

Implementation Block Parsing

The `try_parse_impl_block` function extracts:

- The target type (`impl_for`), as a `TypeRef`.
- The optionally implemented trait (`implements_trait`), extracted by searching for the `for` keyword in the node text to distinguish `impl Type` from `impl Trait for Type`.
- Methods, nested types, and type aliases defined within the block.

Type Alias and Free Variable Parsing

Type aliases are extracted with their declared name and target type reference. For languages supporting explicit alias syntax (such as Rust `type X = Y;` or C++ `using X = Y;`), the parser extracts the alias name and target using `extract_type_ref`.

In dynamically typed languages such as Python, type variables and aliases are created via value assignments at module or class scope. When the `AssignmentTypeAliases` feature flag is enabled in the language configuration, the parser detects assignment statements where the right-hand side denotes a type constructor:

- **Type Variable Factories**: Calls to `TypeVar('T', bound=...)` are captured as a type alias mapping `T` to a TypeRef::TypeVar("T", $\vec{\tau}_{bounds}$).
- **Generic Subscriptions**: Expressions such as `Vector = List[float]` are parsed as a type alias whose target is a TypeRef::Generic.

Free variables (global constants, static items) are extracted as `Field` entries with their name, type reference, and annotations.

6.5 Low-Level Extractors

The lowest layer of the extraction subsystem provides atomic functions that interact directly with CST nodes.

Text Extraction

- `node_text`: Extracts the raw source text corresponding to a CST node by reading from the source buffer using the node's byte range. This is the fundamental bridge between the positional CST and the textual content of the source code. qualified name string (such as `java.util.List`, `std::vector`) into a `QualifiedName` vector by splitting on separator characters (`:`, `.`, and whitespace). The `->` operator is normalized to `.` before splitting. When called on plain type text, any trailing generic parameter bracket (`<...>`) is stripped to isolate the nominal base path, while structured generic decomposition is handled beforehand by `extract_type_ref`.

Identifier Extraction

- `extract_identifier`: Extracts the simple name of a component by trying a chain of named fields (`name`, `type_identifier`, `identifier`), with a fallback to the `declarator` field for C and C++ declarations. The declarator fallback recursively descends nested declarator nodes (`pointer_declarator`, `function_declarator`, `reference_declarator`) to reach the base identifier. The declarator unwrapping accepts both normal identifiers and `destructor_name` nodes, preserving the `~` prefix for C++ destructors (e.g., `~Server`) to distinguish them from constructors and prevent naming collisions.

- `extract_qualified_name`: Extracts multi-part names by first trying named fields and applying `split_qualified_name`, then falling back to collecting all child nodes of type `identifier` or `type_identifier` into a vector.

Type Reference Extraction

The function `extract_type_ref` converts a CST node into a `TypeRef`. It operates through a cascade of progressively less specific strategies:

1. **Type wrapper unwrapping**: If the node is a single-child generic wrapper (such as a Python `type` node), extraction recurses directly into its inner child.

2. **Union types**: If the node represents a union type (`union_type`, a binary operator with `|`, or a `Union[...]` annotation), it is recursively decomposed into `TypeRef::Union(`$\vec{\tau}$`)` to represent type disjunctions.

3. **Generic and template types**: The function `try_extract_generic` is called to inspect AST nodes representing parametric types across languages:

- Tree-sitter node kinds `template_type` (C++), `generic_type` (Java, Rust), and `subscript` (Python `List[T]`);
- Textual fallback `parse_generic_from_text` for bracketed type expressions embedded in declarator text (such as `std::vector<int>`).

The base container is extracted recursively as τ_{base}, and each parameter is extracted as $\tau_i \in \vec{\tau}_{args}$, constructing TypeRef::Generic($\tau_{base}, \vec{\tau}_{args}$).

4. **Direct identifiers**: If the node itself is an `identifier`, `type_identifier`, `scoped_identifier`, `scoped_type_identifier`, `field_access`, `field_expression`, or `attribute`, its text is extracted directly and split into a qualified name.

5. **Named fields**: The node's children are inspected for named fields `type`, `return_type`, `field_type`, `value_type`, and `right`. If found, the field's node is recursively processed with `extract_type_ref`.

6. **Child identifier search**: All children are iterated, looking for nodes of kind `type_identifier`, `primitive_type`, `identifier`, or `type`. A guard rejects children whose text contains whitespace, to avoid capturing complex expressions.

7. **Textual fallback**: The node text is scanned for `:` or `->` characters, and the substring after the delimiter is parsed as a qualified name.

8. **Failure**: If all strategies fail, `TypeRef::Failed(vec![])` is returned.

Parameter Extraction

The function `extract_parameters` locates the `parameters` named field and iterates over its children of type "parameter". For each parameter, it extracts:

- The parameter name via `extract_identifier`.
- The parameter type via `extract_type_ref`.
- The variadic flag, set to `true` if the parameter text contains `...` or `*args`.

A **sanitization step** is applied to prevent a common false positive in dynamically typed languages: if the extracted type text matches the parameter name exactly (e.g., `self` in Python, where the parameter has no type annotation), the extractor recognizes that the fallback captured the parameter name instead of a type, and replaces the result with `TypeRef::Failed`. This prevents untyped parameters from generating spurious dependencies.

Type Parameter and Constraint Extraction

Generic type parameters and template variables are extracted by the function `extract_type_parameters`, constructing an ordered vector of TypeParameter tuples ($\theta = \langle \iota, \vec{\tau}_{bounds} \rangle$). The extraction pipeline operates in four coordinated steps:

1. **Container Discovery**: The extractor searches for type parameter lists under named fields (`type_parameters`, `parameters`) or child node kinds (`type_parameter_list`, `template_parameter_list`). For C++, if a class or function definition is enclosed within a `template_declaration` node, the template parameters are harvested from the parent container.
2. **Parameter and Inline Bound Extraction**: Each child parameter node is decomposed by `extract_single_type_parameter`. The parameter identifier ι is extracted from the `name` field or inner `type_identifier`. Any immediate bounds (such as Java `T extends Serializable` or Rust `T: Display + Clone`) are parsed into constraint type references $\vec{\tau}_{bounds}$.
3. **Trailing Constraint Clauses**: For languages that separate bounds from the parameter list (such as Rust `where` clauses), `merge_clause_bounds` traverses the `where_clause` node, matching each `where_predicate` target against the extracted parameters and appending the specified trait bounds to the corresponding $\vec{\tau}_{bounds}$.
4. **Generic Base Class Extraction**: Where type parameters on classes are declared by inheriting from `Generic[T, U]`, `extract_generic_base_params` inspects the superclass list to extract the type variables as formal type parameters on the structured type.

Super-Type Extraction

The function `extract_super_types` searches for inheritance references in a variety of named fields across languages:

- Java: `super_type`, `extends`, `implements`, `interfaces`
- Python: `superclass`, `superclasses`
- C++: `base_class_clause` child nodes
- Rust: trait bounds in type parameter lists

For each found node, it traverses the children looking for `type_identifier`, `identifier`, `scoped_type_identifier`, and type lists, extracting each as a `TypeRef::Unresolved`.

The Generic List Combinator

The function `extract_list_of<T, F>` is a higher-order combinator that takes a parser function `F` and applies it to all children of a node. If a child is not matched by the parser but is a container node (its kind contains "body", "list", "block", or "declaration"), the combinator descends recursively into that container.

A guard condition prevents descent into function bodies: this ensures that when extracting fields of a class, local variable declarations inside methods are not mistakenly extracted as class fields. This combinator is used to implement:

- `extract_fields`: Extracts field declarations using an inline field parser.
- `extract_methods`: Extracts methods using `try_parse_function`.
- `extract_nested_types`: Extracts nested type definitions using `try_parse_structured_type`.

6.6 Workspace Orchestration

While `extract_from_cst` operates on individual files, the analyzer is designed for multi-file projects. The workspace orchestration proceeds as follows:

1. The workspace directory is recursively scanned using `walkdir`. Files are filtered by extension to determine the source language (`.rs`, `.java`, `.py`, `.c`, `.cpp`, `.h`, `.hpp`, etc.).
2. Each file is independently extracted via `parse_source`, which invokes `extract_from_cst` and then applies the Module Grouping Strategy.
3. All extracted component trees are accumulated into the global component forest $\vec{\mathcal{M}} \in \mathcal{D}$. Primitive registries from each language are merged into a combined registry.
4. Only after all files have been extracted and unified does the system invoke the subsequent pipeline phases. This design ensures that cross-file dependencies are resolved transparently.

6.6.1 Module Grouping Strategies

The module grouping strategy determines how the resulting flat output of `extract_from_cst` is organized into the hierarchical module tree. The choice of strategy is controlled by the `ModuleConfig` section of the language configuration:

- **Directory-Based** (Python, Rust): The file's relative path within the workspace is used to construct a nested module hierarchy. Common root prefixes (`src`, `lib`, `include`) are stripped, and the file stem becomes the leaf module name. For example, the file `src/core/utils.py` produces the module path $\langle$`core`, `utils`$\rangle$.
- **Package-Declaration-Based** (Java): The file-level package statement (e.g., `package com.example.service;`) determines the module hierarchy. File-scoped imports are distributed to all top-level structured types to ensure that each class carries its own import context, preventing cross-class import leakage.
- **Namespace / Inline Module** (C++, Rust): Namespace or inline module blocks are parsed during CST traversal and directly create sub-modules in the IR tree. No post-hoc grouping is needed.
- **Flat / Fallback** (C): All components are placed into a single root module.

Table 6.3 summarizes the strategy assignment for each supported language.

6.6.2 Out-of-Line Method Linking

In C++, methods are frequently declared inside a class header (`.hpp`) and defined out-of-line in a separate source file (`.cpp`) using qualified names (e.g., `void MyClass::myMethod() { ... }`). Since extraction processes files independently, these definitions are initially extracted as free functions with multi-part qualified names.

Grouping Strategy	Python	Rust	Java	C++	C
File-Based	✓	✓			
Directory-Based	✓	✓			
Package-Declaration			✓		
Namespace				✓	
Inline Module		✓			
Flat (Root Module Only)					✓

Table 6.3: Module grouping strategy assignment for each supported language.

During module grouping, the function `link_out_of_line_methods` performs a universal post-extraction linking step:

1. Free functions with qualified names (names containing more than one segment) are identified and removed from the module's free function list.
2. For each such function, the class prefix is extracted and matched against the structured types in the current module.
3. If the target class is found in the same module, the method is stripped of its class prefix and appended to the class's method list.
4. If the target class is not found (because the declaration and definition are in separate files), a synthetic `ImplBlock` is created with the target type set to a `ResolutionQuery(Find(ClassName))`. This defers the linking to Phase 2, where the name resolution engine will locate the class across file boundaries.
5. The process is applied recursively to all sub-modules.

This approach unifies the handling of Rust `impl` blocks and C++ out-of-line methods under the same `ImplBlock` abstraction, reinforcing the language-agnostic design of the system.

Chapter 7

Name Resolution

7.1 Overview

The name resolution phase transforms the symbolic type references ($\tau =$ `Unresolved`) produced by the extraction phase into semantically resolved pointers. Formally:

$$\rho : \mathcal{D} \to \mathcal{D}_{\text{res}}$$

where $\mathcal{D}$ is the domain of extracted component hierarchies $\vec{\mathcal{M}}$, and $\mathcal{D}_{\text{res}}$ represents the domain of semantically resolved component trees. The transformation preserves the isomorphic structural topology of the codebase, while type references undergo a state transition: every `TypeRef::Unresolved` is lowered through the Query Algebra ($\mathcal{Q}$) and evaluated into one of the six terminal or compound post-resolution states (as formalized in Chapter 4 and Chapter 5): `Resolved`, `External`, `Primitive`, `Union`, `EvaluatedAccess`, or `Failed`. This phase operates entirely on the IR domain and is independent of the source language.

The resolution operator is decomposed into two algebraic sub-phases:

$$\rho = \rho_{\text{exec}} \circ \rho_{\text{build}}$$

1. **Query Building** (ρ_{build}): Traverses the module hierarchy with a local *Symbol Stack* (Δ), replacing raw `Unresolved` paths with algebraic `ResolutionQuery` expressions ($q \in \mathcal{Q}$).

2. **Query Execution** (ρ_{exec}): Builds a global *Scope Tree* ($\mathcal{E}$) from the entire workspace and evaluates all query expressions against the immutable Execution Context $\Gamma = \langle \mathcal{E}, \mathcal{P}, \mathcal{K} \rangle$.

In classical compiler construction and program analysis, resolving symbolic names across heterogeneous source files presents substantial theoretical and practical hurdles due to forward references, cyclic module imports, and diverse lexical scoping conventions. In recent formal language engineering, language-independent name binding has been grounded in the theory of *Scope Graphs* [33] and industrially scaled in GitHub's *Stack Graphs* [34], both of which abstract name resolution as generalized path-finding over graph-structured lexical scopes.

OmniDeps adopts a pragmatic, two-phase algebraic formulation specifically optimized for static architectural dependency analysis. Rather than resolving raw strings eagerly during file parsing or constructing monolithic global constraint systems:

- **Decoupling Intra-procedural State (ρ_{build})**: During syntactic extraction (Phase 1), analysis is strictly isolated file-by-file without visibility into external units. The query building phase resolves local concerns (formal parameter types, local variables, and block-level lexical shadowing). By rewriting syntactic references into self-contained algebraic formulas $q \in \mathcal{Q}$, ρ_{build} eliminates all transient intra-procedural state. Once this phase completes, the Symbol Stack is safely deallocated, and all method bodies are represented uniformly by closed algebraic expressions.
- **Global Topological Resolution (ρ_{exec})**: Because intra-procedural substitutions have already taken place, the execution engine receives pure navigation formulas. Operating on the unified workspace Scope Tree $\mathcal{E}$, ρ_{exec} handles inter-procedural references, cross-module imports, and class inheritance hierarchies in a single, well-defined topological traversal, completely unencumbered by source file ordering or local lexical complexities.

Figure 7.1 summarizes the operational flow between these two phases.

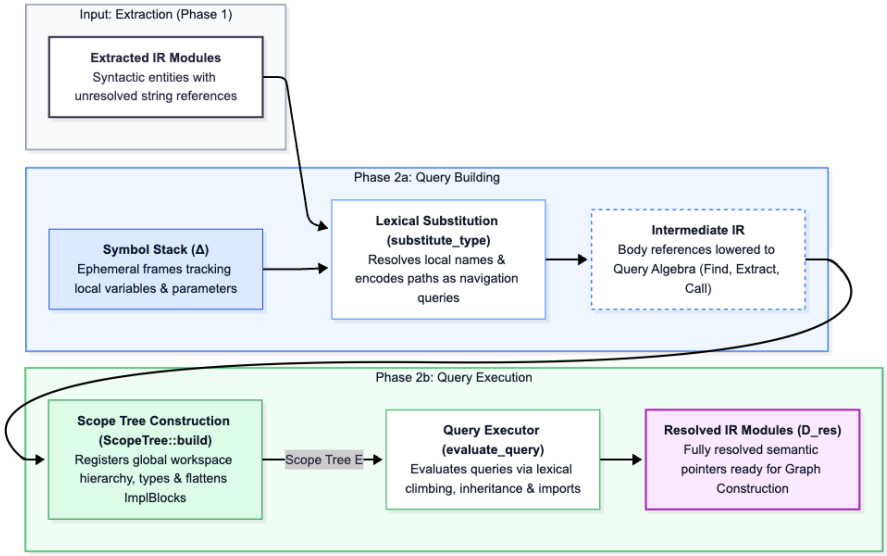


Figure 7.1: Two-phase resolution architecture: query building (ρ_{build}) transforms AST paths into algebraic queries using the local Symbol Stack (Δ), while query execution (ρ_{exec}) evaluates queries across the global Scope Tree ($\mathcal{E}$).

7.2 The Query Algebra ($\mathcal{Q}$)

Rather than resolving raw strings eagerly, the system encodes resolution intent as an algebraic formula $\mathcal{Q}$ composed of three navigation primitives:

- `Find`(ι): **Ascending search**. Starting from the current lexical scope, climb up the scope tree until a symbol named $\iota \in \mathcal{I}$ is found. This models lexical scope resolution, where a name is first searched locally, then in enclosing scopes, then globally.
- `Extract`(q, ι): **Structural member descent**. Evaluate sub-query q to obtain a scope (a module or a type), then search for member $\iota \in \mathcal{I}$ within that scope, including inherited members from super-types. This models qualified access paths such as `obj.field`, `Module::Type`, or `package.Class`.
- `Call`(q): **Transitive invocation / Return type evaluation**. Evaluate sub-query q to obtain a callable symbol (function, method, or constructor), then resolve to its declared or inferred return type. This models return type propagation during local variable type inference (e.g., `let x = f()`) as well as fluent method chaining patterns where the returned instance serves as the receiver of a subsequent member access.

These three primitives are sufficient to encode the resolution intent of any type reference that appears in practice. Table 7.1 illustrates several representative encodings.

Source Expression	**Type-Level Query Encoding**
`Foo`	`Find(Foo)`
`a.b.T`	`Extract(Extract(Find(a), b), T)`
`foo()` (return type)	`Call(Find(foo))`
`obj.method()` (return type)	`Call(Extract(Find(obj), method))`
`a.f().g()`	`Call(Extract(Call(Extract(Find(a), f)), g))`

Table 7.1: Examples of source expressions and their corresponding query algebra encodings for type inference and member chaining.

A crucial architectural distinction must be emphasized between *behavioral invocation dependencies* and *type-level invocation queries*:

- In call graph construction (Phase 3), a function call statement in a block body (e.g., `validate()`) constitutes a control-flow coupling. Here, the target of the call must resolve to the **callable entity itself** (`Find(validate)`), allowing OmniDeps to emit a direct `Calls` edge between the caller and the callee. Wrapping such statements in `Call`($\cdot$) would evaluate their return type (yielding e.g. `void` or `bool`), which would erroneously redirect call edges to return types.
- In contrast, the algebraic constructor `Call`(q) is utilized strictly in **type inference contexts**: when an untyped local variable is initialized with a function call (`let x = build()`) or when chaining member accesses across invocations (`conn.get().open()`).

7.3 Phase 2a: Query Building (ρ_{build})

The first sub-phase traverses the extracted module hierarchy and transforms every `TypeRef::Unresolved` into a `TypeRef::ResolutionQuery`. The transformation is performed by the function `substitute_type`, which applies the *Symbol Stack* (Δ) to resolve local variable references and encode the remaining path segments as `Extract` and `Call` queries.

7.3.1 The Symbol Stack (Δ)

The Symbol Stack is an ephemeral lexical environment that tracks local variable and parameter bindings within function scopes, adhering to the classical compiler formulation of block-structured lexical environments with scoped symbol tables [16]. It is structured as a stack of frames:

$$\Delta = [\delta_1, \delta_2, \ldots, \delta_n] \quad \text{where} \quad \delta_i : \mathcal{I} \to \mathcal{Q}$$

Each frame δ_i maps identifier names $\iota \in \mathcal{I}$ to their corresponding query expressions. Frames are pushed when entering a function or block scope, and popped when exiting. Lookup proceeds from the top of the stack downward (most-recently-pushed frame δ_n first), naturally modeling lexical shadowing.

7.3.2 The Substitution Algorithm

The lexical substitution procedure transforms unresolved syntactic references into algebraic queries via the function `substitute_type(ctx, tr, is_call)`. When encountering a `TypeRef::Unresolved` consisting of a qualified path $\pi = \langle \iota_1, \iota_2, \ldots, \iota_k \rangle$, the builder proceeds as follows:

1. **Base Identifier Resolution**: Look up the leading segment ι_1 in the Symbol Stack Δ (iterating through frames from δ_n down to δ_1). If a local binding exists ($\iota_1 \mapsto q_{\text{local}}$), adopt q_{local} as the base query; otherwise, emit an ascending lexical search $\texttt{Find}(\iota_1)$.

2. **Structural Member Chaining**: For each subsequent segment ι_j ($j > 1$), wrap the accumulated query into a member extraction:

$$q \leftarrow \texttt{Extract}(q, \iota_j)$$

3. **Contextual Invocation Wrapping**: The Boolean flag `is_call` specifies whether the reference is evaluated as a return-type query:

   - For references representing *behavioral invocation targets* ($c \in b.\text{calls}$), *instantiations* ($i \in b.\text{instantiates}$), and *direct accesses* ($a \in b.\text{accesses}$), the builder invokes `substitute_type` with `is_call` = false. The query directly targets the callee symbol or accessed member, preserving the target's identity for call graph edge creation.
   - When a call expression serves as a value provider in variable type inference (e.g., extracting the inferred type of an untyped local declaration $x = f()$ in Section 6.3.2), the resulting query is wrapped in $\texttt{Call}(\cdot)$ (i.e., $q \leftarrow \texttt{Call}(q)$), signaling the executor to unpack the callee's return type.

For `TypeRef::Union` variants, the substitution is applied recursively to each member of the union.

7.3.3 Variable and Parameter Binding

As the builder traverses function bodies, it pushes bindings onto the Symbol Stack:

- **Parameters**: For each parameter with identifier ι_{param} and declared type τ, the type is transformed via ρ_{build}:

  $$q_\tau = \text{substitute_type}(\text{ctx}, \tau, \text{false})$$

  If q_τ evaluates to a `ResolutionQuery`, the binding $\iota_{\text{param}} \mapsto q_\tau$ is registered in the current frame δ_n. For variadic parameters and parameters with failed type extraction, no binding is pushed.

- **Local Variables**: For each local variable declaration ι_{var} in a block with declared or inferred type τ, the type is similarly substituted into q_τ. If $q_\tau \in \mathcal{Q}$, the binding $\iota_{\text{var}} \mapsto q_\tau$ is pushed into the top frame δ_n.

- **Block Scoping**: When entering a sub-block (an `if`, `while`, or anonymous scope), a fresh frame δ_{n+1} is pushed onto Δ. When exiting, the frame is popped. This correctly models lexical scoping, where a variable declared in an inner block shadows one in an outer block but does not outlive it.

Implicit Self/This Injection

For languages with an implicit receiver parameter, the builder injects special bindings based on the language configuration:

- If `self_keyword` is set (e.g., `"self"` for Rust, `"this"` for Java/C++), a binding $kw \mapsto q_{\text{self}}$ is pushed when entering a method scope, where q_{self} is the query targeting the enclosing structured type.

- If `implicit_first_param_as_self` is true (Python), the first parameter of each method (typically named `self`) is automatically bound to the enclosing class, regardless of its declared type.

This mechanism is what allows references like `self.field` or `this.method()` to be correctly resolved to members of the enclosing type.

7.3.4 Output of Phase 2a

At the end of the query building phase, the Symbol Stack is destroyed. Intra-procedural type references in the IR are now:

- `TypeRef::ResolutionQuery(`q`)`: An algebraic expression ready for global evaluation against the Scope Tree.

- Composite type structures (`Generic`, `TypeVar`, `Union`, and `EvaluatedAccess`) whose constituent references contain substituted queries.

No raw local identifiers remain in executable blocks. The classification of built-in primitive types and the identification of unresolvable symbols are deliberately deferred to Phase 2b, where they are evaluated globally against the Primitive Registry ($\mathcal{P}$) and the Scope Tree ($\mathcal{E}$).

7.4 Phase 2b: Query Execution ($\rho_{\mathbf{exec}}$)

The second sub-phase builds a global Scope Tree from the entire workspace and evaluates all `ResolutionQuery` expressions. The key property of this phase is that it runs **after** the entire workspace has been parsed into memory, making source file order and forward references irrelevant.

7.4.1 The Scope Tree ($\mathcal{E}$)

The Scope Tree is a hierarchical data structure that mirrors the lexical scoping rules of the source code. Each scope represents a named lexical environment (a module, a class, a function, or a block) and contains a symbol table mapping identifiers to their semantic roles.

Unlike the generalized, unguided graph traversal algorithms employed by Stack Graphs [34], which can experience severe combinatorial path-explosion during whole-program dependency extraction (as discussed in Chapter 3), the Scope Tree $\mathcal{E}$ models the workspace as a deterministic, single-parent lexical hierarchy. Lexical climbing for `Find` ascends deterministically toward the root, while member descent for `Extract` follows unambiguous nominal containment. By restricting name navigation to this deterministic tree topology, OmniDeps completely avoids non-deterministic path exploration.

Arena Allocator Implementation

The Scope Tree is implemented using an **Arena Allocator** pattern: all scopes reside in a contiguous flat vector (`Vec<Scope>`), indexed by integer `ScopeId` values (`type ScopeId = usize`). Each `Scope` structure contains:

- `id: ScopeId`: The scope's index in the arena.
- `parent: Option<ScopeId>`: A pointer to the enclosing scope.
- `name: String`: The scope's identifier (e.g., the module name, the class name).
- `symbols: HashMap<String, Symbol>`: The symbol table.
- `imports: Vec<Import>`: The import declarations visible in this scope.
- `super_types: Vec<TypeRef>`: The inheritance or trait implementation references (for type scopes).
- `is_module: bool`: Whether this scope represents a module (affects import resolution behavior).
- `language: Option<String>`: The language of the module (for configuration lookup).
- `type_parameters: Vec<String>`: The formal generic type parameter names declared by the scope (used for type substitution).
- `is_phantom: bool`: Indicates whether this scope is a synthetic phantom scope introduced for a generic type parameter.

- `children: Vec<ScopeId>`: The ordered list of child scope indices allocated under this scope node in the arena.
- `children_by_name: HashMap<String, Vec<ScopeId»`: An $\mathcal{O}(1)$ downward child index associating names with their allocated sub-scopes, enabling instantaneous sub-scope dispatch (functions, nested types, and blocks) without scanning the arena.

The `Symbol` enum distinguishes four kinds of entries:

- `Module(ScopeId)`: A sub-module, pointing to its scope.
- `Type(ScopeId)`: A structured type (class, struct, interface, trait), pointing to its scope.
- `TypeAlias(TypeRef)`: A type alias, storing its target reference.
- `Value(TypeRef)`: A concrete value (field, function, parameter, local variable), storing its type reference.

The Arena Allocator pattern represents a fundamental systems-level architectural choice in OmniDeps. In garbage-collected languages, hierarchical scope structures are typically modeled using bidirectional object references between parent and child scopes. In Rust, replicating such pointer graphs naively would necessitate shared mutable pointer cycles, such as `Rc<RefCell<Scope»`. This introduces several critical limitations:

1. **Memory Leaks via Reference Cycles**: Bidirectional links between parent scopes and child scopes naturally create strong reference cycles that prevent automated deallocation, requiring fragile manual cleanup routines or complex weak pointer schemes (`Weak<RefCell<Scope»`).
2. **Borrow Checker Violations and Runtime Overhead**: Climbing parent scopes or inspecting enclosing lexical frames during recursive traversals would trigger dynamic borrow checking (`RefCell::borrow_mut`), introducing runtime performance penalties and the constant hazard of runtime panics due to conflicting concurrent borrows.
3. **Cache Locality**: Heap-allocated node graphs scatter individual scope structures across disjoint memory regions, degrading CPU cache line utilization during intensive symbol lookup phases.

By storing all scopes in a flat, contiguous vector (`Vec<Scope>`) and modeling pointers as plain scalar indices (`type ScopeId = usize`), the Arena Allocator completely decouples lexical graph topology from Rust's memory ownership model. Each `ScopeId` operates as an implicit relative pointer, representing an offset into the arena buffer that guarantees instantaneous $\mathcal{O}(1)$ random access. Because the compiler treats these scalar indices as copyable primitive values rather than tracked memory references, this implicit pointer architecture provides key guarantees:

- The entire Scope Tree $\mathcal{E}$ is owned by a single contiguous memory container with $O(1)$ amortized allocation and deterministic deallocation, completely eliminating reference counting overhead;

- The parent relation `parent: Option<ScopeId>` is a primitive 64-bit integer, inherently immune to circular reference memory leaks;
- Traversal algorithms can simultaneously borrow the arena immutably while traversing parent and child indices freely, ensuring absolute compile-time memory safety without locks or runtime checks;
- Canonical path reconstruction (computing the fully qualified path π of any entity) is achieved by climbing ancestor `ScopeId` integers with zero pointer chasing overhead.

Beyond scalar parent pointers, the Scope Tree maintains auxiliary index structures to guarantee $\mathcal{O}(1)$ performance during traversal:

- **Localized Child Indexing (`children_by_name`)**: Each scope indexes its immediate descendants by name. When the execution phase resolves sub-blocks, functions, or member definitions, matching sub-scopes are retrieved in constant time $\mathcal{O}(1)$, completely bypassing any linear scanning over the arena.
- **Workspace Nominal Type Index (`type_scopes_by_name`)**: The root `ScopeTree` container maintains a global hash map mapping structured type names to their corresponding `ScopeId` allocations. This provides instantaneous $\mathcal{O}(1)$ resolution during cross-module out-of-line method linking and implementation block flattening, eliminating potential $\mathcal{O}(N^2)$ whole-arena bottlenecks on enterprise-scale projects containing hundreds of thousands of scopes.

Figure 7.2 illustrates this arena-allocated topology.

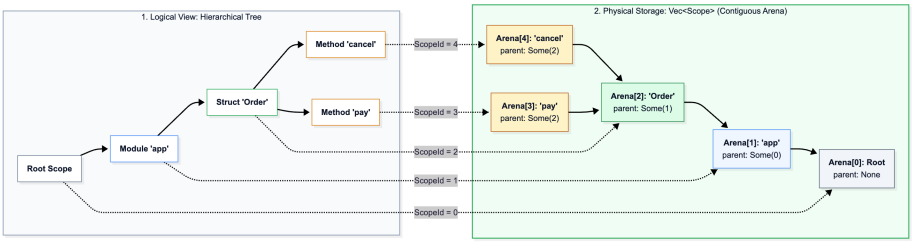


Figure 7.2: Contiguous arena layout of the Scope Tree: scopes reside linearly in `Vec<Scope>` indexed by integer `ScopeId`, with parent relations modeled as scalar integers to eliminate pointer cycles.

Scope Tree Construction

The Scope Tree is constructed by the function `ScopeTree::build`, which recursively registers all components from the workspace modules:

1. **Root**: A root scope (id: 0, name: `"root"`) is created.
2. **Modules**: `register_module` walks module paths, creating intermediate module scopes or merging with existing ones (if the same module path was

contributed by multiple files). Imports, type aliases, structured types, free functions, free variables, and sub-modules are registered within the module scope.

3. **Structured Types**: `register_structured_type` creates a type scope, registers fields as `Symbol::Value`, records super-types, and recursively handles nested types. Methods are registered as callable symbols in the class scope. Constructors and destructors are excluded from the class's dynamic symbol lookup table to prevent shadowing the nominal type of the class during base-class climbing in object-oriented languages (such as C++). Instead, constructors and destructors are registered as independent sub-scopes and linked via structural `NestedIn` relationships, ensuring that their parameters, bodies, and calls are fully analyzed without contaminating type queries.

4. **Type Parameters and Phantom Scopes**: When registering a generic structured type or function, `register_structured_type` creates synthetic placeholder scopes, termed *Phantom Scopes* (Σ_{phantom}), for each generic type parameter $\theta = \langle \iota, \vec{\tau}_{bounds} \rangle$. The parameter identifier ι is registered as `Symbol::Type`(TypeRef::TypeVar($\iota, \vec{\tau}_{bounds}$)) in the enclosing scope, and the phantom scope retains $\vec{\tau}_{bounds}$ as its super-types, enabling bound-based member dispatch.

5. **Functions and Blocks**: `register_function` and `register_block` create hierarchical scopes for parameters, local variable declarations, and nested code blocks (`block_0`, `block_1`, etc.).

6. **Implementation Blocks**: `ImplBlock` registrations are **deferred** into a `pending_impl_blocks` list. After all types are declared, these are processed: the target type is located in the tree (via local parent symbols or through the global nominal index `type_scopes_by_name` in $\mathcal{O}(1)$ time if not found in the immediate parent), and the block's methods, nested types, and type aliases are flattened into the target type's scope.

7.4.2 The Execution Context (Γ)

Query evaluation operates within an Execution Context:

$$\Gamma = \langle \mathcal{E}, \mathcal{P}, \mathcal{K}, \mathcal{M} \rangle$$

where $\mathcal{E}$ is the Scope Tree, $\mathcal{P}$ is the Primitive Registry, $\mathcal{K}$ is the language configuration, and $\mathcal{M} : \Sigma \to 2^{\Sigma}$ represents the memoization mapping of resolved super-scopes.

Although $\mathcal{M}$ is maintained with interior mutability in the implementation for runtime efficiency, query evaluation remains referentially transparent and idempotent. Because the structure of $\mathcal{E}$ is fixed prior to Phase 2b, the ancestor scope set $\mathcal{M}(\Sigma)$ for any scope Σ is deterministic and strictly independent of the order in which individual type references are evaluated.

7.4.3 Lexical Scope Climbing (`Find`)

Figure 7.3 depicts the general evaluation pathways for resolution queries across lexical climbing, structural member extraction, and call unwrapping.

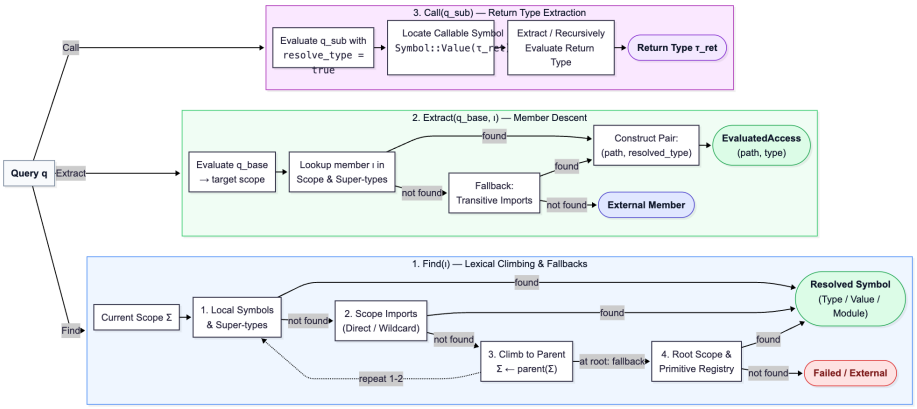


Figure 7.3: Evaluation pathways of resolution queries: parallel execution tracks for lexical climbing (`Find`), structural member descent (`Extract`), and return type extraction (`Call`).

Cycle detection in lexical climbing and symbol evaluation is tracked via composite keys combining the active scope identifier and the symbol name:

$$k_{\text{query}} = \langle \Sigma, \text{base}(q) \rangle \in \Sigma \times \mathcal{I}$$

and symbol keys:

$$k_{\text{sym}} = \langle \Sigma, \iota \rangle \in \Sigma \times \mathcal{I}$$

This contextual scoping prevents false cycle alarms across homonymous symbols in unrelated scopes, while reliably terminating self-referential assignments (such as Python $x = x[k]$) and cyclic re-exports.

When evaluating a `Find`(ι) query starting from scope Σ_0, the executor implements the following algorithm:

1. **Special Keywords**: If ι is `super` or `super()`, invoke the super-type resolver. If ι is `Self`, resolve to the target type of the current `impl` block context.

2. **Local Symbol Search**: Search Σ_0's symbol table for ι. If found, resolve the symbol (see below). Additionally, search inherited scopes via super-types (Section 7.4.4).

3. **Import Search**: Iterate over Σ_0's imports:

   - If an import path ends with ι (direct import), attempt to resolve the full path globally via `find_global`. If the path is not found in the workspace, classify it as `TypeRef::External`.
   - If an import is a wildcard (`*`), construct a specific path by replacing the wildcard with ι and attempt global resolution.

4. **Climb**: Set $\Sigma_0 \leftarrow \text{parent}(\Sigma_0)$ and repeat from step 2.

5. **Root Fallback**: If the root scope is reached without a match, attempt a global search from the root using `find_global`.

6. **Primitive Check**: If the global search also fails, consult the Primitive Registry ($\mathcal{P}$). If ι is a known primitive type for the current language, return `TypeRef::Primitive(`ι`)`.

7. **Failure**: Return `TypeRef::Failed`.

Global Path Navigation: `find_global`

The function `find_global` resolves absolute qualified paths by navigating from the root scope downward. For a path $\pi = \langle \iota_1, \iota_2, \ldots, \iota_k \rangle$, it looks up ι_1 in the root scope, navigates to the resulting scope, looks up ι_2, and so on. If at any point a segment is not found, the function returns `None`, and the caller may classify the reference as `External`.

A special case handles single-segment paths: if $k = 1$ and ι_1 is recognized by the Primitive Registry, the function returns `TypeRef::Primitive` directly.

7.4.4 Member Extraction and Inheritance (`Extract`)

When evaluating an `Extract`(q, ι) query, the executor first evaluates q to obtain a target scope, then searches for member $\iota \in \mathcal{I}$ within that scope. If ι is not found directly, the search proceeds through the type's super-type hierarchy.

When evaluating super-type references of a structured type with scope Σ_C, resolution initiates from Σ_C itself so that formal type parameters (e.g., T) declared locally are accessible. To prevent combinatorial explosion in multiple inheritance hierarchies (such as classes implementing dozens of interfaces, where naive recursive exploration incurs factorial complexity $\mathcal{O}(b!)$), the resolver employs a memoization sentinel:

$$\mathcal{M}(\Sigma_C) \leftarrow \emptyset$$

Before recursively exploring the base types of Σ_C, an empty set is inserted as a sentinel in $\mathcal{M}(\Sigma_C)$. Any re-entrant inquiry for Σ_C's super-scopes during base-type evaluation immediately terminates in $\mathcal{O}(1)$, allowing lexical climbing to ascend directly to parent(Σ_C) to find base types without divergence. Once direct super-types are evaluated, $\mathcal{M}(\Sigma_C)$ is updated with the resolved ancestor scopes. A decoupled visitation set isolates super-scope exploration from the caller's query stack. This reduces inheritance traversal from exponential complexity to a linear depth-first search over ancestor scopes:

$$\mathcal{O}(|V_{\text{supers}}| + |E_{\text{supers}}|)$$

guaranteeing instant convergence even on complex multi-interface hierarchies.

The function `find_symbol_in_scope_and_supers` implements the following algorithm:

1. Search the target scope's symbol table for ι. If found, return the resolved symbol.

2. Retrieve or resolve the ancestor scopes via $\mathcal{M}(\Sigma)$.

3. For each ancestor scope in depth-first order:

   (a) Search the ancestor scope's symbol table for ι.

(b) If found, return the resolved symbol.

4. A `visited` set prevents infinite loops in the presence of diamond inheritance or circular trait dependencies.

Nominal Projection Fallback for External and Dynamic Members If member ι is not found within the candidate scopes, their super-type hierarchies, or through transitive imports, the executor avoids prematurely discarding the reference. When the base receiver evaluates to a known path (Resolved(π), External(π), or Unresolved(π)), the executor synthesizes an updated reference by appending ι to the path:

$$\pi' = \pi \cup \langle \iota \rangle$$

producing EvaluatedAccess(Base(π), Base(π')). This conservative fallback guarantees that references to external library members (such as `requests.Session.get`) or unannotated dynamic attributes are preserved, enabling Phase 3 to project valid architectural dependency edges toward external endpoints.

Rust Deref Coercion as Inheritance

Rust does not have classical class inheritance, but the `Deref` trait provides a form of implicit type coercion [36, Chapter 15]: if a type `T` implements `Deref<Target = U>`, then methods and fields of `U` can be called directly on values of type `T`. To model this within the resolution framework, the Scope Tree construction detects `Deref` implementations:

When registering an `ImplBlock` that implements the `Deref` trait and contains a type alias named `Target`, the target type is added to the implementing type's `super_types` list. This allows the standard inheritance resolution algorithm to search through `Deref` targets, effectively treating `Deref` coercion as a form of structural inheritance.

Transitive Import Resolution in Modules

When the `transitive_imports` configuration flag is enabled (e.g., for Python), the resolver can follow import chains through intermediate modules. This models the common Python pattern of importing and re-exporting names through `__init__.py` files.

The function `resolve_via_transitive_imports` is invoked when a member ι is not found directly in a module scope. It inspects the module's imports for entries that match ι (either by name or via a wildcard). If a matching import is found, the import path is resolved globally. If the path points to a known workspace entity, the resolution succeeds; otherwise, the reference is classified as `External`.

Wildcard imports are handled by removing the terminal `*` token before appending the queried member name ι. Mutual recursion between global path lookup and transitive import exploration is prevented by maintaining dedicated visitation guards: a path set $V_{\text{global}} \subseteq \Pi$ for global navigation and a composite set $V_{\text{trans}} \subseteq \Sigma \times \mathcal{I}$ for module-member re-exports.

This mechanism is controlled per-language: it is enabled for Python (where re-exports are idiomatic) and disabled for languages like Java and Rust (where import semantics are strictly non-transitive).

7.4.5 Call Query Evaluation (`Call`)

The evaluation of invocation queries in Phase 2b mirrors the operational distinction between type-level queries and behavioral call targets:

1. **Type-Level Evaluation (`Call`(q))**: When evaluating an explicit `Call`(q) query, the executor calls `evaluate_query` with the flag `resolve_type` = true. The sub-query q is evaluated to locate the callee's symbol in the Scope Tree. When a callable symbol is found (represented as `Symbol::Value`(τ_{ret}) carrying the declared or inferred return type), the executor unpacks and returns τ_{ret} rather than the callee's own identifier path. If τ_{ret} is itself a deferred `ResolutionQuery`, it is evaluated recursively within the callee's scope. This enables seamless type propagation across method chains of arbitrary depth (e.g., `repo.find().as_dto()`).

2. **Behavioral Call Target Resolution (b.calls)**: Conversely, when evaluating the statement-level invocation targets of an executable block ($c \in b$.calls), the executor invokes `evaluate_typeref` with `resolve_type` = false. Under this setting, locating a callable symbol preserves its canonical symbol path (e.g., `Resolved`([app, OrderService, validate])) instead of collapsing resolved target is subsequently passed through constructor redirection (Section 7.4.5) and semantic cast disambiguation (Section 7.4.5) to handle class instantiations or type casts masquerading as calls, and finally emitted as an edge in the dependency graph during Phase 3.

Constructor Redirection

A subtle but vital semantic discrepancy in static analysis occurs during object instantiation and constructor execution. In source code across major programming languages, object creation is syntactically expressed by invoking or referencing the structured type itself:

- In Java and C++: `new Car(engine)` or direct stack initialization `Car car(engine)`;
- In Python: `car = Car(engine)`;
- In JavaScript / TypeScript: `new Car(engine)`.

During syntactic extraction (Phase 1), the expressions recorded in `Block::instantiates` and `Block::calls` capture the nominal identifier of the target class (e.g., Unresolved($\langle$`Car`$\rangle$)). When Phase 2 evaluates this query, standard lexical lookup initially yields the structured type itself: Resolved($\langle$`models`, `Car`$\rangle$).

However, from an architectural and behavioral standpoint, invoking an object constructor is fundamentally an invocation of executable behavior rather than merely a passive type-level reference. If the resolution engine terminated at the class node, Phase 3 (Graph Construction) would only be able to emit a generic type dependency, obscuring the direct call coupling toward the constructor method.

To bridge this syntactic gap, the function `redirect_to_constructor` inspects all resolved references in `Block::calls` and `Block::instantiates`:

1. If the reference evaluates to a structured type (Resolved(π)), the executor retrieves the corresponding type scope in the Scope Tree $\mathcal{E}$ via `find_scope_for_type`.

2. It checks the scope's symbol table for known constructor naming conventions across the supported languages:

$$\mathcal{I}_{\text{ctor}} = \{\texttt{"__init__"}, \texttt{"constructor"}, \text{last}(\pi)\}$$

   capturing Python's `__init__`, TypeScript's `constructor`, and Java/C++ constructors that match the class name itself.

3. If an explicit constructor method is found, the resolved path is specialized by appending the constructor identifier:

$$\pi_{\text{ctor}} = \pi \cup \langle \iota_{\text{ctor}} \rangle$$

   transforming Resolved($\langle$`models`, `Car`$\rangle$) into Resolved($\langle$`models`, `Car`, `__init__`$\rangle$).

4. If the reference is a compound EvaluatedAccess($\tau_{\text{base}}, \tau_{\text{inner}}$), the redirection recursively applies to τ_{inner}, preserving the base access while updating the resolved method target.

5. If the class defines no explicit constructor (relying on synthesized or default language semantics), the reference remains unaltered, pointing directly to the structured type.

This mechanism allows Phase 3 to cleanly differentiate between pure type-level instantiations and explicit behavioral constructor calls, emitting `DependencyKind::ConstructorCall` whenever a dedicated constructor method is invoked.

Disambiguating Function Invocations and Type Coercions

A notorious grammatical challenge in parsing languages like C is the syntactic ambiguity between function calls and type casts [16]. Consider a parenthesized expression of the form:

$$(T)(e) \tag{7.1}$$

In traditional C compiler frontends (such as GCC or Clang), this ambiguity is resolved during lexical analysis via the classic "lexer hack": the scanner queries the symbol table and yields distinct token types (`TYPE_NAME` versus `IDENTIFIER`) depending on whether T has been registered as a `typedef`.

In contrast, Tree-sitter parses source files in isolation without lexical feedback from an active symbol table. Consequently, Tree-sitter's context-free parsing tables interpret $(T)(e)$ as a `call_expression` whenever T is an arbitrary typedef identifier or when the operand e is itself parenthesized, classifying (T) as a parenthesized callable function and (e) as its argument list.

Because OmniDeps strictly separates local syntactic extraction (Phase 1) from global semantic resolution (Phase 2), the framework does not attempt to resolve this ambiguity through ad-hoc syntactic heuristics during CST traversal. Instead, Phase 1 faithfully unrolls the parenthesized callee (recording T in

$\mathcal{B}$.calls), deferring disambiguation to the query execution phase (ρ_{exec}) where the global Scope Tree $\mathcal{E}$ is fully materialized.

During Phase 2b, the resolution engine inspects every call target $c \in \mathcal{B}$.calls. Crucially, rather than embedding hardcoded language switches (e.g., `if is_c`), OmniDeps addresses this variance through its declarative resolution configuration $\mathcal{K}$ (Section 7.7):

- In object-oriented languages like Python and C++, types are first-class callable entities (`callable_types = true`), where invoking a class syntactically executes an instance constructor.
- In procedural languages like C, types are fundamentally non-callable entities (`callable_types = false`): C possesses neither user-defined constructors nor callable operator overloading (`operator()`).

Formally, let $\tau = \rho_{\text{exec}}(c, s)$ be the evaluated call target within lexical scope s, and let $\mathcal{K}_s = \mathcal{K}(\text{language}(\mathcal{E}, s))$ be the active resolution configuration. The engine evaluates:

$$\text{is_type_or_alias}(\tau) = \begin{cases} \text{true} & \text{if } \tau \in \mathcal{P} \vee \text{lookup}(\mathcal{E}, \tau) \in \\ & \quad \{\text{Symbol::Type}, \text{Symbol::TypeAlias}\} \\ \text{false} & \text{otherwise} \end{cases} \tag{7.2}$$

where $\mathcal{P}$ is the Primitive Registry and $\text{lookup}(\mathcal{E}, \tau)$ retrieves the symbol declaration from the Scope Tree. If $\neg \mathcal{K}_s$.`callable_types` and $\text{is_type_or_alias}(\tau)$ holds true, the reference cannot semantically represent an executable function call. The executor reclassifies the reference as a type cast:

$$\mathcal{B}.\text{type_casts} \leftarrow \mathcal{B}.\text{type_casts} \cup \{\tau\}, \quad \mathcal{B}.\text{calls} \leftarrow \mathcal{B}.\text{calls} \setminus \{\tau\} \tag{7.3}$$

This parameter-driven disambiguation guarantees that Phase 3 emits a precise `DependencyKind::CastsTo` edge for expressions such as `(MyInt)(val)` or `(int)(5.0)`, while genuine function calls enclosed in protective parentheses (such as `(add)(a, b)` or `(square_func)(a)`) resolve to Symbol::Value and correctly remain in $\mathcal{B}$.calls. Furthermore, because this behavior is governed declaratively by $\mathcal{K}$, the core resolution engine remains strictly language-agnostic.

Dual-Tracking of Access Chains: `EvaluatedAccess`

A fundamental architectural challenge in static dependency extraction arises when resolving chained member expressions (such as field dereferences `order.customer.address` or method invocations `conn.getMetaData().getTables()`). Such expressions impose a dual semantic requirement:

1. **Navigational / Typing Requirement**: To iteratively resolve subsequent segments in an access chain (e.g., finding the member `address` within `customer`), the static engine requires the *structural type* of the receiver (i.e., the class `Customer`, its Scope Tree node, and its declared fields and methods).
2. **Architectural Dependency Requirement**: When projecting the resolved IR into the architectural dependency graph $\mathcal{G} = (V, E)$ in Phase

3 (Chapter 8), the analyzer requires the identity of the *concrete physical component accessed* (e.g., the field `Order::customer`), so as to emit fine-grained behavioral dependencies (`Accesses`, `Calls`) rather than coarse, uninformative type-level links.

If the resolution engine resolved an access expression solely to its underlying type (discarding the accessed field and yielding only `Customer`), the graph generator would lose the identity of the member being accessed. Conversely, if the engine returned only the accessed member symbol without its resolved type, it would lack the type context necessary to locate subsequent members in the chain.

To eliminate this trade-off, OmniDeps introduces the compound resolution state:

$$\text{EvaluatedAccess}(\tau_{\text{path}}, \tau_{\text{type}})$$

where:

- τ_{path} captures the accumulated qualified path of the concrete syntactic component accessed in the source code (e.g., Resolved(⟨`Order`, `customer`⟩));
- τ_{type} captures the evaluated structural type of that component (e.g., Resolved(⟨`Customer`⟩)), supplying the target scope for subsequent member resolution.

During query execution in `executor.rs`, `EvaluatedAccess` operates through an unwrapping and re-wrapping protocol:

- When evaluating an `Extract`(q, m) query, the executor evaluates the base sub-query q. If q yields an EvaluatedAccess$(\tau_{\text{base}}, \tau_{\text{inner}})$, the executor unwraps τ_{inner} to locate the target `ScopeId` via `find_scope_for_type`, while preserving τ_{base} as the active access prefix.
- Once member m is located within the target scope or its super-type hierarchy (producing resolved symbol τ_{res}), the executor updates the access chain:

  $$\text{EvaluatedAccess}(\tau_{\text{base}}, \tau_{\text{res}})$$

  If the base was a simple type reference rather than an existing chain, the parent itself is encapsulated as the base access.
- When resolving symbols in `symbol_to_typeref`, looking up a `Symbol::Value` or `Symbol::TypeAlias` with resolve_type = true automatically packages the symbol's declared path and its resolved type into an `EvaluatedAccess` pair.
- Global path resolution (`find_global`) similarly encapsulates the final scope into EvaluatedAccess(Resolved(π), Resolved(π)).

This mechanism bridges the semantic gap between Phase 2 and Phase 3: the graph construction engine can simultaneously project an `Accesses` or `Calls` edge to τ_{path} and a `TypedAs` dependency to τ_{type}, guaranteeing complete architectural fidelity without loss of behavioral information. Figure 7.4 traces the progressive accumulation of paths and types across a member access chain.

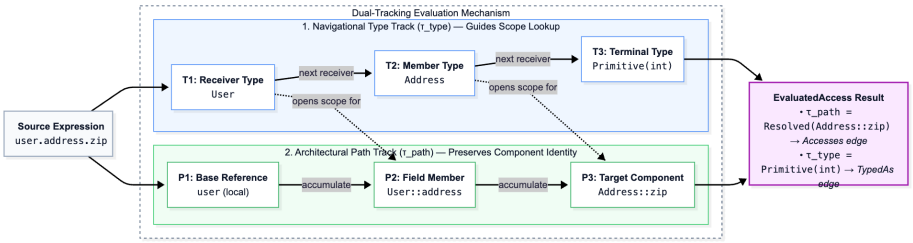


Figure 7.4: Dual-tracking evaluation of chained member access `user.address.zip`: maintaining the concrete syntactic access path alongside the resolved structural type for architectural dependency projection.

Parametric Polymorphism and Generic Resolution

Parametric polymorphism (generics in Java, Rust, and Python, or templates in C++) introduces an orthogonal dimension to static symbol resolution: it decouples the structural definition of an algorithmic abstraction from the concrete type arguments supplied at instantiation sites.

To formalize parametric polymorphism without sacrificing language agnosticism, the Intermediate Representation distinguishes between generic application and type parameter declarations:

$$\tau_{\text{app}} = \text{Generic}(\tau_{\text{base}}, \vec{\tau}_{\text{args}})$$
$$\tau_{\text{var}} = \text{TypeVar}(\iota, \vec{\tau}_{\text{bounds}})$$

Resolution of Disjunctive Types: `Union`

In heterogeneous, multi-language codebases, type references frequently exhibit disjunctive characteristics. To faithfully capture these constructs without premature approximation, the intermediate representation introduces the compound type reference:

$$\text{Union}(\vec{\tau}) = \text{Union}(\langle \tau_1, \tau_2, \ldots, \tau_k \rangle)$$

Such variants originate during syntactic extraction (Phase 1) in two primary scenarios:

1. **Explicit Union Annotations**: In dynamically typed languages with gradual typing features (such as Python's `int | str` or `Union[User, Guest]`), an identifier can evaluate to multiple distinct types at runtime.
2. **Multi-Branch Return Type Inference**: When inferring the return type of unannotated functions exhibiting multiple return statements with differing types.

During Phase 2a (Query Building), the substitution function `substitute_type` applies intra-procedural lexical substitutions recursively across each constituent member of the union.

During Phase 2b (Query Execution), query evaluation distributes homomorphically over the union variants in `evaluate_typeref`:

$$\rho_{\text{exec}}(\text{Union}(\{\tau_1, \tau_2, \ldots, \tau_k\})) = \text{Union}(\{\rho_{\text{exec}}(\tau_1), \rho_{\text{exec}}(\tau_2), \ldots, \rho_{\text{exec}}(\tau_k)\})$$

Each inner reference τ_i is evaluated independently against the Scope Tree $\mathcal{E}$ and may resolve to any valid post-resolution state (such as `Resolved`, `Primitive`, or `External`).

Finally, during Phase 3 (Graph Construction, Chapter 8), the dependency projector processes `TypeRef::Union` by flattening its underlying targets (`flat_map`). Rather than heuristically selecting a single candidate or dropping ambiguous alternatives, OmniDeps emits dependency edges toward all resolved targets present within the union. This conservative strategy ensures that all potential behavioral and structural couplings induced by polymorphic type choices are faithfully represented in the architectural dependency graph.

Phantom Scopes in the Scope Tree

When building the Scope Tree during Phase 2a, registering a generic structured type or function allocates synthetic scopes termed *Phantom Scopes* (Σ_{phantom}) within the arena for each declared type parameter $\theta = \langle \iota, \vec{\tau}_{\text{bounds}} \rangle$. A phantom scope is characterized by:

1. is_phantom = true, marking it as an ephemeral lexical placeholder that does not generate independent vertices during graph construction;
2. name = $\langle \iota \rangle$, binding the parameter identifier;
3. super_types = $\vec{\tau}_{\text{bounds}}$, recording the upper bound constraints (e.g., interface implementations or trait bounds).

Within the enclosing lexical scope of the generic declaration, ι is defined as `Symbol::Type`(TypeRef::TypeVar($\iota, \vec{\tau}_{\text{bounds}}$)). Consequently, method signatures and local variable declarations inside the generic entity can refer to ι as a valid type.

Two-Phase Bound Lifecycle

Constraint bounds undergo a two-phase evaluation lifecycle:

1. **Lexical Binding (Phase 2a)**: The bounds $\vec{\tau}_{\text{bounds}}$ are recorded as raw or locally substituted type references.
2. **Bound Evaluation (Phase 2b)**: During query execution, `evaluate_typeref_inner` evaluates each bound $\tau \in \vec{\tau}_{\text{bounds}}$ against the Scope Tree. If a bound resolves to a known structured type (Resolved(π) or External(π)), the phantom scope inherits the member suite of that bound.

Candidate Scopes and Multi-Bound Dispatch

When an expression accesses a member on a target whose evaluated type is TypeRef::TypeVar($\iota, \vec{\tau}_{\text{bounds}}$), member lookup cannot be restricted to a single nominal scope. Instead, the function `find_candidate_scopes_for_type` computes a set of candidate scopes:

$$\text{scopes}(\text{TypeVar}(\iota, \vec{\tau}_{\text{bounds}})) = \bigcup_{\tau_b \in \vec{\tau}_{\text{bounds}}} \text{scopes}(\tau_b)$$

This multi-scope dispatch allows variables constrained by compound bounds (for instance, $T : \text{Display} + \text{Clone}$ in Rust) to resolve members originating from any of the specified traits. If any candidate scope provides the queried member, resolution succeeds.

Type Parameter Substitution (σ)

When a generic type application $\text{Generic}(\tau_{\text{base}}, \vec{\tau}_{\text{args}})$ is accessed (e.g., calling a method on an instance of `Repository<User>`), the receiver scope is determined by evaluating τ_{base} to locate the underlying generic definition $\mathcal{S}$ with formal parameters $\vec{\theta} = \langle \theta_1, \ldots, \theta_k \rangle$.

Upon retrieving a member whose signature involves formal type parameters, the engine constructs a substitution mapping:

$$\sigma = [\theta_1.\iota \mapsto \tau_{\text{args}}[1], \ldots, \theta_k.\iota \mapsto \tau_{\text{args}}[k]]$$

The helper function `instantiate_generic_member_type` applies σ recursively across the member's return type or field type:

$$\sigma(\text{TypeVar}(\iota, _)) = \begin{cases} \tau_{\text{arg}} & \text{if } (\iota \mapsto \tau_{\text{arg}}) \in \sigma \\ \text{TypeVar}(\iota, _) & \text{otherwise} \end{cases}$$

$$\sigma(\text{Generic}(\tau_b, \vec{\tau}_a)) = \text{Generic}(\sigma(\tau_b), \vec{\sigma}(\vec{\tau}_a))$$

$$\sigma(\text{Union}(\vec{\tau})) = \text{Union}(\vec{\sigma}(\vec{\tau}))$$

Thus, if `Repository<T>` defines `find(id: ID) -> T`, evaluating `repo.find(1)` on a `Repository<User>` substitutes $T \mapsto \text{User}$, resolving the invocation's return type directly to `User`.

Chained Access Propagation

When evaluating multi-segment dot-accesses (e.g., `user.profile.settings`), the intermediate structural transitions are preserved using TypeRef::Evaluated-Access($\tau_{\text{path}}, \tau_{\text{type}}$). This composite reference retains both the evaluated path and the resolved structural type, enabling Phase 3 to emit dependency edges for every intermediate component participating in the access chain.

7.5 Implementation Block Flattening

During Scope Tree construction, implementation blocks (`ImplBlock`) are flattened into the scope of their target type. This process unifies the syntactic separation of declaration and definition (common in languages like Rust with `impl` blocks, and C++ with out-of-line method definitions) with the conceptual unity of a structured type and its member suite.

The flattening algorithm operates as follows:

1. **Target Type Identification**: Extracts the canonical target name from `ib.impl_for`.

2. **Scope Association**: Searches for the target structured type's scope within the parent module scope. If not found locally, a **global fallback**

search queries the global nominal index `type_scopes_by_name` in $\mathcal{O}(1)$ time (handling cross-unit C++ `.h`/`.cpp` separations or multi-file Rust modules without linear arena traversals).

3. **Self-Type Alias Injection**: If a `self_type_keyword` is configured for the language (e.g., `"Self"` for Rust), a `Symbol::TypeAlias` pointing to `ib.impl_for` is defined in the target type's scope.

4. **Member Transfer**: All methods declared inside the `ImplBlock` are registered as callable symbols within the target type's scope, enabling subsequent lexical and member searches to discover them directly.

5. **Associated Types and Nested Definitions**: All nested types and type aliases declared in the block are similarly absorbed into the target type scope.

6. **Trait Handling vs. Deref Coercion**:

   - Standard trait implementations (`ib.implements_trait`) are *not* treated as structural inheritance in the Scope Tree; instead, the trait reference is preserved directly on the IR `ImplBlock` node, resolved to its canonical symbol during Phase 2b (`execute_impl_block`), and used in Phase 3 (Chapter 8) to emit an explicit TargetType $\xrightarrow{\text{Implements}}$ Trait architectural dependency.
   - The sole exception is Rust's `Deref` trait: when an `ImplBlock` implements `Deref` and defines an associated type alias `type Target = U`, the target type U is appended to the implementing type's `super_types` list (Section 7.4.4). This allows the inheritance search algorithm to naturally traverse method dereferencing chains without altering general trait semantics.

Within the Scope Tree $\mathcal{E}$, implementation blocks do not maintain separate scope nodes; their member definitions are directly amalgamated into the target type's lexical scope. However, within the intermediate representation ($\vec{\mathcal{M}} \in \mathcal{D}$), the `ImplBlock` structures remain preserved as first-class architectural containers, ensuring that Phase 3 can reliably extract trait realization dependencies and method nesting relations.

7.6 The Primitive Registry ($\mathcal{P}$)

To prevent built-in types (e.g., `int`, `bool`, `String`, `void`, `usize`) from being classified as `Failed` or `External`, the analyzer maintains a data-driven Primitive Registry. The registry is loaded from a JSON file (`primitives.json`) that maps language names to lists of primitive type identifiers:

```
{
  "rust": ["i8", "i16", "i32", "i64", "u8", ..., "bool", "str", "String", ...],
  "java": ["int", "long", "float", "double", "boolean", "char", "void", ...],
  "python": ["int", "float", "str", "bool", "None", "bytes", ...],
  ...
}
```

When analyzing a multi-language workspace, the registries for all encountered languages are merged into a single combined registry. During query execution, the executor consults the registry as a last resort before marking a reference as `Failed`: if the single-segment name matches a known primitive, the reference is resolved as `TypeRef::Primitive`.

This data-driven approach allows adding primitives without modifying the resolver's code, and ensures that common types like `int` or `bool` are correctly classified regardless of the language context.

7.7 Resolution Configuration ($\mathcal{K}$)

Due to the semantic differences between supported languages, the resolution phase is parameterized by a per-language configuration. Table 7.2 summarizes the key parameters and their effects.

Parameter	**Type**	**Effect on Resolution**
`transitive_imports`	`bool`	Enables following import chains through intermediate modules (Python).
`self_keyword`	`Option<String>`	Injects a self/this binding pointing to the enclosing type in method scopes.
`self_type_keyword`	`Option<String>`	Injects a `Self` type alias in `impl` block scopes (Rust).
`implicit_first_param_as_self`	`bool`	Treats the first method parameter as an implicit self reference (Python).
`extract_dynamic_fields`	`bool`	Enables extraction of dynamically assigned instance fields (Python).
`assignment_type_aliases`	`bool`	Enables extraction of assignment-based type definitions and TypeVar declarations (Python).
`callable_types`	`bool`	Controls whether types can be invoked directly as call expressions (Python, C++), as opposed to type casts (C).

Table 7.2: Key resolution configuration parameters and their semantic effects.

Table 7.3 shows the concrete configuration values for each supported language.

Through this configuration map $\mathcal{K} : \mathcal{L} \to \mathcal{C}_{params}$, the resolution engine specializes its behavior for each language while maintaining a strictly language-agnostic core algorithm.

Parameter	Python	Rust	Java	C++	C
`transitive_imports`	✓				
`self_keyword`	-	`self`	`this`	`this`	-
`self_type_keyword`	-	`Self`	-	-	-
`implicit_first_param_as_self`	✓				
`extract_dynamic_fields`	✓				
`assignment_type_aliases`	✓				
`callable_types`	✓			✓	

Table 7.3: Concrete resolution configuration values for each supported language.

7.8 Termination and Convergence Guarantees

A critical requirement for static architectural analysis on uncurated, large-scale codebases is guaranteed termination and convergence. Real-world software projects frequently exhibit cyclic patterns that risk causing infinite recursion or combinatorial explosion during name resolution. These patterns include circular or diamond inheritance hierarchies, mutual module re-exports, self-referential type assignments, and wildcard package imports.

Because any target workspace produces a finite set of extracted modules $\vec{\mathcal{M}}$, the resulting Scope Tree $\mathcal{E}$ comprises a finite number of scopes ($|\mathcal{E}| = N < \infty$) and a finite collection of declared symbols ($|\text{Sym}| = S < \infty$). Under the execution operator ρ_{exec}, query evaluation `evaluate_query`(q, Σ) traverses this discrete structure. To ensure that every resolution path terminates deterministically in a finite number of steps, the resolution engine enforces convergence through four structural mechanisms:

1. **Monotonic Lexical Climbing**: When resolving an unqualified identifier query $q = \text{Id}(\iota)$, the engine searches the current lexical scope Σ. If the identifier is not found locally, resolution delegates recursively to parent(Σ). This climbing traversal is strictly monotonic with respect to scope depth: each ascending step decreases depth by one (depth(parent(Σ)) = depth(Σ) -1). Because the Scope Tree has finite depth and terminates at the unique root scope Σ_{root} (where parent(Σ_{root}) = `None`), lexical climbing is bounded by at most depth(Σ) steps, either locating the target symbol or safely exhausting the ancestor chain.

2. **Inheritance Traversal and Sentinel Memoization**: Querying members of structured types requires searching ancestor scopes along the class inheritance graph. This traversal introduces two distinct hazards: circular inheritance (such as class A extending B while B extends A) and exponential path re-evaluation in diamond or multiple-inheritance hierarchies. OmniDeps addresses both hazards through the super-scope memoization table $\mathcal{M}$ in the execution context Γ. When `resolve_super_scopes` begins inspecting a class scope Σ_C, it immediately records an empty sentinel entry $\mathcal{M}(\Sigma_C) \leftarrow \emptyset$ prior to recursively evaluating any base types. If a recursive descent re-enters Σ_C, the sentinel is encountered immediately, yielding an empty set and breaking the cycle. Once all direct and transitive ancestors are resolved, the complete set of scopes is committed to $\mathcal{M}(\Sigma_C)$. Subsequent member queries for Σ_C retrieve the cached set in $\mathcal{O}(1)$ time.

Furthermore, the active traversal maintains a visited set $V_{\text{scopes}} \subseteq \Sigma$, ensuring that each ancestor scope in the directed acyclic graph is visited at most once.

3. **Scope-Aware Cycle Guards for Aliases and Queries**: When resolving type alias chains, dynamic field assignments, or re-exports, the engine follows references across intermediate definitions. To prevent infinite loops caused by self-referential assignments (such as $x = x[0]$ in dynamic scripts) or mutually recursive aliases, the engine maintains visited tracking sets. These cycle guards are indexed by composite, scope-aware keys: $k_{\text{sym}} = \langle \Sigma, \iota \rangle$ for symbol alias resolution and $k_{\text{query}} = \langle \Sigma, \text{base}(q) \rangle$ for query evaluation. Because the universe of valid scopes and symbol identifiers across the workspace is finite, the visited set monotonically expands with each dereference step. If a key is encountered that already exists in the set, the engine halts that branch immediately and yields a failed resolution (`None` or `TypeRef::Failed`).

4. **Transitive and Wildcard Import Decoupling**: In languages supporting transitive imports or wildcard package re-exports, resolving a symbol across modules risks oscillating indefinitely between global namespace lookups and transitive module traversals. To prevent mutual recursion, the engine strips terminal wildcard tokens before recursive lookup and partitions visitation tracking into separate sets: V_{global} for workspace-level module resolution and V_{trans} for transitive import chains. This separation ensures that each module boundary is crossed at most once per import branch.

Through these monotonic reductions and finite-state tracking mechanisms, every execution branch in `evaluate_query` either converges to a resolved terminal reference or terminates upon cycle detection within finite steps. This design guarantees both algorithmic stability and predictable execution times across heterogeneous codebases.

Chapter 8

Graph Construction

8.1 Overview

The graph construction phase is the third and final stage of the analysis pipeline. It receives as input the module hierarchy $\vec{\mathcal{M}} \in \mathcal{D}$ produced by the extraction phase and refined by the name resolution phase, in which all type references τ have been resolved to their final states (`Resolved`, `External`, `Primitive`, or `Failed`). The output is a **Dependency Graph** $\mathcal{G} = (V, E)$, a directed multigraph where vertices $V \subset \mathcal{C}$ represent software components and edges E represent semantic dependencies between components.

Formally, the graph construction function γ is defined as:

$$\gamma : \vec{\mathcal{M}} \to \mathcal{G}$$

The construction pipeline operates in two sequential traversal passes followed by a post-processing phase:

1. **Pass 1 (Node Extraction)**: The module tree is recursively traversed to collect all workspace components (modules, structured types, functions, fields, and type aliases) into the vertex set V, computing their globally unique qualified names.

2. **Pass 2 (Edge Generation)**: The tree is traversed a second time to inspect resolved references within signatures and code blocks, emitting directed dependency edges of the form $e = \langle \pi_{src}, \pi_{tgt}, k \rangle \in E$.

3. **Post-processing**: Ensures referential integrity by dynamically adding implicit endpoints (`Primitive` and `External` types referenced by edges but absent from the workspace) to V, and deduplicates identical edges.

The overall architecture of this construction pipeline is summarized in Figure 8.1.

8.2 Node Flattening

The Intermediate Representation is structured as a recursive tree of modules ($\mathcal{M}$), where each module can contain sub-modules, structured types, functions,

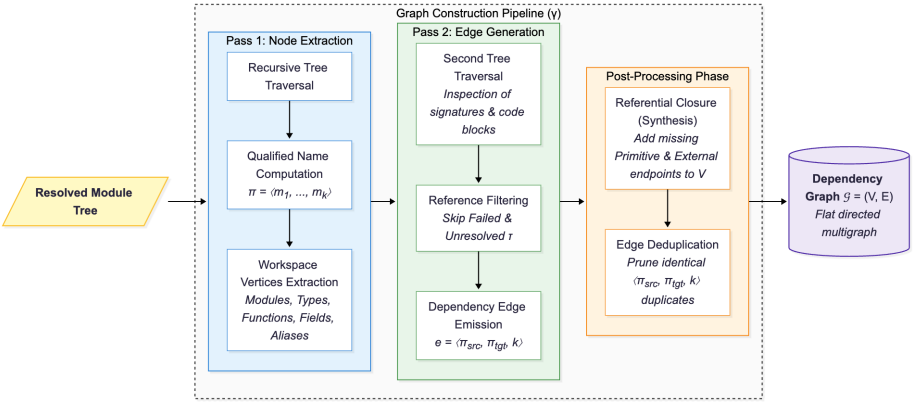


Figure 8.1: High-level pipeline of the graph construction phase (γ). The resolved module hierarchy ($\dot{\mathcal{M}} \in \mathcal{D}$) undergoes a first traversal pass to extract workspace vertices (V), a second pass to emit directed dependency edges (E), and a post-processing stage that guarantees referential closure and eliminates duplicate edges.

type aliases, and free variables. The dependency graph, however, requires a flat set of vertices. The flattening algorithm recursively traverses the module tree and promotes every component to an independent graph node.

8.2.1 Qualified Name Computation

During flattening, each component's name is recomputed as a globally unique **Qualified Name** π by prepending the names of all ancestor modules. For instance, a class `User` declared inside module `auth`, which is itself nested in the root module, receives the qualified name $\langle \texttt{auth}, \texttt{User} \rangle$. The root module name is filtered out from the prefix to avoid redundant path segments.

This globally qualified naming scheme guarantees that every vertex in the graph has a unique identifier, even when multiple entities share the same local name across different modules.

8.2.2 Entities Promoted to Vertices

The flattening process generates vertices for the following five IR constructs extracted from the workspace:

- **Modules** ($\mathcal{M}$): Every module and sub-module in the hierarchy becomes a vertex of type `Module`. Because Phase 1 extracts individual source files independently into modular trees, multiple compilation units within the same package or namespace share common ancestor paths (e.g., `org.apache.flink`). During flattening, canonical module paths are deduplicated via a tracking set ($seen_modules \subseteq \Pi$) so that each logical module is emitted exactly once as a vertex in V, while structural parent-child relationships are established through explicit containment edges.

- **Structured Types** ($\mathcal{S}$): Every class, struct, interface, trait, enum, and enum variant becomes a vertex of type `StructuredType`. Nested types

(inner classes, nested structs) are recursively flattened and promoted to independent vertices.

- **Functions** ($\mathcal{F}$): Every free function and every method declared inside a structured type becomes a vertex of type `Function`.
- **Fields** (f): Every field of a structured type and every free variable declared at module level becomes a vertex of type `Field`, carrying its qualified name and its type reference.
- **Type Aliases** ($\mathcal{A}$): Every type alias becomes a vertex of type `TypeAlias`.

Unlike the five entities above, **Implementation Blocks** ($\mathcal{X}$) do not produce independent vertices in the graph. Instead, they act as re-scoping containers: all methods, nested types, and type aliases declared inside an `ImplBlock` are re-scoped under the target type's qualified name and flattened into V as if they were directly declared inside the target structured type. Coupled with the dynamically synthesized vertices described below, the resulting vertex set V covers all seven variants of the universal component domain $\mathcal{C}$.

8.2.3 Dynamic Node Synthesis

After the initial flattening, edges may reference types that are not present as vertices in the workspace. This occurs when a component depends on a primitive type (e.g., `int`, `bool`, `String`) or on an external type from a third-party library (e.g., `java.util.List`, `std::collections::HashMap`).

To maintain referential integrity, the post-processing algorithm scans all generated edges and identifies target endpoints that do not match any existing workspace vertex:

- If the target is recognized as a primitive type by the *Primitive Registry* (introduced in Chapter 4), a `Primitive` node is synthesized.
- If the target corresponds to a certified third-party reference promoted through an explicit import directive, an `External` node is synthesized, representing an external dependency outside the analyzed workspace.
- Unresolvable references lacking a valid import directive remain in the `Failed` state and are pruned during edge generation, avoiding the creation of spurious or phantom nodes.

This approach guarantees that every emitted edge in the final multigraph connects two valid endpoints in V, and that external dependencies are explicitly observable as nodes.

8.3 Edge Generation

The second sub-phase traverses the module tree recursively, inspecting every component and its resolved type references to generate directed dependency edges. Formally, the resulting dependency graph is a directed multigraph $\mathcal{G} =$

(V, E), where $V \subset \mathcal{C}$ denotes the finite set of flattened components and E is the multiset of typed dependency edges. Each edge $e \in E$ is formalized as a triple:

$$e = \langle \pi_{src}, \pi_{tgt}, k \rangle$$

where $\pi_{src}, \pi_{tgt} \in \Pi$ designate the qualified names of the source and target components, and $k \in \text{Kind}$ identifies the semantic nature of the dependency from the alphabet of fifteen supported dependency kinds.

8.3.1 Type Reference Filtering and Referential Integrity

A fundamental invariant governs edge generation: dependency edges are produced predominantly from type references in the terminal states `Resolved` and `External`.

During the Name Resolution phase (Chapter 7), a type reference is promoted to the `External` state if and only if two conditions are strictly satisfied:

1. The symbol's identifier matches an explicit import directive ($\delta \in \vec{\delta}_{imp}$) encountered during lexical scope climbing or module-level transitive import traversal;
2. The imported path is verified to be absent from the workspace-wide Scope Tree ($\mathcal{E}$).

This rigorous verification certifies that the entity genuinely originates from an external third-party library or runtime platform (e.g., `java.util.List`, `serde::Serialize`), rather than an unresolvable local symbol or an extraction defect. For every such certified reference, the edge generation phase emits a dependency edge and post-processing dynamically synthesizes a corresponding `External` vertex in V, making third-party architectural couplings fully observable.

Conversely, references in the `Failed` state (symbols that could neither be located in the workspace hierarchy nor substantiated by any matching import directive), as well as un-evaluated `Unresolved` references with empty path specifications, are deliberately skipped. This filtering ensures that unresolvable syntax fragments or dynamic idioms do not introduce phantom nodes or spurious dependencies into the architectural graph.

Finally, references categorized as `Primitive` by the Primitive Registry ($\mathcal{P}$) generate dependency edges and synthesize `Primitive` vertices in the multigraph, thereby guaranteeing formal referential closure (every edge has valid source and target endpoints in V). However, to prioritize high-level software couplings and prevent visual clutter, primitive nodes and their incoming edges are toggled off by default in the interactive visualizer.

8.3.2 Decomposition of Compound and Generic Targets

When projecting resolved type references into directed dependency edges, the helper function `type_ref_targets` decomposes compound and parameterized variants into a sequence of concrete target endpoints $\vec{\pi}_{tgt} \subset \Pi$:

- **Terminal Variants**: For Resolved(π), External(π), and Primitive(ι), the target path is emitted directly.

- **Generic Applications (Generic($\tau_{\mathbf{base}}, \vec{\tau}_{\mathbf{args}}$))**: Both the container type τ_{base} and all specialized type arguments $\tau_i \in \vec{\tau}_{\text{args}}$ represent architectural dependencies. The decomposition collects targets from both:

$$\text{targets}(\text{Generic}(\tau_{\text{base}}, \vec{\tau}_{\text{args}})) = \text{targets}(\tau_{\text{base}}) \cup \bigcup_{i=1}^{k} \text{targets}(\tau_i)$$

  For example, a field declared as `items: std::vector<models::User>` emits `UsesFieldType` edges connecting the field to both `std::vector` and `models::User`.

- **Type Variables (TypeVar($\iota, \vec{\tau}_{\mathbf{bounds}}$))**: Because a type variable is an ephemeral placeholder, its architectural couplings are governed by its constraint bounds:

$$\text{targets}(\text{TypeVar}(\iota, \vec{\tau}_{\text{bounds}})) = \bigcup_{\tau_b \in \vec{\tau}_{\text{bounds}}} \text{targets}(\tau_b)$$

  A function parameter `x: T` with T : Display emits a `UsesParamType` dependency toward the `Display` trait.

- **Union Types (Union($\vec{\tau}$))**: The decomposition distributes over all alternative branches, emitting edges toward every reachable target in $\vec{\tau}$.

- **Evaluated Access Chains (EvaluatedAccess($\tau_{\mathbf{path}}, \tau_{\mathbf{type}}$))**: Targets are collected from both the intermediate receiver path τ_{path} and the resulting terminal type τ_{type}, capturing the complete access lineage.

8.3.3 Taxonomy of Dependency Edge Kinds

The system defines 16 distinct edge kinds, organized into five categories. Table 8.1 provides a complete classification.

Category	Edge Kind	Source	Target
Structural	`ModuleContainment`	Module	Direct child
	`NestedIn`	StructuredType	Field, method, or nested type
Inheritance	`IsA`	StructuredType	Super type
	`Implements`	Module / StructuredType	Type / Trait
Type-Level	`UsesFieldType`	Field	Type of a field
	`UsesLocalType`	Function	Type of a local variable
	`UsesParamType`	Function	Type of a parameter
	`UsesReturnType`	Function	Return type
	`UsesType`	StructuredType / Function	Generic constraint bound
	`Aliases`	TypeAlias	Aliased target type
Behavioral	`Calls`	Function	Invoked function/type
	`Instantiates`	Function	Instantiated type
	`AccessesField`	Function	Accessed component
Auxiliary	`Imports`	Module / Type	Imported path
	`CastsTo`	Function	Cast target type
	`AnnotatedWith`	Any component	Annotation type

Table 8.1: Complete taxonomy of the 16 dependency edge kinds, organized by category.

Structural Edges

Structural edges encode the hierarchical containment relationships of the original source code:

- **ModuleContainment**: Generated from a module to each of its direct children (sub-modules, structured types, free functions, free variables, and type aliases). The traversal is recursive: when a sub-module is encountered, the current module's qualified name is passed as the parent identifier.
- **NestedIn**: Generated from a structured type to each of its member fields, methods, and nested types. This edge captures structural ownership, establishing that a field or method "belongs to" a class, or that an inner type is defined within the scope of an outer type.

Inheritance Edges

These edges model sub-typing, contract conformance, and decoupled implementation extensions:

- **IsA**: Generated from a structured type to each of its resolved super-types ($\vec{\tau}_{sup}$). In nominal object-oriented languages, this edge unifies class inheritance (`extends`), interface implementation (such as Java's `implements`), and trait super-traits into a single subtyping relation.
- **Implements**: Reserved for decoupled implementation blocks (`ImplBlock`), such as Rust's `impl` blocks or out-of-line type extensions. Because implementation blocks are dissolved during node flattening, two distinct edges model their semantic bindings:
  1. An edge from the enclosing module to the target type ($M \xrightarrow{\text{Implements}} T$), capturing the architectural dependency that module M provides an extension or method implementation for type T.
  2. If the block implements a trait or interface (T implements τ_{trait}), an edge from the target type to the implemented trait ($T \xrightarrow{\text{Implements}}$ Trait), recording contract fulfillment.

Type-Level Edges

Type-level edges capture static dependencies introduced through type declarations:

- **UsesFieldType**: Generated from a field f (or a module-level free variable) to its resolved type. Under the fine-grained component model, the enclosing structured type is connected to its member field via a `NestedIn` edge, while the field itself carries the type dependency: for instance, if class `Car` has a field `engine: Engine`, the graph contains `Car` $\xrightarrow{\text{NestedIn}}$ `Car::engine` and `Car::engine` $\xrightarrow{\text{UsesFieldType}}$ `Engine`.
- **UsesLocalType**: Generated from a function to the resolved type of each local variable declared in its body blocks. This edge kind was introduced to distinguish local variable dependencies from field-level dependencies.

- **UsesParamType**: Generated from a function to the resolved type of each of its formal parameters.
- **UsesReturnType**: Generated from a function to its resolved return type.
- **UsesType**: Generated from a structured type or function to each resolved constraint bound declared in its generic type parameters ($\vec{\theta}$). For instance, a generic entity `Handler<T: Service>` emits a `UsesType` edge from `Handler` to `Service`, recording the structural dependency on the trait or interface.
- **Aliases**: Generated from a type alias to its resolved target type, capturing the semantic binding between the alias and its original type.

Listing 8.1 and Figure 8.2 illustrate this distinction. In Listing 8.1, the struct `Car` declares a field `engine` of type `Engine` and a method `drive` that invokes `self.engine.start()`. In the resulting fine-grained dependency graph (Figure 8.2), `Car` contains its members via `NestedIn` edges, while the type dependency `UsesFieldType` originates strictly from the field node `Car::engine` toward `Engine`. Concurrently, the method `Car::drive` exhibits an `AccessesField` dependency toward `Car::engine` and a `Calls` dependency toward `Engine::start`.

```
struct Car {
    engine: Engine,
}

impl Car {
    fn drive(&self) {
        self.engine.start();
    }
}
```

Listing 8.1: Example illustrating fine-grained field and call dependencies.

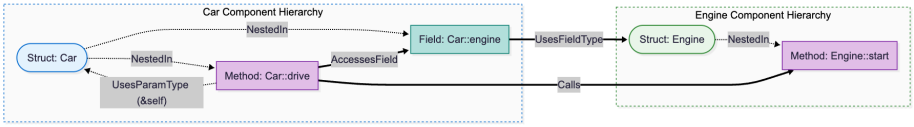


Figure 8.2: Fine-grained dependency multigraph generated for Listing 8.1. Containment relationships are captured by dashed `NestedIn` edges, while the type dependency `UsesFieldType` originates from the member field `Car::engine` rather than from the enclosing struct. Behavioral relationships are recorded via `AccessesField` and `Calls`.

Behavioral Edges

Behavioral edges model runtime interactions extracted from function bodies. The traversal of function bodies follows the recursive `Block` ($\mathcal{B}$) structure: the root block is inspected first, then all sub-blocks (corresponding to `if`, `while`, `for`, and similar control-flow constructs) are visited recursively. All behavioral

edges are attributed to the enclosing function, regardless of the nesting depth of the block in which the dependency was found.

- **Calls**: Generated for each resolved function invocation found in the block's $\vec{\tau}_{calls}$ vector.
- **Instantiates**: Generated for each resolved type instantiation (e.g., `new`, struct expressions) found in the block's $\vec{\tau}_{inst}$ vector.
- **AccessesField**: Generated for each resolved field access (read or write) found in the block's $\vec{\tau}_{accesses}$ vector.

Auxiliary Edges

- **Imports**: Generated from a module or structured type to each explicitly imported path ($\vec{\delta}_{imp}$).
- **CastsTo**: Generated from a function to the target type of each explicit type cast found in the block's $\vec{\tau}_{casts}$ vector.
- **AnnotatedWith**: Generated from any annotated component (function, structured type, field) to the resolved type of each annotation or decorator.

8.3.4 Edge Deduplication

During the edge generation pass, multiple identical dependencies may be recorded if a component references another entity repeatedly (e.g., a function invoking the same method multiple times across different control-flow blocks).

To normalize the graph, the complete edge set is sorted and deduplicated. Two edges $e_1 = \langle \pi^1_{src}, \pi^1_{tgt}, k_1 \rangle$ and $e_2 = \langle \pi^2_{src}, \pi^2_{tgt}, k_2 \rangle$ are considered identical if and only if $\pi^1_{src} = \pi^2_{src}$, $\pi^1_{tgt} = \pi^2_{tgt}$, and $k_1 = k_2$.

While $\mathcal{G}$ is a multigraph that permits multiple distinct dependency kinds between the same pair of entities (for example, a function may simultaneously exhibit a `Calls` and an `Instantiates` edge toward the same target), identical duplicate edges are pruned. From an architectural perspective, this normalization is crucial: standard software metrics, such as Efferent Coupling (Ce) and Coupling Between Objects (CBO), measure structural inter-component dependencies rather than syntactic repetition frequency. Retaining duplicate edges would distort topological analyses and artificially inflate coupling metrics.

8.4 Graph Representations and Visualization

To support diverse downstream analytical workflows, the pipeline provides distinct representations of the extracted architectural model, catering to graph-theoretic visualization, programmatic consumption, and syntactic debugging:

- **Cytoscape.js Compound Format**: The primary external representation of the dependency graph, optimized for interactive rendering and graph-theoretical libraries. By transforming structural hierarchy into spatial compound containment, it eliminates redundant visual edges while preserving full semantic fidelity.

- **Raw Dependency Graph Format**: A direct serialization of the in-memory `DependencyGraph` multigraph $\mathcal{G} = (V, E)$, retaining explicit structural containment edges and structured qualified name vectors ($\pi \in \Pi$) for automated architectural checking and formal validation tools.
- **Unpruned Intermediate Representation**: The pre-flattened module forest $\vec{\mathcal{M}}$, maintaining the full lexical scope tree and detailed intra-procedural syntactic blocks $\mathcal{B}$ (including statement-level invocations and local declarations) for compiler diagnostics and front-end verification.

This section describes the Cytoscape transformation process and the companion web-based visualizer.

Cytoscape.js and Compound Nodes

Cytoscape.js is an open-source JavaScript library for graph visualization and analysis. A notable feature leveraged by this project is its support for **Compound Nodes**: nodes that visually contain other nodes, rendering hierarchical groupings as nested bounding boxes rather than as explicit edges. In software visualization literature, representing structural encapsulation through nested compound boundaries rather than dense networks of containment edges is a canonical technique to reduce edge congestion and maintain architectural readability [37].

The Cytoscape export process transforms the flat `DependencyGraph` into an array of Cytoscape elements (nodes and edges) by applying three transformations:

1. **Parent Assignment**: For each node, the qualified name is truncated to obtain the parent's identifier. If the parent exists in the node set and is not the root module, the node's `parent` property is set accordingly. This converts the structural containment information (previously encoded as `ModuleContainment` and `NestedIn` edges) into Cytoscape's native compound node hierarchy.
2. **Structural Edge Filtering**: Since containment is now represented spatially through compound nesting, all `ModuleContainment` and `NestedIn` edges are filtered out from the exported edge set. This reduces visual clutter and allows the viewer to focus on the architecturally meaningful dependencies (type-level, inheritance, and behavioral edges).
3. **External Node Synthesis**: If an edge references an endpoint that does not exist as a node in the graph (e.g., an external library type), a placeholder node of type `External` is automatically created.

This structural transformation is illustrated in Figure 8.3. While the flat dependency graph requires explicit containment edges that visually congest the layout, the Cytoscape export translates containment relationships into spatial enclosures, preserving the visual focus on functional and behavioral dependencies.

Each exported node carries the following data fields:

- `id`: The fully qualified name, joined with `::` separators (e.g., `auth::User`).

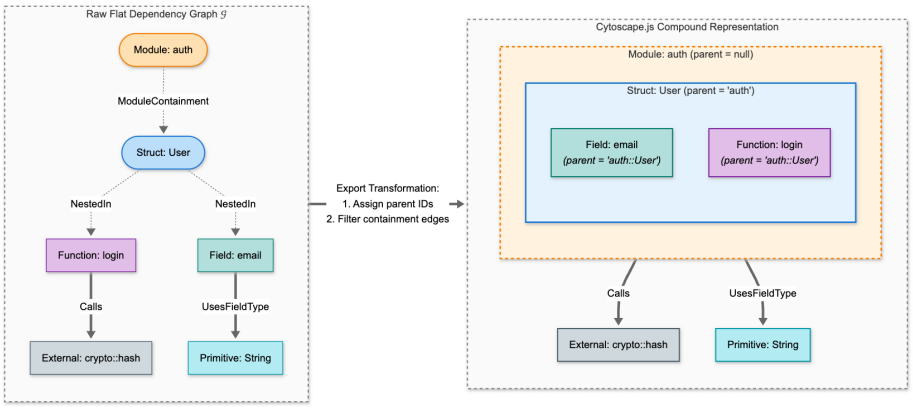


Figure 8.3: Mapping structural containment to Cytoscape.js compound nodes. The raw flat graph (left) connects containers and members with explicit `ModuleContainment` and `NestedIn` edges. The exported representation (right) transforms containment into nested bounding boxes via the `parent` property, pruning structural edges and highlighting architectural dependencies.

- `label`: The last segment of the qualified name (e.g., `User`).
- `type`: The component category (`Module`, `Class`, `Struct`, `Function`, etc.).
- `parent`: The `id` of the enclosing compound node, if applicable.

Each exported edge carries:

- `source` and `target`: The `id`s of the source and target nodes.
- `label`: The edge kind as a string (e.g., `Calls`, `IsA`).

Edges are deduplicated during export using a hash set that tracks the combination of source, target, and label to prevent visual duplication.

The Web-Based Graph Visualizer

To provide an interactive exploration environment for the generated graphs, a companion web application was developed using **SvelteKit** (Svelte 5) [38] and the Cytoscape.js library. The application communicates with the Rust analyzer through local API endpoints that invoke the CLI binary and return the Cytoscape-formatted JSON. The high-level architecture of the visualization system is depicted in Figure 8.4.

Graph Layout

The visualizer employs the **fCoSE** (Fast Compound Spring Embedder) layout algorithm [39], a force-directed layout specifically designed for compound graphs with nested node hierarchies. The algorithm treats compound nodes as elastic containers and arranges child nodes within their parent boundaries while simultaneously optimizing the overall graph topology through spring-electric forces. Key parameters configured in the application include:

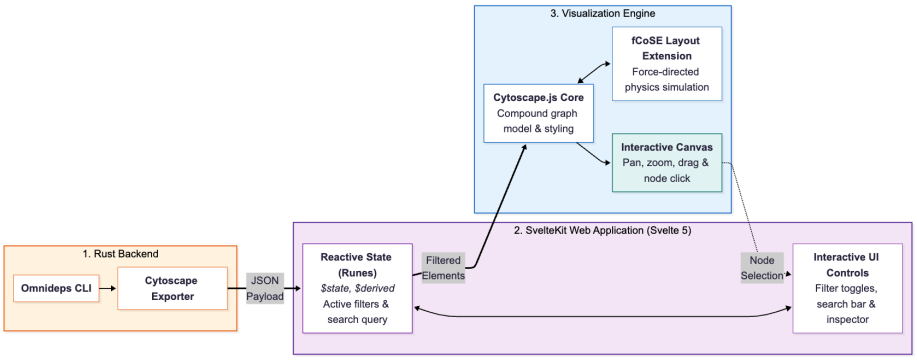


Figure 8.4: Architecture of the interactive graph visualizer. The Rust analyzer backend exposes CLI and local HTTP API endpoints delivering Cytoscape-formatted JSON payloads. The SvelteKit frontend coordinates reactive state management (using Svelte 5 runes for filter toggles and search queries) with the Cytoscape.js canvas and the `fCoSE` physics layout extension.

- `quality: "proof"`: Requests the highest quality layout computation.
- `idealEdgeLength`: Controls the target distance between connected nodes.
- `nodeRepulsion`: Controls the repulsive force between non-adjacent nodes.
- `gravityCompound`: Controls how strongly child nodes are pulled toward the center of their parent compound node.
- `packComponents`: Enables compact packing of disconnected graph components.

Interactive Features

To facilitate software architecture exploration and mitigate the visual clutter typical of large dependency networks, the visualizer implements interactive exploration and filtering techniques [37]:

- **Node Search**: A text input field filters the graph in real time. Nodes whose labels match the query (case-insensitive substring match) are highlighted, while all other nodes are visually dimmed. Matching nodes and their ancestor compound containers remain visible to preserve spatial context.
- **Node Inspection**: Tapping a node opens a detail sidebar displaying the node's type, full qualified name, and a breakdown of its incoming and outgoing edges, grouped by edge kind. Each related node in the sidebar is clickable, allowing navigation through the graph.
- **Neighborhood Focus**: When a node is tapped, the graph automatically dims all nodes and edges except the selected node, its direct neighborhood (connected nodes and their mutual edges), and its ancestor/descendant compound containers.

- **Edge Filtering**: The control panel provides two groups of toggle switches for edge filtering:
  - *Structural filters*: `IsA`, `Implements`, `Imports`, `UsesFieldType`, `UsesParamType`, `UsesReturnType`, `UsesLocalType`, `NestedIn`, `ModuleContainment`, `AnnotatedWith`.
  - *Behavioral filters*: `Calls`, `Instantiates`, `AccessesField`.

  Each filter can be toggled independently, with a master toggle for each group. The filtering logic operates in batch mode: edges of disabled kinds are hidden, and edges whose source or target node is itself hidden are also suppressed.
- **Node Type Filtering**: Separate toggles allow hiding or showing entire categories of nodes (e.g., all `Module` nodes, all `External` nodes, all `Primitive` nodes). By default, `Primitive` nodes are hidden to reduce noise.
- **JSON Export**: Two export options are provided: exporting the Cytoscape-formatted elements (for re-importing into visualization tools) and exporting the raw analyzer JSON output (for programmatic consumption).

Chapter 9

Evaluation and Results

Evaluating a language-agnostic static analysis framework presents significant empirical challenges. First, obtaining an absolute, manually verified ground truth for large-scale real-world codebases is practically infeasible. Second, comparative evaluation against existing analyzers is complicated by substantial behavioral and granularity disparities: different tools operate at varying levels of abstraction (from fine-grained function calls and variable references to coarse-grained class or translation-unit components), employ divergent relational taxonomies, and encode things differently. To address these challenges, our evaluation is divided into three parts:

1. **Ground-Truth Validation**: Verifying extraction accuracy against synthetic micro-benchmarks with known oracles, covering both basic language constructs and language-specific idioms in isolation.
2. **Cross-Tool Structural Comparison**: Comparing OmniDeps against established analyzers on real-world repositories, using a graph harmonization framework that aims to reconcile differences in granularity, metamodels, and edge taxonomies.
3. **Runtime Performance Benchmarking**: Measuring execution time, analysis throughput, and scalability across projects of different sizes under a controlled environment.

The remainder of this chapter is structured as follows. Section 9.1 introduces the research questions. Sections 9.2, 9.3, and 9.4 describe the methodology for each of the three evaluation parts, and Section 9.5 presents the experimental results.

9.1 Research Questions

The empirical and theoretical evaluation is guided by four central Research Questions (RQs):

RQ1 (Language Construct Coverage):
Does OmniDeps adequately cover the structural constructs and dependencies of all supported programming languages?

RQ2 (Cross-Tool Structural Alignment):
How closely do the dependency graphs extracted by OmniDeps align with the results produced by established single-language and multi-language architectural analyzers on large-scale open-source software systems?

RQ3 (Topological Concordance and Discrepancy Analysis):
In which specific categories of software dependencies (structural containment, inheritance hierarchies, behavioral invocations, module inclusion) do cross-tool discrepancies concentrate, and what semantic assumptions or architectural trade-offs account for observed divergences?

RQ4 (Computational Scalability and Throughput):
How do the execution runtime and analysis throughput of OmniDeps compare against state-of-the-art baselines when analyzing repositories of varying scale, from compact libraries to multi-million-line codebases?

Motivation and Rationale. These research questions systematically evaluate whether a lightweight, syntax-query static analyzer can serve as an effective alternative to traditional compiler-coupled tools:

- **RQ1** verifies the baseline soundness of our extraction heuristics in a controlled setting before scaling to complex repositories. Answering this question establishes whether Concrete Syntax Tree queries are sufficiently expressive to capture core language constructs across diverse programming paradigms.
- **RQ2** evaluates real-world applicability, as synthetic benchmarks cannot capture the architectural complexity of realistic software. Answering this question demonstrates whether the language-agnostic extraction strategy generalizes to large repositories and produces dependency graphs comparable to mature, compiler-based analyzers.
- **RQ3** examines the nature of discrepancies, as global similarity metrics hide where and why tools disagree. By isolating divergences across containment, inheritance, call dependencies, and imports, this question reveals where syntax queries match compiler precision and where compiler-level type checking remains necessary.
- **RQ4** evaluates computational efficiency and scalability in comparison to existing analyzers. Answering this question determines whether OmniDeps achieves execution runtimes comparable to or better than established tools, ensuring that it remains practical across real-world software systems of varying size and that analysis time scales predictably without exploding on large codebases.

Together, answering these questions allows us to evaluate the central hypothesis of this thesis: that declarative pattern queries on Concrete Syntax Trees, paired with a language-agnostic scope and symbol resolution engine, can construct reliable architectural dependency graphs directly from raw source code, achieving structural alignment with established compiler-based analyzers while maintaining practical and scalable execution performance.

9.2 Ground-Truth Validation

During the development of a static analysis framework, verifying that individual language constructs and dependencies are faithfully captured requires a controlled testing environment. In this validation track, the dependency graph automatically extracted by the system is programmatically compared against a manually defined ground truth for each benchmark. To evaluate construct coverage across paradigms, a dedicated benchmark suite of synthetic source files was created for each supported language (C, C++, Java, Rust, and Python). Rather than complete software applications, these suites consist of minimal source files specifically crafted to exercise and isolate key architectural constructs and language-specific idioms.

The Oracle

For each language benchmark, the ground truth is formalized in a YAML manifest file that acts as the architectural oracle against which the extracted graph is verified. The manifest explicitly defines:

- **Expected Nodes**: The architectural entities that the system must recognize. Each entry specifies the entity's Fully Qualified Name and its structural category (such as modules, classes, structs, traits, enums, or functions).
- **Expected Edges**: The semantic dependencies connecting the entities. Each relationship specifies a unique test identifier, the source and sink entity names, and the expected relationship kind (such as function calls, type inheritance, type usages, field accesses, or module imports).

Graph Verification and Matching Logic

To automate the evaluation, a dedicated test harness within OmniDeps executes the complete extraction pipeline on the benchmark source files and programmatically verifies adherence to the YAML manifest. The validation logic operates through three steps:

1. **Graph Flattening and Indexing**: The runner flattens the extracted component hierarchy into dot-separated fully qualified names. It constructs two lookup indices: a node index mapping each canonical identifier to its extracted component kind, and an adjacency index mapping each source identifier to its set of target sinks.
2. **Node Verification**: The algorithm iterates through the expected nodes in the manifest and checks whether their identifier exists in the extracted node index, recording whether the entity was discovered and whether its extracted kind matches the specification.
3. **Edge Verification**: For each expected edge, the runner performs a two-step validation:
    - *Endpoint Verification*: It ensures that both the source and sink entities were successfully extracted. If either endpoint is missing, the edge immediately fails validation due to a missing endpoint, isolating

whether the defect stems from entity extraction or relational resolution.

- *Adjacency Verification*: If both endpoints exist, the algorithm queries the adjacency index to verify whether a directed dependency linking the source to the sink was established.

Metrics and Reporting

The benchmark measures extraction accuracy for both nodes and edges as the fraction of expected elements successfully recovered:

$$\text{Acc}_{\text{nodes}} = \frac{|\{v \in V_{\text{expected}} \mid v \in V_{\text{extracted}}\}|}{|V_{\text{expected}}|} \tag{9.1}$$

$$\text{Acc}_{\text{edges}} = \frac{|\{e \in E_{\text{expected}} \mid e \in E_{\text{extracted}}\}|}{|E_{\text{expected}}|} \tag{9.2}$$

In addition to summary statistics, the test harness generates detailed reports in Markdown and JSON formats. For each expected node, the report records whether it was extracted and whether its component kind matches the specification. For each expected edge, it details the existence of both endpoints, the presence of the connecting edge, and the correctness of the relationship kind. This detailed feedback facilitates debugging during development and ensures continuous regression tracking across commits.

9.3 Cross-Tool Structural Comparison

While synthetic micro-benchmarks validate language features in isolation, answering **RQ2** and **RQ3** requires evaluating real-world software systems. However, directly comparing dependency graphs produced by independent static analysis tools is difficult. Analyzers operate under different metamodels, export formats, and levels of granularity. Without a dedicated alignment process, directly intersecting these graphs results in artificially low agreement caused by representational differences rather than actual semantic disagreements.

This section presents the harmonization framework used to map heterogeneous dependency graphs into a common canonical representation, defines the comparative metrics, and introduces the evaluation dataset and baseline tools.

9.3.1 Graph Harmonization Framework

The Metamodel Heterogeneity Problem

State-of-the-art analyzers conceptualize software structure at differing levels of abstraction:

- **Granularity Divergence**: Analyzers operate at different structural granularities. Fine-grained tools (such as *Depends* [27] and *Entangle* [26]) extract individual members within types (such as methods, functions, and fields) as distinct graph vertices, representing calls and usages as edges between these member-level entities. Conversely, architectural analyzers

(such as *Arcan* [7]) emit vertices only for coarse-grained structural components (specifically packages, classes, and interfaces in object-oriented environments, and physical source/header compilation units in C/C++ codebases), aggregating behavioral dependencies directly at the class or file level.

- **Implicit vs. Explicit Structural Hierarchy**: Certain graph models represent modular containment through explicit directed edges (e.g., `belongsTo` or `ModuleContainment`), whereas compound graph specifications (such as Cytoscape.js [35]) represent hierarchy via node nesting attributes (`parent` pointers), omitting explicit containment edge primitives.
- **Relational Taxonomy Mismatch**: Each tool employs its own relationship vocabularies. For instance, a method invocation may be classified as `Calls` in one tool, `call` in another, or subsumed into a generic `dependsOn` relation in an architectural smell detector. Similarly, subtype relationships may be split into `extend` and `implement`, or unified as `isChildOf` or `IsA`.
- **Namespace and Identifier Disparities**: Differing path separators (`::` in C++/Rust, `/` in file paths, `.` in Java/Python), varying handling of nested anonymous scopes, and synthetic root prefixes prevent exact string matching of entity names.

To overcome these structural disparities, we define a graph normalization pipeline that transforms any tool's raw extracted graph $G_{\text{raw}} = (V_{\text{raw}}, E_{\text{raw}})$ into a standardized architectural dependency graph $G_{\text{arch}} = (V_{\text{arch}}, E_{\text{arch}})$.

Entity Canonicalization and Classification

Because static analyzers emit identifiers in tool and language specific formats (e.g., using double colons in C++ or slashes in file paths), the harmonization pipeline first normalizes raw identifier strings into canonical dot-separated qualified paths (implemented via `normalize_identifier` in the `graph-synchron` adapters).

Extracted entities are then classified by architectural granularity. We partition entities into two disjoint groups:

- **Architectural Entities** (V_{arch}): Coarse-grained structural containers that define module and type boundaries, including packages, modules, classes, interfaces, traits, and enums. These represent the primary units of architectural organization (`is_type_not_volatile` in the adapters).
- **Fine-Grained Entities** (V_{vol}): Member-level sub-components residing within architectural containers, such as methods, functions, fields, and variables.

Semantic Entity Lifting

To enable consistent structural comparisons across analyzers operating at different granularities, member-level entities and their dependencies must be projected onto a unified architectural plane. While fine-grained tools model dependencies whose endpoints are individual members (such as methods, functions,

and fields), coarse-grained analyzers report couplings directly between enclosing components (such as classes, interfaces, packages, or translation units). Semantic entity lifting bridges this disparity by projecting member-level entities onto their enclosing architectural containers.

For each fine-grained entity, the adapter navigates up the syntactic containment hierarchy using `get_parent` until it reaches the nearest enclosing architectural entity. This projection $\lambda(v)$ maps any vertex to its architectural container:

$$\lambda(v) = \begin{cases} v & \text{if } v \text{ is an architectural entity} \\ \lambda(\text{parent}(v)) & \text{if } v \text{ is a fine-grained member} \end{cases} \tag{9.3}$$

When applied to edges, any raw dependency $\langle u, v \rangle$ between fine-grained members is lifted to an architectural dependency between their respective containers $\langle \lambda(u), \lambda(v) \rangle$.

Crucially, when both endpoints belong to the same architectural entity ($\lambda(u) = \lambda(v)$), the interaction represents an internal implementation detail, such as a private helper method invocation or an access to an internal field. These intra-component self-loops are suppressed, ensuring that comparative evaluation focuses exclusively on couplings between distinct architectural components.

Hierarchical Containment Synthesis

Architectural analysis relies heavily on hierarchical namespace structures to evaluate modular boundaries and detect package-level design flaws. However, static analysis tools represent containment in fundamentally different ways:

- Compound graph formats (such as Cytoscape JSON) encode containment implicitly through node nesting attributes (a `parent` pointer) rather than explicit dependency edges.
- Other tools emit only leaf classes, leaving intermediate namespace packages implicit.

To ensure structural consistency across formats, `graph-synchron` applies two harmonization steps during loading:

1. **Intermediate Package Synthesis**: For every entity with a qualified path, the adapter automatically generates any missing intermediate package nodes along the hierarchy.
2. **Explicit Containment Closure**: For every entity with a valid parent, an explicit directed containment edge (`belongsTo`) is created connecting the child to its enclosing container.

This synthesis guarantees that compound graphs and edge-based graphs can be compared under identical structural assumptions.

Unified Relational Taxonomy

Existing static analysis tools use disparate vocabularies to describe dependency relationships. For instance, a method invocation may be classified as `Calls` in one tool, `call` in another, or subsumed into a generic `dependsOn` relation in

an architectural smell detector. Similarly, subtype relationships may be distinguished into `extend` and `implement`, or unified under a general `isChildOf` relation.

To reconcile these differences, `graph-synchron` defines a translation function (`translate_label`) that maps tool-specific edge labels into a common taxonomy of five architectural relation classes:

- `belongsTo`: Physical or logical containment, including classes nested in packages or inner classes within outer types.
- `isChildOf`: Generalization and subtyping relationships between structured types of the same paradigm.
- `isImplementationOf`: Contract implementations, such as interface realizations or Rust trait implementation blocks.
- `includes`: File inclusion directives (`#include`) and explicit module imports.
- `dependsOn`: Behavioral, usage, and data-flow dependencies, aggregating method calls, field references, parameter types, return types, instantiations, and exception throws.

Table 9.1 details the mapping from tool-specific labels to this unified taxonomy.

Unified Relation ($\mathcal{R}_{\text{unified}}$)	Tool-Specific Source Labels	Architectural Semantic Definition
`belongsTo`	`belongsTo`, `ModuleContainment`, `NestedIn`, compound parent	Physical or logical structural containment (e.g., class nested in package, inner class in outer class).
`isChildOf`	`isChildOf`, `extend`, `IsA`, `subclass`	Direct generalization / inheritance relationship between structured types of the same paradigm.
`isImplementationOf`	`isImplementationOf`, `implement`, `Implements`	Interface realization, abstract contract implementation, or trait realization blocks.
`includes`	`includes`, `import`, `Imports`	File-level inclusion directives (`#include`) or explicit module import statements.
`dependsOn`	`dependsOn`, `Calls`, `call`, `use`, `cast`, `parameter`, `return`, `create`, `throw`	Semantic, behavioral, or data-flow dependency (method invocations, field references, type usage in signatures, instantiations).

Table 9.1: Mapping of heterogeneous tool edge labels into the unified architectural taxonomy $\mathcal{R}_{\text{unified}}$.

9.3.2 Comparative Graph Metrics

With all graphs mapped to canonical, lifted representations $G = (V, E)$, structural concordance and divergence are quantified through set-based and statistical indices.

Graph Similarity via Smoothed Jaccard Index

The primary measure of cross-tool structural agreement between two normalized graphs $G_A = (V_A, E_A)$ and $G_B = (V_B, E_B)$ is the **Jaccard Similarity Index** [18]. To ensure numerical stability when evaluating sparse graphs, submodules with empty edge sets, or boundary components, we employ **Laplace-smoothed** Jaccard formulations:

- **Smoothed Node Jaccard Similarity**: The structural agreement between the extracted vertex sets V_A and V_B is defined as:

$$J_{\text{nodes}}(G_A, G_B) = \frac{|V_A \cap V_B| + 1}{|V_A \cup V_B| + 1} \tag{9.4}$$

- **Smoothed Edge Jaccard Similarity**: Let each edge $e \in E$ be uniquely indexed by the tuple $\langle \text{source}, \text{target} \rangle$ (for unlabelled structural comparison) or $\langle \text{source}, \text{target}, \text{label} \rangle$ (for typed relational comparison). The edge agreement is defined as:

$$J_{\text{edges}}(G_A, G_B) = \frac{|E_A \cap E_B| + 1}{|E_A \cup E_B| + 1} \tag{9.5}$$

Properties of the Metric:

- $J(G_A, G_B) \in (0, 1]$ for all non-empty graphs, reaching 1.0 if and only if $V_A = V_B$ (or $E_A = E_B$).
- In the presence of empty edge sets ($|E_A \cup E_B| = 0$), the smoothed index evaluates to 1.0, correctly reflecting the agreement that no dependencies exist, rather than producing an undefined $\frac{0}{0}$ indeterminate form.
- For realistic codebases where $|V| \gg 1$ and $|E| \gg 1$, the additive Laplace smoothing factor (+1) introduces negligible variance while asymptotically converging to standard Jaccard similarity.

Directional Discrepancy Indices

While the Jaccard index summarizes global bidirectional overlap, it is inherently symmetric: it cannot diagnose *which* tool extracted a richer architectural graph or identify the nature of missing dependencies. To measure directional asymmetry, we define two complementary discrepancy metrics normalized by the shared agreement space:

- **Superiority Index (Relative Novelty)**: The degree to which analyzer A discovers entities or dependencies that are absent in analyzer B, relative to their shared intersection, is given by:

$$\text{Sup}(A, B) = \begin{cases} \frac{|A \setminus B|}{|A \cap B|} & \text{if } |A \cap B| > 0 \\ 1.0 & \text{if } |A \cap B| = 0 \text{ and } |A \setminus B| > 0 \\ 0.0 & \text{otherwise} \end{cases} \tag{9.6}$$

- **Inferiority Index (Relative Omission)**: Conversely, the degree to which analyzer A fails to discover entities or dependencies that are successfully extracted by analyzer B is:

$$\text{Inf}(A, B) = \text{Sup}(B, A) = \begin{cases} \frac{|B \setminus A|}{|A \cap B|} & \text{if } |A \cap B| > 0 \\ 1.0 & \text{if } |A \cap B| = 0 \text{ and } |B \setminus A| > 0 \\ 0.0 & \text{otherwise} \end{cases} \tag{9.7}$$

These indices provide intuitive diagnostic semantics:

- $\mathrm{Sup}(A, B) = 0$ indicates that all elements extracted by A are confirmed by B ($A \subseteq B$).
- $\mathrm{Sup}(A, B) = 0.5$ implies that for every two shared dependencies agreed upon by both tools, tool A identifies one additional dependency not seen by B.
- Comparing $\mathrm{Sup}(\mathrm{OmniDeps}, \mathrm{Baseline})$ against $\mathrm{Inf}(\mathrm{OmniDeps}, \mathrm{Baseline})$ indicates the directional balance between the two extractors, highlighting the relative volume of entities or dependencies extracted exclusively by OmniDeps compared to those identified only by the baseline tool.

Per-Type Edge Discrepancy

To understand where structural agreements and divergences concentrate, discrepancies are analyzed per relationship type (function `pairwise_edge_diff` in `graph-synchron`). For each relationship category κ (such as `belongsTo`, `isChildOf`, `dependsOn`, and `includes`), we compute the directional discrepancy rates:

$$\Delta_{A \setminus B}(\kappa) = \frac{|E_A^{(\kappa)} \setminus E_B^{(\kappa)}|}{|E_A^{(\kappa)}|}, \qquad \Delta_{B \setminus A}(\kappa) = \frac{|E_B^{(\kappa)} \setminus E_A^{(\kappa)}|}{|E_B^{(\kappa)}|} \tag{9.8}$$

where $E_A^{(\kappa)}$ denotes the set of edges of type κ extracted by tool A. This per-category breakdown isolates whether observed divergences stem from containment hierarchies, inheritance modeling, or behavioral call dependencies.

9.3.3 Evaluation Dataset and Baseline Analyzers

Evaluation Dataset

To evaluate structural alignment and performance, an empirical dataset of 156 open-source software projects was collected across the five supported programming languages (C, C++, Java, Python, and Rust). In total, the dataset accounts for nearly 98 million lines of code and over 550,000 source files (measured via CLOC).

The repositories were gathered from GitHub to cover a broad spectrum of software sizes and domains. They range from small utilities and libraries (such as `fd` or `requests`) to large-scale distributed systems, runtime environments, and engines (such as `elasticsearch`, `godot`, or `redis`). Selecting widely adopted open-source projects ensures that the static analyzers are evaluated against realistic codebases, featuring varied architectural patterns, module layouts, and idiomatic language constructs.

Baseline Analyzers

OmniDeps is benchmarked against three static analysis tools, chosen to contrast different parsing and extraction strategies:

- **Arcan** [7]: An architectural analysis framework and architectural smell detector. Arcan relies on compiler-based front-ends (Spoon for Java, Clang AST for C/C++), providing semantic type resolution at the cost of strong language coupling and high runtime overhead.
- **Depends** [27]: A multi-language dependency analyzer that employs ANTLR-generated grammar parsers for C/C++, Java, and Python, emitting Dependency Structure Matrices (DSM) and fine-grained call and usage dependencies.
- **Entangle** [26]: A direct predecessor research prototype developed within our laboratory. Entangle abandoned compiler front-ends in favor of Tree-sitter CST parsing and dynamic scope climbing for C and Java, serving as an architectural baseline for syntax-based heuristics.

Table 9.2 summarizes the programming languages supported by each tool in this evaluation.

Tool	C	C++	Java	Python	Rust
OmniDeps	✓	✓	✓	✓	✓
Depends	✓	✓	✓	✓	—
Arcan	✓	✓	✓	✓	—
Entangle	—	—	✓	—	✓

Table 9.2: Declared language support across evaluated static analysis tools.

9.4 Runtime Performance Benchmarking

To answer **RQ4**, an experimental protocol was defined to measure execution time, scalability, and computational throughput across repositories of varying scale.

Execution Environment and Measurement Protocol

To account for operating system scheduling noise, cache warmup effects, and runtime variations, the benchmarking framework enforces the following execution protocols:

- **Execution Sampling**: Every tool is executed across sampling runs to mitigate operating system scheduling noise and transient cache effects. For each project and tool, the sample mean execution time μ_t is recorded:

$$\mu_t = \frac{1}{k} \sum_{i=1}^{k} t_i \tag{9.9}$$

- **Hardware and OS Isolation**: On supported platforms, the benchmarking harness pins execution to a dedicated physical CPU core (e.g., via affinity masks) to eliminate context switching, thread migration, and scheduler jitter.

- **Configurable Execution Timeouts**: When analyzing large codebases, complex macro expansions or cyclic grammar constructs can induce pathological backtracking loops in baseline tools. A configurable timeout $T_{\max}$ (e.g., 60 minutes) is enforced. Runs exceeding $T_{\max}$ are recorded as timeouts without stalling batch benchmark execution.

Throughput and Scalability Metrics

Scalability is evaluated across two primary dimensions:

- **Analysis Throughput** (Θ): The processing speed measured in source lines of code analyzed per unit of wall-clock time:

$$\Theta = \frac{\text{CLOC}}{\mu_t} \quad \left[\frac{\text{lines}}{\text{second}}\right] \tag{9.10}$$

- **Relative Speedup** (S): The execution speedup achieved by OmniDeps relative to comparator baseline B:

$$S_B = \frac{\mu_{t,B}}{\mu_{t,\text{OmniDeps}}} \tag{9.11}$$

  where $S_B > 1.0$ indicates superior performance by OmniDeps.

9.5 Experimental Results

This section presents the empirical findings obtained across the evaluation suite, organized around the four guiding research questions: ground-truth fidelity on synthetic benchmarks (**RQ1**), deliverable extraction feasibility across the complete 156-project dataset, cross-tool structural alignment on real-world repositories (**RQ2**), topological discordance and semantic discrepancies (**RQ3**), and runtime scalability and throughput scaling (**RQ4**). A broader qualitative discussion of these findings and threats to validity is presented in Chapter 10.

9.5.1 RQ1: Ground-Truth Benchmarks Results

To answer **RQ1**, OmniDeps was evaluated against hand-crafted ground-truth architectural oracles constructed for each target language. Table 9.3 details the expected versus extracted node and edge counts, along with extraction accuracy metrics.

Language	Expected Nodes	Actual Nodes	Node Acc.	Expected Edges	Actual Edges	Edge Acc.
C	54	54	100%	47	47	100%
C++	52	52	100%	47	47	100%
Java	Topology-based oracle (Edges only)			46	46	100%
Rust	56	56	100%	84	84	100%
Python	60	60	100%	59	59	100%

Table 9.3: Evaluation of extraction fidelity against ground-truth architectural oracles.

As reported in Table 9.3, OmniDeps achieved **100% Node accuracy and 100% Edge accuracy** across all five languages on the synthetic benchmark suites.

While these results confirm the soundness of the extraction pipeline, their significance must be interpreted within their intended scope. These synthetic test suites were designed and maintained during the development of OmniDeps as regression benchmarks to exercise individual language constructs, such as struct declarations and function pointers in C, class templates and namespaces in C++, type parameter bounds and interface inheritance in Java, algebraic data types and trait implementations in Rust, and modular imports in Python. Achieving full adherence demonstrates that the language-agnostic intermediate representation (Chapter 4) and the scoped query resolution engine (Chapter 7) are sound and operate correctly in a controlled, isolated testing environment.

However, synthetic tests isolate specific language constructs in minimal files, and do not guarantee that the behavior will generalize to arbitrary, large-scale software projects. This limitation motivates the empirical evaluation on real-world repositories presented in the subsequent sections.

9.5.2 RQ2: Cross-Tool Structural Alignment

Moving from controlled synthetic benchmarks to real-world software, we first evaluate the execution completion rate of each analysis tool across the dataset of 156 repositories. Table 9.4 reports the results across all tools and programming languages. For all tools, a timeout of one hour was enforced, which is well above the average analysis time of any codebase in the dataset (in the order of seconds, minutes), ruling out runs that find themself in pathological states such as infinite loops. Analyzers that failed to complete within this time limit were recorded as failures, and the run was not counted towards the next evaluation steps.

Analysis Completion on Real-World Projects

Language	Total Projects	OmniDeps	Arcan	Depends	Entangle
C	31	31/31 (100%)	27/31 (87%)	30/31 (97%)	—
C++	31	30/31 (97%)	16/31 (52%)	25/31 (81%)	—
Java	32	32/32 (100%)	18/32 (56%)	31/32 (97%)	19/32 (59%)
Python	30	30/30 (100%)	0/30 (0%)	29/30 (97%)	—
Rust	32	32/32 (100%)	—	—	0/32 (0%)
TOTAL	156	**155/156 (99%)**	61/124 (49%)	115/124 (93%)	19/64 (30%)

Table 9.4: Deliverable completion and extraction feasibility summary across 156 open-source projects.

As summarized in Table 9.4, OmniDeps completed execution on **155 out of 156 projects** (99.4%). Its only failure occurred on `mongodb/mongo` (a codebase of nearly 10 million lines of code), which terminated prematurely due to a stack overflow.

Baseline analyzers exhibited lower completion rates:

- **Depends** attained an overall completion rate of 92.7% (115/124 projects). Most of its failures were caused by the one-hour execution timeout on large repositories (such as `tensorflow`, `ClickHouse`, and `mongodb`), while on one project (`checkstyle` in Java) the tool halted due to a syntax parsing error.

- **Arcan** completed 61 out of 124 evaluated projects (49.2%), succeeding on 27/31 projects in C, 16/31 in C++, and 18/32 in Java, but producing no deliverables on Python (0/30). None of these failures were caused by timeouts, as the tool exited prematurely with unclear execution errors.
- **Entangle** completed 19 out of 64 projects (29.7%), succeeding on 19/32 Java repositories, but failing on all 32 Rust repositories due to unhandled runtime exceptions in its prototype implementation.

To address **RQ2**, we evaluated the pairwise structural concordance between OmniDeps and the comparator baseline analyzers across the real-world software dataset. Tables 9.5, 9.6, and 9.7 summarize the mean and median smoothed Jaccard similarity indices for nodes (J_{nodes}) and edges (J_{edges}), alongside directional discordance rates (Sup, Inf) against Depends, Arcan, and Entangle, respectively.

Table 9.5: Pairwise structural similarity and discordance: OmniDeps vs. Depends.

Language / Target	N	J_{nodes}	$\text{Sup}_{\text{nodes}}$	$\text{Inf}_{\text{nodes}}$	J_{edges}	$\text{Sup}_{\text{edges}}$	$\text{Inf}_{\text{edges}}$
Java	31	0.814 (0.849)	0.286	0.021	0.587 (0.609)	0.406	0.345
Python	29	0.852 (0.882)	0.161	0.048	0.323 (0.300)	1.474	0.805
Python (Excl. imports)	29	0.852 (0.882)	0.161	0.048	0.368 (0.387)	1.017	0.834
C (Raw)	30	0.191 (0.167)	14.017	0.619	0.115 (0.042)	44.307	1.636
C (Preprocessed)	31	0.204 (0.158)	7.218	0.733	0.136 (0.083)	37.777	3.993
C++ (Raw)	25	0.143 (0.127)	12.493	2.969	0.081 (0.039)	58.836	4.769
C++ (Preprocessed)	22	0.192 (0.151)	4.911	3.229	0.165 (0.083)	9.962	8.320

Values report means (medians in parentheses for Jaccard indices).

Table 9.6: Pairwise structural similarity and discordance: OmniDeps vs. Arcan.

Language / Target	N	J_{nodes}	$\text{Sup}_{\text{nodes}}$	$\text{Inf}_{\text{nodes}}$	J_{edges}	$\text{Sup}_{\text{edges}}$	$\text{Inf}_{\text{edges}}$
Java	18	0.630 (0.693)	0.796	0.003	0.580 (0.575)	0.759	0.092
C (Raw)[*]	27	0.012 (0.005)	124.745	55.130	0.004 (0.000)	189.370	1713.444
C (Preprocessed)[*]	27	0.012 (0.005)	120.744	56.049	0.004 (0.000)	1.000	1.000
C++ (Raw)[*]	16	0.008 (0.004)	323.708	230.044	0.000 (0.000)	620.438	526.854

Values report means (medians in parentheses for Jaccard indices).

[*]In C and C++, Arcan models dependencies at the file and header level (`IMPL_FILE`, `HEADER`), whereas OmniDeps extracts user-defined types, resulting in disjoint entity sets.

Table 9.7: Pairwise structural similarity and discordance: OmniDeps vs. Entangle.

Language / Target	N	J_{nodes}	$\text{Sup}_{\text{nodes}}$	$\text{Inf}_{\text{nodes}}$	J_{edges}	$\text{Sup}_{\text{edges}}$	$\text{Inf}_{\text{edges}}$
Java	19	0.770 (0.782)	0.199	0.196	0.600 (0.598)	0.359	0.366
Rust[†]	0 (32)	—	—	—	—	—	—

Values report means (medians in parentheses for Jaccard indices).

[†]Entangle encountered unhandled runtime exceptions on all 32 Rust repositories, producing no valid dependency output.

The empirical results in Tables 9.5, 9.6, and 9.7, alongside Figure 9.1, highlight several overarching patterns across the benchmark suite.

The Polyglot Workspace Traversal Effect A fundamental architectural factor that influences comparative metrics across the entire 156-project evaluation is OmniDeps' language-agnostic workspace traversal. Traditional static analysis tools operate as single-language silos governed by build-system configurations: Depends Python parses only `.py` files, Arcan parses only `.java` files within Maven or Gradle targets, and Entangle parses only Cargo crates. In contrast, OmniDeps recursively traverses the entire repository workspace, parsing all source files that match any supported grammar (C, C++, Java, Python, Rust) without relying on build-system manifests.

Real-world software repositories are frequently polyglot. Modern Python packages regularly incorporate native extensions for performance: in `pydantic`, OmniDeps also analyze the Rust core (`pydantic-core/src/`), extracting native structs, validators, and serializers that the Python-only analyzer in Depends completely ignores. Similarly, C and C++ repositories frequently contain auxiliary Python build scripts, code generators, and test harnesses (as in `redis`, `git`, and `FFmpeg`), while Java projects contain native JNI C/C++ bindings. Because OmniDeps faithfully extracts entities and dependencies across all languages present in the repository, whereas single-language baselines restrict extraction strictly to primary source trees, the multi-language components appear exclusively in OmniDeps' graph ($A \setminus B$). This structural scope difference naturally inflates the superiority metric (Sup) across the dataset, reflecting OmniDeps' holistic polyglot coverage rather than false-positive over-extraction.

Structural Agreement in Java. In Java, OmniDeps exhibits strong and consistent structural alignment across all three comparator tools, representing the highest overall concordance in the evaluation.

When compared against Depends, OmniDeps attains a mean J_{nodes} of 0.814 and an edge similarity J_{edges} of 0.587. The entity omission rate remains exceptionally low ($\text{Inf}_{\text{nodes}} = 0.021$), showing that OmniDeps recovers essentially all classes and packages identified by Depends. On the dependency tier, discrepancy rates are balanced between the two tools ($\text{Sup}_{\text{edges}} = 0.406$ and $\text{Inf}_{\text{edges}} = 0.345$), indicating that both analyzers capture a large shared relational core alongside tool-specific dependency subsets.

Against Arcan, OmniDeps achieves an edge similarity of $J_{\text{edges}} = 0.580$ with a near-zero entity omission rate ($\text{Inf}_{\text{nodes}} = 0.003$). This indicates that OmniDeps misses virtually none of the entities extracted by Arcan. The node superiority rate ($\text{Sup}_{\text{nodes}} = 0.796$) shows that OmniDeps consistently identifies a larger set of architectural vertices than Arcan ($J_{\text{nodes}} = 0.630$), while edge omission remains low ($\text{Inf}_{\text{edges}} = 0.092$).

Against Entangle, alignment is remarkably balanced across both entities and dependencies. Node similarity reaches $J_{\text{nodes}} = 0.770$ with symmetric discordance rates ($\text{Sup}_{\text{nodes}} = 0.199$ and $\text{Inf}_{\text{nodes}} = 0.196$). Edge similarity reaches $J_{\text{edges}} = 0.600$ (with $\text{Sup}_{\text{edges}} = 0.359$ and $\text{Inf}_{\text{edges}} = 0.366$), representing the highest pairwise edge concordance observed in the entire evaluation suite.

Across all three baselines, edge similarity converges tightly to approximately 0.58–0.60. This consistent level of concordance indicates that Java systems possess the most stable cross-tool architectural representations in the dataset, with OmniDeps reliably capturing the structural and behavioral dependencies identified by existing analyzers.

Structural Alignment in Python. On Python repositories, OmniDeps and Depends attain an average node Jaccard similarity of 0.852, with an entity omission rate of just $\text{Inf}_{\text{nodes}} = 0.048$. Several projects exceed 0.90 in node agreement, including `uvicorn` (0.973), `networkx` (0.970), `requests` (0.958), `fastapi` (0.952), `celery` (0.949), and `sqlalchemy` (0.946). This high concordance reflects the harmonization of method-local test fixtures, which in dynamic testing frameworks declare throwaway mock classes within test methods.

When evaluating all extracted dependency edges, edge agreement averages 0.323. This metric is heavily influenced by syntactic import asymmetry: OmniDeps captures module-level `import` directives as typed `includes` edges, whereas Depends Python omits them entirely. When isolating purely behavioral relations by excluding unmodeled import statements, edge agreement reaches an average of 0.368, with edge superiority dropping to 1.017 (down from 1.474). A detailed architectural analysis of the factors governing this edge disparity, including syntactic import asymmetry, relative inheritance resolution, and polyglot workspace traversal, is presented in Section 10.1.2.

Structural Alignment in C and C++. In C and C++ codebases, evaluating structural concordance reveals two distinct phenomena across the comparator tools: the impact of translation-unit preprocessing when comparing against Depends, and a fundamental granularity mismatch when comparing against Arcan.

When compared against **Depends**, analyzing raw source code without macro expansion and header inclusion makes it difficult to link types across files. As reported in Table 9.5, preprocessing translation units (`clang -E`) substantially improves concordance in both languages:

- In C, node superiority ($\text{Sup}_{\text{nodes}}$) decreases from 14.017 to 7.218, while edge Jaccard similarity increases from 0.115 to 0.136. On small, self-contained libraries such as `antirez/sds`, both tools extract identical graphs ($J_{\text{nodes}} = J_{\text{edges}} = 1.0$). On larger systems, preprocessing achieves moderate agreement: `sqlite` reaches $J_{\text{nodes}} = 0.399$ and $J_{\text{edges}} = 0.340$, `redis` achieves 0.311 and 0.157, and `FFmpeg` attains 0.308 and 0.326.

- In C++, preprocessing increases average edge similarity from 0.081 to 0.165, while reducing edge superiority from 58.836 to 9.962. Notable agreement is observed on modular codebases such as `ninja` ($J_{\text{nodes}} = 0.491$, $J_{\text{edges}} = 0.410$), `duckdb` ($J_{\text{nodes}} = 0.358$, $J_{\text{edges}} = 0.415$), `aria2` ($J_{\text{nodes}} = 0.402$, $J_{\text{edges}} = 0.387$), and `google/re2` ($J_{\text{nodes}} = 0.360$, $J_{\text{edges}} = 0.459$).

An in-depth analysis of the architectural trade-offs between macro expansion, header inlining, and compiler dependency resolution in C and C++ is presented in Section 10.1.3.

When compared against **Arcan**, direct pairwise comparison produces near-zero Jaccard overlap ($J_{\text{nodes}} \leq 0.012$, $J_{\text{edges}} \leq 0.004$, Table 9.6). This divergence is not an extraction defect, but reflects a fundamental modeling disparity between the tools:

- In Java, both tools represent classes and interfaces as nodes, aligning consistently (Table 9.6).

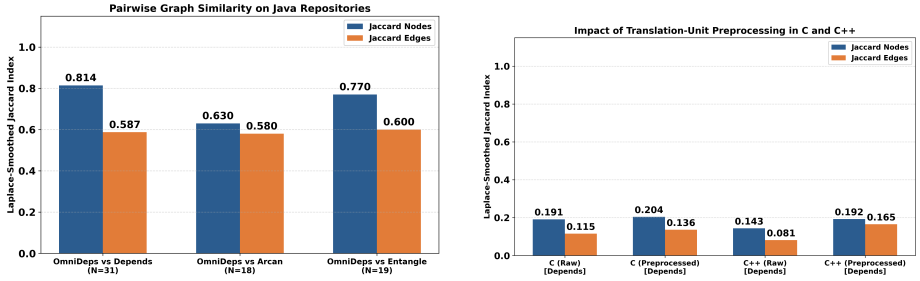
- In C and C++, Arcan focuses on detecting architectural smells at the file level, representing codebases strictly as implementation files (`IMPL_FILE`) and header files (`HEADER`), without extracting user-defined types (structs, unions, classes) as separate nodes.

- In contrast, OmniDeps and Depends represent C and C++ at the type level, extracting structs, classes, and unions, and grouping functions under their enclosing types.

Because Arcan outputs file paths while OmniDeps outputs type names, their node sets do not intersect ($V_{\text{Arcan}} \cap V_{\text{OmniDeps}} = \emptyset$). This difference illustrates how smell detection tools prioritize file-level organization over type hierarchies when analyzing C and C++ code. A formal analysis of this construct validity limitation is provided in Section 10.4.

Overall, structural concordance in C and C++ remains lower than in Java and Python. This disparity reflects the separate compilation model of C and C++: unlike the explicit package and module hierarchies of Java and Python, C and C++ rely on preprocessor inclusion across physical header files. In single-pass analyzers operating without a whole-program linker, shared header types are parsed repeatedly within each translation unit, producing duplicated representations and limiting cross-file entity alignment. A comprehensive analysis of these compilation and preprocessing trade-offs is presented in Section 10.1.3.

Extraction Feasibility in Rust. Finally, while pairwise cross-tool concordance cannot be computed for Rust due to the absence of viable baseline extractors (Depends and Arcan lack Rust support, while Entangle encountered unhandled runtime exceptions across all 32 repositories, as reported in Section 9.5.2), OmniDeps achieved a 100% extraction completion rate across the entire Rust dataset. The analyzer successfully recovered complete module trees, structs, traits, and method invocations directly from source workspaces without requiring compiler artifacts. A qualitative discussion of Rust's explicit modularity and its structural advantages for static analysis is provided in Section **??**.



(a) Java: OmniDeps vs. Baselines.

(b) C and C++: Raw vs. Preprocessed.

Figure 9.1: Pairwise architectural graph similarity (J_{nodes} and J_{edges}) on Java repositories (a) and under translation-unit preprocessing in C and C++ (b).

9.5.3 RQ3: Discrepancy Analysis Across Dependency Kinds

To address **RQ3**, we analyze how graph discrepancies are distributed across the five relationship types in our unified taxonomy: structural containment (`belongsTo`), type inheritance (`isChildOf`), interface implementation (`isImplementationOf`), behavioral coupling (`dependsOn`), and module inclusion (`includes`). Table 9.8 presents the shared edges and directional discrepancy rates evaluated across Java repositories for the two baseline tools providing fine-grained edge classifications (Depends and Arcan). In addition, Table 9.9 presents the corresponding categorical breakdown for Python against Depends, evaluated under the harmonized extraction model that accounts for method-local test closures and import asymmetries.

Table 9.8: Categorical edge discrepancy rates across relationship types between OmniDeps (A) and baseline analyzers (B) on Java repositories.

Relation Class	**OmniDeps vs. Depends** ($N = 31$)			**OmniDeps vs. Arcan** ($N = 18$)		
	Shared $\lvert A \cap B \rvert$	$\Delta_{A \setminus B}$	$\Delta_{B \setminus A}$	Shared $\lvert A \cap B \rvert$	$\Delta_{A \setminus B}$	$\Delta_{B \setminus A}$
`belongsTo`	6,419.1	17.0%	**1.8%**	3,295.7	36.7%	**0.3%**
`isChildOf`	3,115.1	24.9%	**8.7%**	880.1	61.3%	**2.4%**
`isImplementationOf`	0.0	100.0%	0.0%	0.0	11.8%	100.0%
`dependsOn`	27,890.7	26.1%	26.1%	11,374.1	43.4%	29.9%
`includes`	0.6	100.0%	100.0%	0.0	100.0%	0.0%

Table 9.9: Categorical edge discrepancy rates across relationship types between OmniDeps (A) and Depends (B) on Python repositories under harmonized extraction ($N = 29$).

Relation Class	**OmniDeps vs. Depends** ($N = 29$)		
	Shared $\lvert A \cap B \rvert$	$\Delta_{A \setminus B}$	$\Delta_{B \setminus A}$
`belongsTo`	1,415.1	10.9%	**4.4%**
`isChildOf`	246.9	39.0%	**22.2%**
`isImplementationOf`	0.0	–	–
`dependsOn`	629.2	72.7%	72.2%
`includes`	0.1	100.0%	61.1%

The breakdown by relationship type highlights the main sources of agreement and divergence between the tools:

Structural Containment (`belongsTo`). In Java, OmniDeps shows strong agreement with baseline tools on package and class containment hierarchies.

Against **Depends in Java**, the omission rate of OmniDeps is low: Depends contains only 1.8% of containment edges not found by OmniDeps ($\Delta_{B \setminus A} = 1.8\%$). OmniDeps identifies 17.0% additional containment edges ($\Delta_{A \setminus B} = 17.0\%$) because it synthesizes intermediate packages (Section 9.3.1) and links inner classes and static member types hierarchically.

Against **Arcan in Java**, the omission rate is nearly zero ($\Delta_{B \setminus A} = 0.3\%$). OmniDeps extracts 36.7% additional containment edges ($\Delta_{A \setminus B} = 36.7\%$) due

to synthetic intermediate package nodes that Arcan does not generate when explicit package declarations are absent.

Against **Depends in Python**, structural containment exhibits similarly high fidelity. The omission rate of OmniDeps is only 4.4% ($\Delta_{B\setminus A} = 4.4\%$), with an average of 1,415.1 shared containment edges per repository. OmniDeps extracts 10.9% additional containment edges ($\Delta_{A\setminus B} = 10.9\%$), reflecting complete synthesis of intermediate module packages for nested directory trees where explicit package declarations are omitted.

Subtyping and Interfaces (`isChildOf` and `isImplementationOf`). For inheritance and interface implementation, OmniDeps captures the vast majority of hierarchy edges extracted by baseline tools:

Against **Depends in Java**, OmniDeps achieves an omission rate of $\Delta_{B\setminus A} = 8.7\%$, with class inheritance and interface implementation unified under `isChildOf`.

Against **Arcan in Java**, the omission rate drops to $\Delta_{B\setminus A} = 2.4\%$. The discrepancy observed for `isImplementationOf` stems from taxonomy mapping differences: Arcan separates class extension (`extend`, mapped to `isChildOf`) from interface implementation (`implement`, mapped to `isImplementationOf`). In contrast, both OmniDeps and Depends group class inheritance and interface implementation under a unified subtyping relation (`isChildOf`). As a result, OmniDeps does not emit edges labeled `isImplementationOf`, but captures interface contracts directly under `isChildOf`.

Against **Depends in Python**, OmniDeps achieves a low omission rate of 22.2% ($\Delta_{B\setminus A} = 22.2\%$), sharing an average of 246.9 inheritance edges. OmniDeps identifies 39.0% additional subtyping edges ($\Delta_{A\setminus B} = 39.0\%$). This difference primarily stems from whole-workspace traversal, capturing inheritance in test suites and examples that Depends omits, as well as multiple-inheritance mixins where Depends classifies unresolved targets as speculative candidate types that are pruned during harmonization. Because Python does not provide an explicit interface construct, interface implementation is unified under `isChildOf`, leaving `isImplementationOf` unpopulated.

Behavioral Coupling and Method Calls (`dependsOn`). Behavioral coupling and method invocations account for the largest volume of edges across the benchmark, showing distinct resolution dynamics across ecosystems:

Against **Depends in Java**, OmniDeps and Depends share an average of 27,890.7 edges per repository, with balanced discrepancy rates ($\Delta_{A\setminus B} = 26.1\%$ and $\Delta_{B\setminus A} = 26.1\%$). OmniDeps extracts field-level type references and constructor instantiations that Depends omits, while Depends (which incorporates compiled classpath dependencies and standard library symbol tables) resolves chained method calls traversing standard library or external types, where OmniDeps deliberately bounds extraction to source code without loading external binaries.

Against **Arcan in Java**, OmniDeps shares an average of 11,374.1 behavioral edges, with $\Delta_{A\setminus B} = 43.4\%$ and $\Delta_{B\setminus A} = 29.9\%$. The divergence primarily reflects Arcan's AST filter, which prunes fine-grained field usages while resolving byte-code bindings.

Against **Depends in Python**, behavioral coupling shows symmetric discrepancy rates ($\Delta_{A\setminus B} = 72.7\%$ and $\Delta_{B\setminus A} = 72.2\%$), with 629.2 shared call edges per repository. This divergence highlights the inherent challenge of static call resolution in dynamically typed ecosystems: in the absence of runtime type feedback or whole-program abstract interpretation, untyped receiver invocations are resolved heuristically, causing each tool to infer different call targets. A detailed architectural investigation of this dynamic typing barrier is provided in Section 10.1.2.

Import Directives (`includes`). Baseline tools handle import directives differently across ecosystems:

Against **Arcan in Java**, file-level `import` statements are removed entirely ($\Delta_{B\setminus A} = 0.0\%$), retaining only resolved type references.

Against **Depends in Java**, both analyzers capture thousands of syntactic import statements per repository, but differ in their structural representation: while OmniDeps links imports to the declaring classes, Depends models them at the package level. Because the two representations bind dependencies to different abstraction levels (`class` vs. `package`), their edges do not overlap directly in strict pairwise matching (averaging only 0.6 shared edges).

Against **Depends in Python**, OmniDeps extracts all explicit syntactic import directives (`import` and `from ... import`), capturing modular file and package dependencies directly from the syntax tree. In contrast, Depends Python does not model import directives as architectural edges in its standard dependency graph ($\Delta_{A\setminus B} = 100.0\%$), leading to a complete directional asymmetry in this category.

Structural and Preprocessing Dynamics in C and C++. Unlike languages with first-class module systems (such as Java, Python, and Rust), categorical dependency decomposition in C and C++ is predominantly governed by the separate compilation model and translation-unit preprocessing. Because preprocessor macros obscure raw syntax, evaluating preprocessed sources (`clang -E`) resolves macro-masked declarations but inlines shared headers into every translation unit. This produces duplicate type declarations and external standard library dependencies across translation units, creating an unavoidable trade-off between complete type visibility and vertex multiplication in single-pass analyzers. A thorough architectural breakdown of preprocessor macro masking, header inlining, and the trade-offs of linemarker-based header filtering is presented in Section 10.1.3.

9.5.4 RQ4: Runtime Scalability, Throughput, and Speedup

To answer **RQ4**, we evaluated the execution runtimes, throughput scaling, and speedup factors of OmniDeps against Arcan, Depends, and Entangle across the 156-project dataset.

Table 9.10 presents the runtime and throughput metrics broken down by programming language and comparator tool.

To ensure methodologically sound comparisons despite differing completion rates and baseline failure modes (as detailed in Section 9.5.2), the evaluation

follows a strictly pairwise protocol:

- **Projects (N):** The mutual intersection of repositories successfully analyzed by both OmniDeps (Tool A) and the comparator (Tool B). OmniDeps' metrics vary across blocks because each tool successfully completed a different subset of projects.
- **Speedup Mean (Med):** Computed per project as $S_p = t_{B,p}/t_{A,p}$. We report the arithmetic mean alongside the sample median in parentheses, which provides a robust central measure against right-skewed runtime ratios.
- **Time and Throughput: Time (s)** is the average wall-clock execution time per project, and **Throughput** is the average rate in source lines of code per second (LOC/second).

Table 9.10: Runtime performance, analysis throughput, and speedup factors against baseline analyzers.

Language	Projects	(A) Time (s)	(A) Throughput	(B) Time (s)	(B) Throughput	Speedup Mean (Med)
OmniDeps (A) vs. Depends (B)						
C	30	4.31	97,382 LOC/s	30.36	10,576 LOC/s	**15.25x** (9.25x)
C++	25	22.62	49,297 LOC/s	76.48	14,266 LOC/s	**5.30x** (3.22x)
Java	31	14.87	86,601 LOC/s	55.82	13,878 LOC/s	**6.20x** (5.65x)
Python	30	6.96	59,905 LOC/s	50.60	2,906 LOC/s	**25.02x** (23.86x)
Overall	116	11.76	74,445 LOC/s	52.34	10,270 LOC/s	**13.21x** (7.72x)
OmniDeps (A) vs. Arcan (B)						
C	27	3.06	97,348 LOC/s	61.65	3,379 LOC/s	**62.07x** (34.77x)
C++	16	2.52	51,379 LOC/s	85.81	1,427 LOC/s	**52.36x** (38.21x)
Java	18	8.08	112,623 LOC/s	116.67	5,052 LOC/s	**23.75x** (13.01x)
Overall	61	4.40	89,798 LOC/s	84.22	3,361 LOC/s	**48.21x** (29.93x)
OmniDeps (A) vs. Entangle (B)						
Java	19	8.38	102,639 LOC/s	66.79	51,810 LOC/s	**4.01x** (2.86x)

As reported in Table 9.10 and illustrated in Figures 9.2 and 9.3, OmniDeps achieves higher throughput and shorter execution runtimes compared to the baseline analyzers across all evaluated languages.

Analysis Throughput. Across the entire dataset, OmniDeps sustains an overall mean throughput of **74,445 lines of code per second**. Throughput is highest on C (97,382 LOC/s), Rust (97,487 LOC/s), and Java (86,601 LOC/s), analyzing most codebases in under three seconds.

In comparison, baseline tools achieve noticeably lower throughput: Depends averages 10,270 LOC/s, Arcan averages 3,361 LOC/s, and Entangle reaches 51,810 LOC/s on Java.

Speedup Relative to Baseline Analyzers. Runtime speedup varies depending on the baseline tool and the target language:

- Against **Depends**, OmniDeps achieves an average speedup of **13.21x**. The speedup is most pronounced on Python (averaging 25.02x, peaking at 74.44x on `peewee`) and on C (averaging 15.25x, peaking at 79.55x on `sds`). On compact repositories such as `pydantic`, Depends completed in comparable time, resulting in a minimum speedup of 0.88x.

- Against **Arcan**, OmniDeps achieves an average speedup of **48.21x**, with individual speedups reaching up to 116.73x on Java (`okhttp`) and 536.60x on C (`sds`).
- Against **Entangle**, OmniDeps achieves an average speedup of **4.01x**, peaking at 15.31x on `flink`.

Runtime Scaling Trends. Figures 9.2 and 9.3 plot execution runtimes against codebase size (CLOC) on a semi-log scale across the four comparative languages. Across all languages, OmniDeps exhibits predictable, near-linear scaling with respect to the number of lines of code. Even on repositories exceeding one million lines of code (such as `FFmpeg`, `qemu`, `elasticsearch`, or `godot`), execution completes within a few seconds to low tens of seconds, without exhibiting super-linear latency spikes.

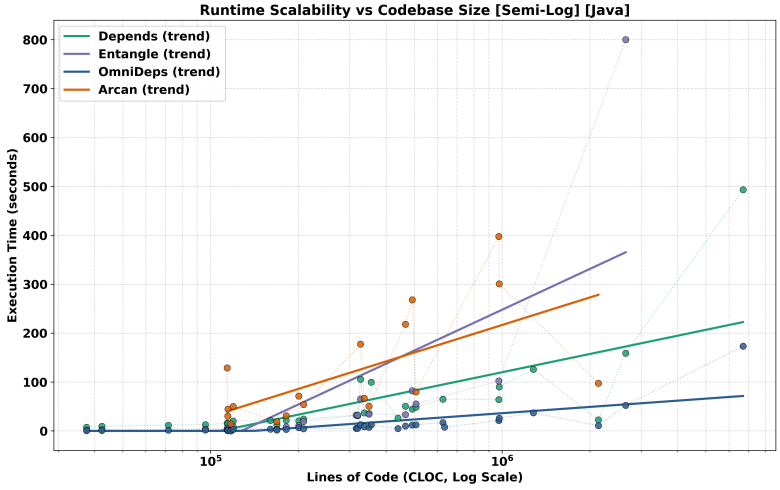
In contrast, the comparator tools show steeper scaling curves and higher dispersion as codebase size increases. In Java and Python, Depends scales consistently on small utilities but exhibits substantial runtime growth on large codebases (such as `django` or `sympy`), climbing into hundreds of seconds. Arcan exhibits both a noticeable initial latency even on modest projects and a steeper growth curve on larger systems, reflecting the overhead of full compiler frontend loading and resolution. As a result, the speedup of OmniDeps tends to grow with the scale of the target repository, making its lightweight parsing approach particularly advantageous on large codebases.

Overall, the empirical evaluation demonstrates that OmniDeps matches or surpasses the execution speed of existing baseline tools across all evaluated languages. Execution times remain reliably on the order of seconds across the vast majority of repositories, scaling predictably without runtime explosions even on systems with hundreds of thousands or millions of lines of code. These results confirm that the tool achieves practical efficiency and scalability suitable for real-world software analysis pipelines.

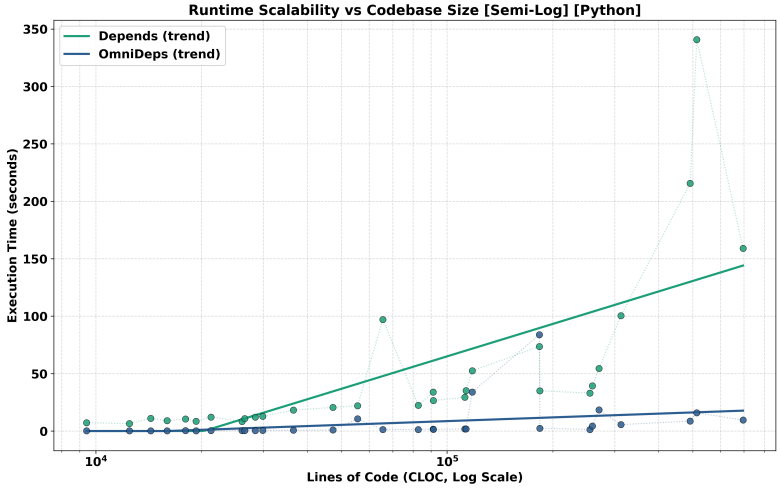
9.6 Answers to Research Questions

This section summarizes the experimental findings by providing direct answers to the four research questions guiding this evaluation:

- **RQ1 (Language Construct Coverage):** *OmniDeps successfully captures the foundational structural constructs and dependencies across all five supported languages in controlled benchmarks.* The synthetic test suite confirms that Concrete Syntax Tree queries correctly identify modular containers, structured types, inheritance links, and method calls in isolated test cases, providing a verified baseline for real-world analysis.
- **RQ2 (Cross-Tool Structural Alignment):** *OmniDeps demonstrates high structural alignment with established analyzers in structured object-oriented codebases, with lower edge overlap in dynamically typed languages and in C and C++ code.* In Java, OmniDeps aligns closely with all three comparators ($J_{\text{nodes}} = 0.814$, $J_{\text{edges}} = 0.587$, and only 2.1% entity omission against Depends). In Python, entity alignment achieves $J_{\text{nodes}} = 0.852$ with an entity omission rate of just 4.8%, while edge agreement reaches 0.323

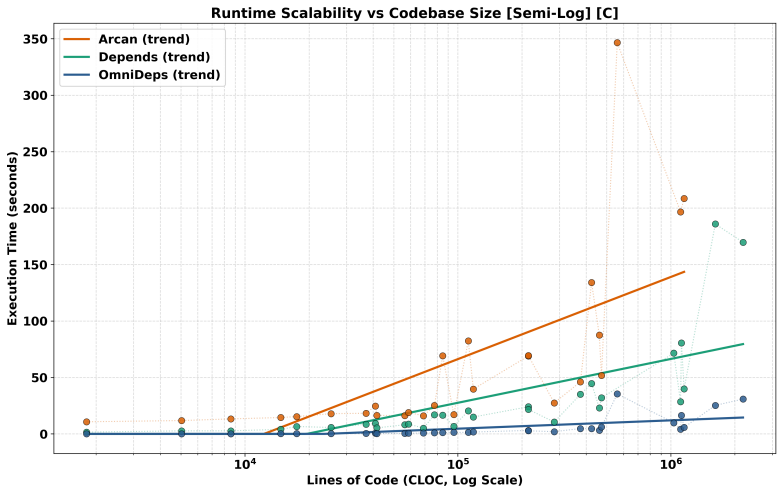


(a) Java: OmniDeps vs. Depends, Arcan, and Entangle.

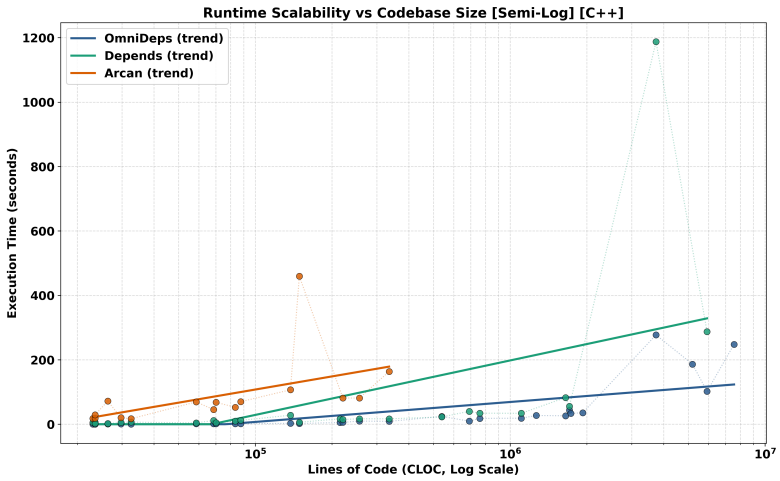


(b) Python: OmniDeps vs. Depends.

Figure 9.2: Empirical runtime scalability versus codebase size (CLOC) on semi-log scale for Java and Python.



(a) C: OmniDeps vs. Depends and Arcan.



(b) C++: OmniDeps vs. Depends and Arcan.

Figure 9.3: Empirical runtime scalability versus codebase size (CLOC) on semi-log scale for C and C++.

overall and 0.368 when isolating behavioral interactions. In C and C++, translation-unit preprocessing substantially improves edge agreement with Depends (from 0.081 to 0.165 in C++).

- **RQ3 (Discrepancy Analysis Across Dependency Types):** *Discrepancies concentrate in behavioral call-graph resolution and in differences of representation granularity, while containment hierarchies show near-complete agreement.* On Java, containment hierarchies exhibit low omission rates ($\leq 1.8\%$), whereas behavioral call edges reflect the trade-off between source-level syntax extraction and compiler-backed resolution of standard library and external classpath types. On Python, structural containment similarly achieves high agreement with an omission rate of only 4.4%, while behavioral call edges exhibit symmetric discrepancy ($\approx 72\%$) driven by heuristic dynamic dispatch without runtime type feedback. Furthermore, tools designed for smell detection (like Arcan in C/C++) model code at the file level, producing disjoint vertex sets compared to type-level graphs.
- **RQ4 (Computational Scalability and Throughput):** *OmniDeps achieves substantial speedups over existing tools, scaling essentially linearly with codebase size and completing execution in the order of seconds across the vast majority of repositories.* OmniDeps sustains an average throughput of 74,445 LOC/s, achieving average speedups of 13.2x over Depends, 48.2x over Arcan, and 4.0x over Entangle, while avoiding runtime explosions even on codebases exceeding one million lines of code.

Chapter 10

Discussion

The empirical evaluation presented in Chapter 9 answered the four research questions guiding this study, demonstrating the feasibility, structural concordance, and computational scalability of OmniDeps across 156 real-world software repositories and five programming languages. However, interpreting these quantitative findings requires stepping back from raw similarity metrics and runtime curves to examine the broader architectural, methodological, and practical implications of a language-agnostic approach to static analysis.

This chapter synthesizes the empirical outcomes into a discussion structured around four primary themes:

1. **Empirical Findings Across Paradigms**: We examine how syntax-directed dependency extraction interacts with different type systems and compilation models across object-oriented, dynamic, procedural, and systems programming languages.

2. **Efficacy of the Language-Agnostic Approach**: We evaluate the trade-offs between decoupling the analyzer from native build toolchains and relying on compiler-coupled semantic resolution.

3. **Practical Implications**: We discuss how software engineering practitioners and tool authors can integrate fast, language-agnostic dependency extraction into continuous integration workflows and downstream architectural smell detectors.

4. **Threats to Validity**: We analyze the construct, internal, and external limitations that bound the interpretation and generalizability of our results.

10.1 Empirical Findings Across Paradigms

Our empirical findings uncover fundamental differences in how programming language paradigms interact with syntax-directed static dependency extraction. Rather than reflecting uniform performance across the dataset, the results highlight distinct extraction dynamics across object-oriented, dynamic, procedural, and modern systems languages.

10.1.1 Object-Oriented Parity in Java

The most prominent empirical finding in object-oriented environments is the near-complete convergence of architectural entities and the high degree of relational concordance between OmniDeps and compiler-backed analyzers. As reported in Section 9.5.2, OmniDeps exhibited negligible entity omission against both Depends and Arcan, while achieving consistent edge similarity across all three baseline tools.

This convergence validates a foundational architectural hypothesis: in explicitly typed, modular object-oriented languages, Concrete Syntax Trees contain virtually all structural information required for architectural dependency modeling. In Java, source code maps directly to the architectural structure:

- Source files map cleanly to canonical package hierarchies via explicit package declarations.
- Structured types such as classes, interfaces, enums, and records are explicitly declared together with their subtyping relations.
- Dependency targets are explicitly imported via import statements or qualified inline references.

Because compilation units in Java cannot inject arbitrary global preprocessor macros or alias global namespaces dynamically, OmniDeps' two-phase algebraic query model resolves nearly all compiler-identified architectural entities without requiring access to external binary dependencies or a live Java Virtual Machine.

As shown in Section 9.5.3, containment relationships agree almost completely across tools. The remaining edge differences come from different extraction boundaries for usage dependencies rather than missing entities, confirming that syntax queries achieve accuracy comparable to compiler-based tools.

10.1.2 Resolution Limits in Dynamic Languages

In contrast to object-oriented environments, evaluating Python codebases demonstrates how graph harmonization reconciles divergent extraction scopes in dynamic ecosystems: while entity identification attains high concordance ($J_{\text{nodes}} = 0.852$) with an entity omission rate of just 4.8%, call-level dependency alignment achieves $J_{\text{edges}} = 0.323$, rising to 0.368 when isolating behavioral relations from unmodeled syntactic imports. Comparing the extracted graphs shows that two main factors explain these results: the limits of dynamic typing for method calls, and how method-local test fixtures are handled at the entity level.

Dynamic Typing and Call Divergence. The moderate edge similarity reflects the limits of syntax-only static analysis in dynamically typed languages. In Python:

- **Absence of Static Type Annotations**: In idiomatic Python codebases, function arguments and local variables frequently omit type signatures. Without runtime information or expensive data-flow analysis, a syntax-only analyzer cannot determine the concrete type of an unannotated parameter.
- **Late Binding and Duck Typing**: Method invocations are dispatched dynamically by attribute name lookup rather than static virtual method tables.

While OmniDeps leverages static type annotations [19] whenever present in function signatures and variable declarations, real-world Python repositories in the dataset adopt type annotations only partially or inconsistently [40]. In unannotated code, both OmniDeps and Depends inevitably arrive at divergent heuristics when resolving ambiguous method calls.

Node Disparity and Method-Local Closures. At the entity tier, an initial unharmonized comparison indicated substantial node discrepancies on several test-heavy repositories, such as `sqlalchemy` ($J_{\text{nodes}} = 0.485$), `marshmallow` (0.339), and `pydantic` (0.234). A structural inspection of the discordant entity sets revealed the exact methodological root cause: a fundamental scope disagreement concerning **method-local classes** within unit test suites. In Python testing frameworks, test cases extensively declare throwaway mock models and schemas directly inside individual test functions. Depends extracts all classes found in the AST, treating classes declared inside test functions as regular architectural types. Empirical quantification across the extracted graphs revealed that method-local classes accounted for **70.9%** of all types extracted by Depends in Pydantic, **54.3%** in SQLAlchemy, and **61.2%** in Marshmallow.

In contrast, OmniDeps extracts types only at module and class scope, ignoring method-local classes because test mocks have no visibility outside their enclosing function.

When the graph harmonization adapter applies Semantic Entity Lifting (Section 9.3.1) to classify method-local classes as volatile entities and lift their external references to the enclosing class or module, entity agreement realigns across the dataset:

- In `sqlalchemy`, node Jaccard similarity rises from 0.485 to **0.946**, while node inferiority drops from 1.030 to **0.024**.
- In `marshmallow`, node Jaccard increases from 0.339 to **0.782**, with node inferiority dropping from 1.683 to **0.005**.
- Across all 29 evaluated Python repositories, mean node Jaccard improves from 0.740 to **0.852**, and mean entity omission drops by **85.5%**, from 0.332 down to **0.048**.

This confirms that OmniDeps captures over 95% of architectural entities in Python, showing that the initial gap was caused by test-local classes rather than missing application code.

Edge Superiority and Import Asymmetry. A notable characteristic of the comparative edge metrics on Python repositories is the elevated edge superiority ($\text{Sup}_{\text{edges}} = 1.474$ mean), indicating that OmniDeps extracts considerably more edges than Depends. Comparing the edge differences reveals three main reasons for these extra edges:

- **Syntactic Import Asymmetry**: The largest single component of edges unique to OmniDeps is syntactic module inclusion. OmniDeps extracts every explicit import statement as a first-class `includes` edge, such as 18,494 in `sympy` and 5,982 in `scikit-learn`. In contrast, Depends Python bypasses file-level import statements almost entirely, generating only 47

`includes` edges in `sympy` and 22 in `scikit-learn`. Because the global edge Jaccard metric pools all relation categories into a single unweighted set, this categorical asymmetry accounts for over half of all edges unique to OmniDeps in large scientific libraries.

- **Cross-Module Inheritance Resolution**: In subtyping hierarchies (`isChildOf`), OmniDeps identifies 1,041 inheritance arcs in `scikit-learn` compared to 491 in Depends. A structural diff reveals two main reasons for this difference: over half of the additional edges (50.5%) come from test suites and examples where mock estimators extend base classes, which OmniDeps captures through whole-workspace traversal. Furthermore, on multiple-inheritance mixins, Depends frequently marks unresolved targets as speculative candidate types that are pruned during harmonization, whereas OmniDeps resolves declared base classes directly across the project Scope Tree.
- **Language-Agnostic Polyglot Workspace Traversal**: Modern software projects are increasingly polyglot. While Depends Python parses only `.py` files within declared roots, OmniDeps recursively analyzes all source files matching any supported grammar across the repository workspace. For instance, in `pydantic`, OmniDeps also parses the native Rust core, extracting its struct definitions, validators, and serializers, whereas single-language Python analyzers treat compiled extensions as opaque external binaries.

Thus, the higher edge count comes from broader syntactic coverage, complete workspace traversal, and multi-language files, rather than false-positive dependencies.

Behavioral Concordance Without Imports. To verify that the aggregate edge metric is distorted by the unmodeled import statements in Depends, we performed a controlled evaluation isolating purely behavioral interactions (invocations, type usage, and inheritance) by excluding syntactic `includes` edges across the Python dataset ($N = 29$).

Across all 29 repositories, the median edge Jaccard similarity increases from 0.300 to **0.387** (mean from 0.323 to **0.368**), while the median edge superiority index drops below parity to **0.889** (mean falls from 1.474 to **1.017**). The effect is particularly pronounced in large libraries where OmniDeps captures extensive import graphs, such as `scikit-learn` (J_{edges} rising from 0.209 to **0.317**) and `django` ($\text{Sup}_{\text{edges}}$ dropping from 1.584 to **0.951**). This proves that when comparing purely behavioral dependencies on an identical semantic plane, syntactic static analysis in dynamically typed languages achieves nearly 40% behavioral concordance with compiler-grade analyzers. Table 10.1 summarizes the progressive impact of method-local type scoping and behavioral edge isolation across the Python benchmark suite.

10.1.3 Preprocessing and Headers in C and C++

The empirical evaluation of C and C++ repositories highlights a central trade-off driven by two factors: macro expansion and textual header inclusion.

Table 10.1: Ablation of graph harmonization and import filtering across Python repositories ($N = 29$).

Harmonization Mode	J_{nodes}	$\text{Sup}_{\text{nodes}}$	$\text{Inf}_{\text{nodes}}$	J_{edges}	$\text{Sup}_{\text{edges}}$	$\text{Inf}_{\text{edges}}$
1. Baseline (Raw Entities & Edges)	0.740 (0.801)	0.159	0.332	0.296 (0.279)	1.473	1.211
2. Scoped (Method-Local Types Filtered)	0.852 (0.882)	0.161	0.048	0.323 (0.300)	1.474	0.805
3. Behavioral (Excluding Syntactic Imports)	0.852 (0.882)	0.161	0.048	0.368 (0.387)	1.017	0.834

Values report arithmetic means across 29 Python repositories, with medians in parentheses for Jaccard indices.
Mode 2 prunes method-local test fixtures in Depends via Semantic Entity Lifting, dropping entity omission by 85.5%.
Mode 3 evaluates purely behavioral dependencies, reducing median $\text{Sup}_{\text{edges}}$ from 1.338 to 0.889.

Macro Masking in Raw Sources. In raw, unprocessed C and C++ source code, extensive use of preprocessor macros fundamentally obscures the underlying syntax. Macros are routinely used to declare function wrappers, synthesize struct and class definitions, configure template boilerplate, declare dispatch tables, and implement conditional compilation. As shown in Table 9.5, analyzing raw source code produces modest edge Jaccard similarities of 0.115 in C and 0.081 in C++. Tree-sitter parsers, encountering macro invocations that do not conform to standard C or C++ grammar, frequently classify complex macro statements as generic call expressions or syntax error nodes, causing referenced classes, structs, and types to remain unlinked.

Preprocessing vs. Header Duplication. When translation units are preprocessed via the compiler preprocessor (`clang -E`), macros are fully expanded, conditional directives are evaluated, and all included header files are textually inlined directly into the source stream. As documented in Section 9.5.2, this produces a substantial boost in structural recall: in C++, edge Jaccard agreement doubles from 0.081 to 0.165 (reaching 0.459 on `re2` and 0.320 on `ninja`), while node inferiority drops significantly. At the categorical level (Section 9.5.3), macro expansion resolves masked type declarations and function signatures, cutting structural containment omission in C++ nearly in half (from 93.5% down to 47.4%) and substantially reducing behavioral dependency omission (from 92.6% to 67.8%).

However, preprocessing causes header duplication. In C and C++, headers contain not only type declarations, but class definitions, inline member functions, and template implementations required by the separate compilation model. Because each preprocessed file is parsed as a self-contained translation unit, entities defined in shared headers are replicated across every translation unit that includes them. This replication inflates both node superiority ($\text{Sup}_{\text{nodes}} = 7.22$ in C and 4.91 in C++) and the proportion of edges unique to OmniDeps ($\Delta_{A \setminus B}$), as OmniDeps extracts these duplicated declarations directly from each expanded syntax tree without a whole-program linking step.

System Headers and Enum Granularity. Looking closely at the node differences between OmniDeps and Depends reveals three main causes:

- **Standard Library and System Header Inclusion**: Under translation-unit preprocessing, the preprocessor expands not only local project head-

ers, but also system headers (such as the C runtime, POSIX headers, or C++ Standard Library templates), injecting standard types into each file. While OmniDeps extracts syntax-level struct and class definitions directly from the parsed concrete syntax tree (CST), Depends leverages Eclipse CDT's build configuration and symbol indexer to filter out external system include paths and deduplicate indexed symbols.

- **Enum Variant Harmonization Granularity**: In C and C++, enumerations are widely used (including scoped `enum class` constructs in modern C++). In the graph harmonization model, OmniDeps preserves each individual enum constant (variant) as an architectural component in its hierarchical scope tree. In contrast, Depends models enum constants as scalar variables, which the restricted harmonization filter prunes as volatile entities. In projects like `redis`, this granularity difference alone accounts for roughly half of the entire node divergence (673 out of 1,343 nodes missing in Depends).
- **Vendored Directory Traversal**: While build-system-coupled analyzers restrict extraction to targets declared in primary build scripts, OmniDeps performs a recursive workspace walk, parsing vendored dependencies nested in subdirectories (such as bundled `jemalloc` and `lua` sources in `redis`) as well as anonymous typedefs.

Entity omission dynamics, however, differ markedly between C and C++. In well-structured C codebases such as `git` ($\text{Inf}_{\text{nodes}} = 0.057$), `curl` (0.140), and `redis` (0.202), entity omission remains low, indicating that OmniDeps captures the vast majority of core types extracted by Depends, and that the low Jaccard agreement is primarily driven by header duplication inflating the denominator. Across the full C dataset, the median entity omission remains moderate ($\text{Inf}_{\text{nodes}} = 0.300$), though projects with complex macros (such as `htop` and `tinycc`) increase the mean to 0.733. In contrast, C++ exhibits substantially higher entity omission (mean $\text{Inf}_{\text{nodes}} = 3.229$, median 1.825). While modular C++ projects like `ninja` (0.204), `duckdb` (0.206), and `re2` (0.240) attain low omission, template-heavy codebases (such as `googletest` and `protobuf`) produce high divergence, as Eclipse CDT resolves instantiated template specializations and nested member types that syntax-directed AST extraction cannot synthesize without semantic evaluation. Thus, while header duplication inflates the denominator across both languages, genuine semantic omission also contributes significantly in C++.

Incomplete Types and Header Stripping. To address this duplication, our exploratory investigation evaluated stripping inlined header declarations using compiler linemarkers. However, this experiment highlighted a fundamental constraint of compiler-based frontends: removing header definitions leaves source files with incomplete or undeclared types. In C and C++, semantic analysis strictly requires complete type definitions to resolve member access, verify class inheritance, bind out-of-line member functions (`Class::method`), and instantiate templates. When headers were stripped, Eclipse CDT within Depends failed to resolve symbols and encountered unhandled parse errors across nearly every project, demonstrating that semantic compiler frontends cannot operate

without complete header visibility in each translation unit. In contrast, OmniDeps' query engine tolerates incomplete forward declarations because Tree-sitter performs grammar-directed structural extraction without requiring complete type layouts.

These results show that in C and C++, source files cannot be treated as wholly isolated units of analysis. Handling header-declared entities accurately requires linking or deduplicating identical class, struct, and function declarations across translation units after parsing, while preserving their architectural dependencies.

10.2 Efficacy of the Language-Agnostic Approach

A central question guiding this thesis is whether developing a language-agnostic, syntax-directed static analyzer is architecturally and practically worthwhile compared to maintaining dedicated, compiler-coupled frontends.

The empirical results show that this trade-off is well justified for architectural analysis. While compiler-based tools offer deeper type resolution in specialized cases like complex C++ templates, syntax-based analysis provides three main practical advantages: reliability, speed, and maintainability.

10.2.1 Decoupling from Build Systems

A key advantage of OmniDeps is its much higher completion rate compared to tools coupled to build environments. Tools that integrate with compiler frontends or build systems, such as Arcan and Entangle, require projects to be configured and resolvable on the host machine. When applied to heterogeneous open-source repositories, these prerequisites frequently fail due to missing external dependencies, platform-specific configurations, uninitialized submodules, or outdated build scripts, causing compiler-coupled tools to abort on a large fraction of the benchmark corpus.

In contrast, OmniDeps operates under the assumption that **the analyzed source code already compiles within its native environment**, decoupling static analysis from build-system execution. Because it operates directly on raw source files on disk without requiring external dependencies or build configuration, OmniDeps completed analysis across almost the entire dataset. For large-scale empirical studies or multi-language enterprise repositories, extracting high-fidelity architectural dependencies with zero build setup provides substantially higher practical reliability than relying on fragile build pipelines.

10.2.2 Precision Versus Scalability

Traditional compiler frontends dedicate significant computational resources to whole-program semantic analysis, including control-flow construction, interprocedural points-to resolution, and type checking. However, for **architectural dependency analysis**, much of this verification is redundant when the source code has already been validated by the developer's toolchain.

OmniDeps replaces multi-pass compiler verification with single-pass CST extraction followed by algebraic resolution over the Scope Tree. As shown in

Section 9.5.4, this design enables the analyzer to process large repositories in a few seconds, running one to two orders of magnitude faster than compiler-backed baselines. At the same time, the comparative evaluation in Java demonstrated that this speedup comes with minimal entity omission. For macro-architectural analyses such as cyclic dependency detection or component cohesion assessment, minor differences in fine-grained call edges do not alter the recovered architectural topology, making the trade-off between exhaustive type checking and high-throughput syntax analysis highly effective in practice.

10.2.3 Polyglot Extensibility

Building and maintaining a multi-language static analysis framework on top of compiler frontends requires integrating separate, incompatible ecosystems for each target language, such as Clang for C and C++ or Eclipse JDT for Java. Each compiler API evolves independently, exposes a different internal syntax model, and requires its own runtime environment and language-specific resolution pipeline.

In contrast, OmniDeps maintains a single unified pipeline across all supported languages. By relying on Tree-sitter for parsing, adding support for a new programming language only requires defining its extraction heuristics and scoping parameters in the configuration layer to map CST nodes into the Intermediate Representation. The underlying algebraic query engine, Scope Tree indexing, and graph synthesis stages remain completely unchanged and language-independent, substantially reducing the engineering effort required to support and maintain multiple languages.

10.3 Practical Implications

The findings of this thesis have direct practical implications for software engineering workflows and architectural analysis tools.

Continuous Integration Auditing

Traditional dependency auditing tools often run only in nightly batch jobs because their execution times range from minutes to hours. Because OmniDeps analyzes large codebases in seconds without requiring a build, it can run directly in **continuous integration pipelines** and pre-commit checks. Development teams can thus enforce architectural constraints, such as detecting layer violations or cyclic component dependencies, during routine code review before changes are merged.

Universal Frontend for Smell Detectors

Architectural analysis platforms such as *Arcan* [7], *DV8* [15], and *SonarQube* invest considerable effort in developing and maintaining language-specific parsers and extractors. OmniDeps' Intermediate Representation and normalized dependency graph exports can serve as a **universal dependency extraction frontend** for these downstream tools, allowing smell detectors to extend their coverage across C, C++, Java, Python, and Rust without maintaining separate compiler-coupled pipelines.

Polyglot Monorepo Governance

Modern software systems increasingly adopt polyglot repository structures, where services, native extensions, and shared libraries written in different languages coexist within a single workspace. While traditional analyzers operate within single-language silos, OmniDeps projects components from all supported languages into a single language-neutral Intermediate Representation with canonical qualified naming, providing a unified foundation for inspecting multi-language repositories within a single dependency multigraph.

10.4 Threats to Validity

We discuss the limitations of our study across construct, internal, and external validity.

Construct Validity

Threats to construct validity concern whether the evaluation metrics and graph transformations reflect genuine dependency extraction fidelity rather than representation artifacts.

- **Harmonization and Semantic Lifting**: To compare tools operating at different granularities, we lifted member-level interactions to their enclosing types and synthesized intermediate packages (Section 9.3.1). While this enables a direct comparison, it focuses the evaluation on architectural dependencies rather than statement-level details.
- **Manual Catalogs of Primitive Types**: OmniDeps identifies built-in and primitive types through predefined, per-language catalogs in order to synthesize terminal primitive endpoints rather than attempting workspace symbol resolution. Although these catalogs were constructed from official language specifications across all five languages, maintaining manual lists introduces a risk that uncommon platform-specific aliases may remain unclassified and be treated as external symbols.
- **Granularity Mismatch in C and C++**: In C and C++, while OmniDeps and Depends extract user-defined types such as structs, unions, and classes, Arcan models dependencies primarily at the physical file and header level. Because intersecting file-level and type-level vertex sets produces an empty overlap that would falsely suggest zero extraction capability, we explicitly excluded Arcan from the C and C++ structural similarity comparison (Section 9.5.2).
- **Polyglot Workspace Traversal vs. Single-Language Baselines**: Unlike baseline tools that restrict analysis to a single target language or build manifest, OmniDeps recursively parses all supported source files across a repository workspace. As examined in Section 9.5.2 and Section 10.1.2, secondary languages embedded in polyglot repositories are extracted exclusively by OmniDeps, which systematically increases directional superiority metrics without indicating false-positive couplings.

- **Metric Sensitivity and Smoothing**: Standard Jaccard similarity becomes indeterminate on sparse subgraphs and cannot distinguish between under-approximation and over-approximation. To mitigate this, we combined Laplace-smoothed Jaccard formulations with directional superiority and inferiority indices (Section 9.3.2) to evaluate extra and omitted elements independently.

Internal Validity

Threats to internal validity concern confounding factors in the experimental setup, analysis pipeline, and measurement protocol that could bias the observed results.

- **C and C++ Preprocessing Trade-offs**: As analyzed in Section 10.1.3, evaluating C and C++ codebases involves an inherent trade-off between macro masking in raw source files and header replication in preprocessed translation units, while stripping inlined headers breaks compiler-coupled baselines due to incomplete forward types. To avoid biasing the evaluation toward a single operating point, we evaluated and reported both raw and preprocessed configurations side by side.
- **Unmodeled C-Style Variadic Functions**: OmniDeps does not explicitly model C-style variadic parameter lists in its intermediate representation. Because untyped variadic arguments do not introduce named type references in function signatures, no static type dependency edge is lost at the declaration site; however, concrete argument types passed at variadic call sites cannot be resolved statically without interprocedural data-flow analysis.
- **Dynamic Language Features and Reflection**: As discussed in Section 10.1.2, purely syntactic static analysis cannot observe runtime reflection, dynamic attribute dispatch, or runtime symbol table mutation in dynamically typed languages such as Python. This represents an intrinsic ceiling on static call recall whenever type annotations are absent.
- **Runtime Measurement Variance and Execution Environment**: Comparing a native Rust binary against JVM-based analyzers such as Arcan and Depends introduces potential runtime differences due to virtual machine startup, just-in-time compilation, and garbage collection. To ensure a fair comparison, each repository was analyzed in an isolated process session under identical hardware constraints (Section 9.4), and execution times were aggregated across multiple runs using both arithmetic means and medians to guard against outliers.

External Validity

Threats to external validity concern the generalizability of our conclusions to other programming paradigms and software systems.

- **Generalizability Across Programming Paradigms**: Although our evaluation spans five languages, all five are imperative, procedural, or

object-oriented. Our findings may not generalize directly to purely functional or declarative languages, where software structure is based on function composition or algebraic modules rather than types and call graphs.

- **Dataset Representativeness**: The 156 evaluated repositories were sourced exclusively from open-source projects hosted on GitHub, which may not fully reflect proprietary enterprise codebases with unconventional layouts. To mitigate selection bias, the benchmark corpus was curated to span nearly 98 million lines of code across diverse domains, ranging from compact libraries to monolithic systems exceeding one million lines of code.

Chapter 11

Conclusions and Future Work

11.1 Conclusions

Architectural static analysis and dependency extraction represent fundamental pillars of modern software engineering, underpinning critical activities ranging from architectural smell detection and refactoring recommendation to modularity assessment and technical debt monitoring. However, the growing prevalence of multi-language software systems has highlighted a fundamental limitation in traditional static analysis tools: their reliance on language-specific compiler frontends and dedicated analysis pipelines. Traditionally, supporting multiple programming languages requires engineering a separate extraction and resolution pipeline for each ecosystem, integrating distinct compiler frontends, custom parsers, and language-specific symbol resolvers. This fragmentation introduces substantial engineering complexity, duplicates core analysis logic across languages, and increases the effort required to develop and maintain multi-language tooling.

To address these limitations, this work designed, formalized, and empirically validated **OmniDeps**, a high-performance, language-agnostic static analysis framework that decouples architectural dependency extraction from native compiler toolchains. Operating under the foundational premise that the analyzed source code compiles successfully within its native build environment, OmniDeps shows that Concrete Syntax Trees (CSTs) can be directly queried and projected into normalized architectural dependency graphs without executing compiler type-checking passes or whole-program bytecode verification.

This research delivered three primary contributions:

1. **Intermediate Representation and Extraction Heuristics**: We established a strongly typed, mathematically grounded Intermediate Representation based on algebraic data types, designed to abstract modules, structured types, functions, and imports away from idiosyncratic language grammars. This representation is paired with a configurable, heuristic-driven extraction mechanism that projects heterogeneous Concrete Syntax Trees (CSTs) generated by Tree-sitter across different languages into a uniform structural model.

2. **Multi-Language Resolution Pipeline**: We designed and formalized a unified analysis pipeline in native Rust, achieving uniform dependency extraction across five diverse programming languages: C, C++, Java, Python, and Rust. To perform language-independent name resolution within this pipeline, we formulated an algebraic query language governed by three orthogonal primitives (`Find`, `Extract`, and `Call`) to model lexical lookups, member traversals, and behavioral invocations over a hierarchical Scope Tree.

3. **Rigorous Empirical Validation and Graph Harmonization**: To overcome the fundamental challenge of comparing heterogeneous static analyzers, we formulated a Graph Harmonization Framework based on canonical entity identification, semantic entity lifting, intermediate package synthesis, and unified edge taxonomies. Using this framework, OmniDeps was validated against synthetic benchmark oracles and benchmarked against three established analyzers (*Arcan*, *Depends*, and *Entangle*) across a dataset of **156 open-source repositories** comprising nearly 98 million lines of code.

The empirical evaluation confirmed the effectiveness and practical viability of the proposed approach. On controlled synthetic benchmarks, OmniDeps consistently recovered the expected structural entities and relationships across all five supported languages, validating both the extraction heuristics and the query resolution engine. When evaluated on large-scale open-source repositories, the framework demonstrated strong structural alignment with established analyzers in explicitly typed, object-oriented ecosystems, showing that syntax-directed extraction can closely approximate compiler-backed symbol tables for architectural modeling. At the same time, the cross-language analysis clarified the intrinsic boundaries of purely syntactic static analysis: in dynamically typed languages, entity and hierarchy identification remains accurate while behavioral call resolution is naturally bounded by the absence of runtime type information; in procedural systems, preprocessor macro masking and header inlining introduce an inherent trade-off between type visibility and declaration duplication across compilation units. Finally, by operating directly on source files without requiring build configuration or compilation, OmniDeps achieved high operational reliability across the benchmark corpus and scaled linearly with codebase size, completing analysis in a fraction of the time required by traditional tools.

In conclusion, this work proves that language-agnostic architectural dependency extraction is both theoretically sound and practically viable. By trading redundant compiler-level type verification for declarative CST pattern queries and decoupled lexical scope trees, OmniDeps bridges the gap between polyglot versatility, architectural fidelity, and real-time execution performance.

11.2 Future Work

While the primary focus of this thesis was the formalization and empirical validation of the intermediate representation, extraction heuristics, and resolution pipeline, the modular architecture of OmniDeps opens several avenues for future research and engineering extensions. The remainder of this section outlines the primary directions for future work.

Performance and Concurrency Optimizations

Throughout the development of this thesis, the primary objective was ensuring the correctness, sound formalization, and baseline algorithmic scalability of the intermediate representation and analysis pipeline. While the algorithmic complexity remains well bounded, sustaining high throughput on modern hardware, large-scale enterprise code bases and continuous integration pipelines can benefit from deeper concurrency across all stages of the pipeline.

Currently, OmniDeps exploits data-parallelism during both syntactic extraction, where source files are parsed concurrently across CPU cores, and graph synthesis, where dependency edges are extracted and deduplicated in parallel across workspace modules. However, the intermediate name resolution phase still executes sequentially on a single thread.

Future iterations of the framework can extend multi-core parallelism to the two sub-stages of name resolution:

- **Parallel Query Building**: During lexical substitution, the resolver currently traverses modules sequentially using a single shared symbol stack. Because lexical scope substitution within a top-level compilation unit is independent of sibling modules, maintaining an isolated symbol stack per module would allow query construction to execute in parallel across the module hierarchy.
- **Concurrent Query Execution**: During global query evaluation, all queries inspect a read-only global Scope Tree and a static primitive registry, while sharing a mutable cache of resolved inheritance hierarchies. Pre-computing superclass scope mappings during Scope Tree construction, or adopting a concurrent caching strategy, would allow independent queries to be evaluated in parallel across all modules.

Alternative Programming Paradigms

The design and empirical validation of OmniDeps have thus far focused on procedural and object-oriented languages. These languages share foundational architectural concepts, such as explicit file and package modules, structured types, and identifiable function or method declarations.

Although the Intermediate Representation was designed to be language-agnostic, extending it to support non-imperative paradigms represents a meaningful research opportunity:

- **Functional Languages**: In functional languages such as Haskell, OCaml, or Clojure, software architecture relies heavily on higher-order functions, currying, closures, and algebraic module systems. Capturing architectural coupling in these ecosystems will require refining the query algebra to track function composition and lexical binding without relying on class-like encapsulation.
- **Declarative and Logic Paradigms**: Languages such as Prolog rely on relations rather than traditional functions and classes. However, because relations can be treated similarly to functions for dependency analysis, this approach could also be extended to logic paradigms.

Analysis Engine and Modeling Refinements

Beyond concurrency and paradigm extensions, several architectural refinements to the extraction and resolution algorithms warrant future investigation:

- **Automated Discovery of Primitive Types**: OmniDeps currently maintains explicit per-language catalogs to identify built-in and primitive types and synthesize terminal primitive endpoints. Future work could explore automated primitive type identification through statistical frequency heuristics across syntax nodes or automated scraping of language specification headers, eliminating the need for manual list configuration when onboarding new languages.
- **Early Query Resolution for Superclasses**: Currently, resolving calls delegated to a superclass is executed entirely during the global query evaluation stage against the Scope Tree. An optimization avenue involves pre-resolving direct superclass bindings during the query building stage, rewriting superclass invocations into scoped queries before global execution to reduce query traversal depth.
- **Link-Time Symbol Deduplication for C and C++**: When analyzing preprocessed translation units in C and C++, types declared in shared headers are replicated across compilation units. Future iterations should introduce an intermediate deduplication layer analogous to a systems linker: identifying struct and class declarations with matching signatures originating from shared headers, and consolidating them into unified logical nodes in the Scope Tree. This would eliminate node inflation artifacts and align graph density with compiler-based tools without requiring build-system coupling.
- **Simplified Self-Reference Configuration**: In the current configuration schema, how a language refers to the current instance inside a method is split across separate settings: one for fixed keywords such as `this` or `self`, and another for conventions where the instance is passed as the first method parameter, as in Python. Since these mechanisms are mutually exclusive, they can be consolidated into a single configuration setting.

Inter-Language Dependency Detection

Finally, a more speculative direction emerged as a reflection on the architecture of the framework. Although cross-language interaction was never investigated or addressed during the course of this thesis, the fact that OmniDeps already projects source files written in different programming languages into a single intermediate representation and shared Scope Tree suggests an intriguing long-term research opportunity.

Currently, even when analyzing polyglot repositories, dependency resolution remains strictly confined within the boundaries of each individual language, meaning that calls crossing language boundaries are treated simply as unresolved external references. Future research could explore whether lightweight bridging heuristics can be layered on top of the unified representation to connect definitions and references across different languages. Such bridges could, for instance, link Foreign Function Interface calls such as the Java Native Interface

or Python native bindings to their underlying C, C++, or Rust implementations, or connect distributed services through shared Remote Procedure Call and Interface Definition Language schemas.

List of Figures

1.1 Rust and Python CST comparison from the code *a.b.c().d.e()* . . 2
1.2 The functional pipeline of the analyzer. 3

2.1 Small example of a Dependency Graph 9
2.2 Dependency graph resulting from a Cyclic Dependency smell. . . 10
2.3 Visual representation of the scope climbing algorithm: the search starts from the inner-most block and climbs up the hierarchy until the identifier is found. 15

3.1 Comparison between the $M \times N$ combinatorial explosion and the linear $M + N$ complexity achieved by introducing an IR. 18

4.1 State machine diagram illustrating the lifecycle of a Type Reference (τ) from syntactic extraction to semantic resolution. 30
4.2 UML class diagram representing the complete Intermediate Representation (IR) domain model, highlighting hierarchical composition and semantic references via `TypeRef`. 36

5.1 Architectural overview of the three-phase analysis pipeline, from source code to dependency graph. 40

6.1 Layered architecture of the syntactic extraction subsystem. . . . 47
6.2 Flowchart of the `walk_cst` traversal logic and subtree pruning mechanism. 49

7.1 Two-phase resolution architecture: query building (ρ_{build}) transforms AST paths into algebraic queries using the local Symbol Stack (Δ), while query execution (ρ_{exec}) evaluates queries across the global Scope Tree ($\mathcal{E}$). 62
7.2 Contiguous arena layout of the Scope Tree: scopes reside linearly in `Vec<Scope>` indexed by integer `ScopeId`, with parent relations modeled as scalar integers to eliminate pointer cycles. 68
7.3 Evaluation pathways of resolution queries: parallel execution tracks for lexical climbing (`Find`), structural member descent (`Extract`), and return type extraction (`Call`). 70
7.4 Dual-tracking evaluation of chained member access `user.address.zip`: maintaining the concrete syntactic access path alongside the resolved structural type for architectural dependency projection. 77

8.1 High-level pipeline of the graph construction phase (γ). The resolved module hierarchy ($\vec{\mathcal{M}} \in \mathcal{D}$) undergoes a first traversal pass to extract workspace vertices (V), a second pass to emit directed dependency edges (E), and a post-processing stage that guarantees referential closure and eliminates duplicate edges. 85
8.2 Fine-grained dependency multigraph generated for Listing 8.1. Containment relationships are captured by dashed `NestedIn` edges, while the type dependency `UsesFieldType` originates from the member field `Car::engine` rather than from the enclosing struct. Behavioral relationships are recorded via `AccessesField` and `Calls`. 90
8.3 Mapping structural containment to Cytoscape.js compound nodes. The raw flat graph (left) connects containers and members with explicit `ModuleContainment` and `NestedIn` edges. The exported representation (right) transforms containment into nested bounding boxes via the `parent` property, pruning structural edges and highlighting architectural dependencies. 93
8.4 Architecture of the interactive graph visualizer. The Rust analyzer backend exposes CLI and local HTTP API endpoints delivering Cytoscape-formatted JSON payloads. The SvelteKit frontend coordinates reactive state management (using Svelte 5 runes for filter toggles and search queries) with the Cytoscape.js canvas and the `fCoSE` physics layout extension. 94

9.1 Pairwise architectural graph similarity (J_{nodes} and J_{edges}) on Java repositories (a) and under translation-unit preprocessing in C and C++ (b). 111
9.2 Empirical runtime scalability versus codebase size (CLOC) on semi-log scale for Java and Python. 117
9.3 Empirical runtime scalability versus codebase size (CLOC) on semi-log scale for C and C++. 118

List of Tables

2.1 Taxonomy of software dependencies extracted via static analysis. 7
2.2 Key methods provided by the `TreeCursor` API for stateful CST navigation. 12

4.1 Module organization strategies across supported languages. . . . 38
4.2 Declaration linking and visibility configuration profiles. 39
4.3 Instance model, receiver resolution, and type anchor features. . . 39

5.1 Summary of pipeline transformations: operators, domain mappings, operational contexts, and execution scope. 45

6.1 Logical layer architecture of the extraction subsystem. 48
6.2 Summary of classifier predicates and their heuristic matching rules. 51
6.3 Module grouping strategy assignment for each supported language. 60

7.1 Examples of source expressions and their corresponding query algebra encodings for type inference and member chaining. . . . 63
7.2 Key resolution configuration parameters and their semantic effects. 81
7.3 Concrete resolution configuration values for each supported language. 82

8.1 Complete taxonomy of the 16 dependency edge kinds, organized by category. 88

9.1 Mapping of heterogeneous tool edge labels into the unified architectural taxonomy $\mathcal{R}_{\text{unified}}$. 102
9.2 Declared language support across evaluated static analysis tools. 105
9.3 Evaluation of extraction fidelity against ground-truth architectural oracles. 106
9.4 Deliverable completion and extraction feasibility summary across 156 open-source projects. 107
9.5 Pairwise structural similarity and discordance: OmniDeps vs. Depends. 108
9.6 Pairwise structural similarity and discordance: OmniDeps vs. Arcan. 108
9.7 Pairwise structural similarity and discordance: OmniDeps vs. Entangle. 108
9.8 Categorical edge discrepancy rates across relationship types between OmniDeps (A) and baseline analyzers (B) on Java repositories. 112

9.9 Categorical edge discrepancy rates across relationship types between OmniDeps (A) and Depends (B) on Python repositories under harmonized extraction ($N = 29$). 112
9.10 Runtime performance, analysis throughput, and speedup factors against baseline analyzers. 115

10.1 Ablation of graph harmonization and import filtering across Python repositories ($N = 29$). 124

A.1 Typographic and notational taxonomy adopted across the formal model. 144

Bibliography and Sitography

[1] M. Lehman, "Programs, life cycles, and laws of software evolution," *Proceedings of the IEEE*, vol. 68, no. 9, pp. 1060–1076, 1980, ISSN: 0018-9219. DOI: 10.1109/PROC.1980.11805. [Online]. Available: http://ieeexplore.ieee.org/document/1456074/.

[2] W. Cunningham, "The WyCash portfolio management system," in *Addendum to the proceedings on Object-oriented programming systems, languages, and applications (Addendum) - OOPSLA '92*, Vancouver, British Columbia, Canada: ACM Press, 1992, pp. 29–30, ISBN: 978-0-89791-610-3. DOI: 10.1145/157709.157715. [Online]. Available: http://portal.acm.org/citation.cfm?doid=157709.157715.

[3] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Software*, vol. 29, no. 6, pp. 18–21, Nov. 2012, ISSN: 0740-7459. DOI: 10.1109/MS.2012.167. [Online]. Available: http://ieeexplore.ieee.org/document/6336722/.

[4] M. Fowler and K. Beck, *Refactoring: improving the design of existing code* (The Addison-Wesley object technology series), 28. printing. Boston: Addison-Wesley, 2013, 431 pp., ISBN: 978-0-201-48567-7.

[5] F. Arcelli Fontana, P. Braione, and M. Zanoni, "Automatic detection of bad smells in code: An experimental assessment.," *The Journal of Object Technology*, vol. 11, no. 2, 5:1, 2012, ISSN: 1660-1769. DOI: 10.5381/jot.2012.11.2.a5. [Online]. Available: http://www.jot.fm/contents/issue_2012_08/article5.html.

[6] J. Garcia, D. Popescu, G. Edwards, and N. Medvidovic, "Identifying architectural bad smells," in *2009 13th European Conference on Software Maintenance and Reengineering*, Kaiserslautern, Germany: IEEE, 2009, pp. 255–258, ISBN: 978-1-4244-3755-9. DOI: 10.1109/CSMR.2009.59. [Online]. Available: http://ieeexplore.ieee.org/document/4812762/.

[7] F. A. Fontana, I. Pigazzini, R. Roveda, and M. Zanoni, "Arcan: A tool for architectural smells detection," in *2017 IEEE International Conference on Software Architecture Workshops (ICSAW)*, IEEE, 2017, pp. 282–285.

[8] M. Brunsfeld *et al.*, *TreeSitter: An incremental parsing system for programming tools*, 2018. [Online]. Available: https://tree-sitter.github.io/tree-sitter/.

[9] B. C. Pierce, *Types and programming languages*. Cambridge, Massachusetts: MIT Press, 2002, 1 p., ISBN: 978-0-262-25681-0.

[10] E. Vacchi and W. Cazzola, "Neverlang: A framework for feature-oriented language development," *Computer Languages, Systems & Structures*, vol. 43, pp. 1–40, Oct. 2015, ISSN: 14778424. DOI: 10.1016/j.cl.2015.02.001. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S1477842415000056.

[11] R. C. Martin, J. Grenning, S. Brown, and K. Henney, *Clean Architecture: a craftsman's guide to software structure and design* (Robert C. Martin series). Boston Columbus Indianapolis New York San Francisco Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto Delhi Mexico City São Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo: Prentice Hall, 2018, 400 pp., ISBN: 978-0-13-449416-6.

[12] W. P. Stevens, G. J. Myers, and L. L. Constantine, "Structured design," *IBM Systems Journal*, vol. 13, no. 2, pp. 115–139, 1974, ISSN: 0018-8670. DOI: 10.1147/sj.132.0115. [Online]. Available: http://ieeexplore.ieee.org/document/5388187/.

[13] F. Arcelli Fontana, V. Lenarduzzi, R. Roveda, and D. Taibi, "Are architectural smells independent from code smells? an empirical study," *Journal of Systems and Software*, vol. 154, pp. 139–156, Aug. 2019, ISSN: 01641212. DOI: 10.1016/j.jss.2019.04.066. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0164121219301013.

[14] D. Parnas, "Designing software for ease of extension and contraction," *IEEE Transactions on Software Engineering*, vol. SE-5, no. 2, pp. 128–138, Mar. 1979, ISSN: 0098-5589, 1939-3520, 2326-3881. DOI: 10.1109/TSE.1979.234169. [Online]. Available: https://ieeexplore.ieee.org/document/1702607/ (visited on 10/11/2026).

[15] Y. Cai and R. Kazman, "DV8: Automated architecture analysis tool suites," in *2019 IEEE/ACM International Conference on Technical Debt (TechDebt)*, Montreal, QC, Canada: IEEE, May 2019, pp. 53–54, ISBN: 978-1-7281-3371-3. DOI: 10.1109/TechDebt.2019.00015. [Online]. Available: https://ieeexplore.ieee.org/document/8786011/.

[16] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Eds., *Compilers: principles, techniques, & tools*, 2. ed., Pearson internat. ed, Boston Munich: Pearson Addison-Wesley, 2007, 1009 pp., ISBN: 978-0-321-48681-3.

[17] M. L. Scott, *Programming language pragmatics*, Fourth edition. Amsterdam Boston Heidelberg London New York Oxford Paris San Diego San Francisco Singapore Sydney Tokyo: Morgan Kaufmann/Elsevier, 2016, 956 pp., ISBN: 978-0-12-410409-9.

[18] F. Ortin, J. Quiroga, O. Rodriguez-Prieto, and M. Garcia, "An empirical evaluation of lex/yacc and ANTLR parser generation tools," *PLOS ONE*, vol. 17, no. 3, S. Consoli, Ed., e0264326, Mar. 3, 2022, ISSN: 1932-6203. DOI: 10.1371/journal.pone.0264326. [Online]. Available: https://dx.plos.org/10.1371/journal.pone.0264326.

[19] G. van Rossum, J. Lehtosalo, and L. Langa, *Pep 484 – type hints*, https://peps.python.org/pep-0484/, Python Enhancement Proposal, 2015.

[20] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, San Jose, CA, USA: IEEE, 2004, pp. 75–86, ISBN: 978-0-7695-2102-2. DOI: 10.1109/CGO.2004.1281665. [Online]. Available: http://ieeexplore.ieee.org/document/1281665/.

[21] Microsoft Corporation, *Language server protocol specification*, Accessed: August 2026, 2026. [Online]. Available: https://microsoft.github.io/language-server-protocol/.

[22] J. website, *Joern documentation*, https://docs.joern.io, Joern is a platform for robust analysis of source code, bytecode, and binary code. It generates code property graphs, a graph representation of code for cross-language code analysis. Code property graphs are stored in a custom graph database. This allows code to be mined using search queries formulated in a Scala-based domain-specific query language. Joern is developed with the goal of providing a useful tool for vulnerability discovery and research in static program analysis., 2026.

[23] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *2014 IEEE Symposium on Security and Privacy*, San Jose, CA: IEEE, May 2014, pp. 590–604, ISBN: 978-1-4799-4686-0. DOI: 10.1109/SP.2014.44. [Online]. Available: http://ieeexplore.ieee.org/document/6956589/.

[24] D. Sas, P. Avgeriou, and F. Arcelli Fontana, "Investigating instability architectural smells evolution: An exploratory case study," in *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Cleveland, OH, USA: IEEE, Sep. 2019, pp. 557–567, ISBN: 978-1-7281-3094-1. DOI: 10.1109/ICSME.2019.00090. [Online]. Available: https://ieeexplore.ieee.org/document/8919109/.

[25] F. Refolli, "Un framework multi-linguaggio per l'identificazione delle dipendenze del codice sorgente," Bachelor's thesis, Università degli Studi di Milano-Bicocca, Milan, Italy, 2023.

[26] L. Teruzzi, "Entangle: un framework multi-linguaggio per l'estrazione automatica dei grafi delle dipendenze," Bachelor's thesis, Università degli Studi di Milano-Bicocca, Milan, Italy, 2024.

[27] Depends Contributors. "Depends: A fast, comprehensive multi-language code dependency analysis tool." (2024), [Online]. Available: https://github.com/multilang-depends/depends.

[28] M. L. Collard and J. I. Maletic, "srcML 1.0: Explore, analyze, and manipulate source code," in *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Raleigh, NC, USA: IEEE, Oct. 2016, pp. 649–649, ISBN: 978-1-5090-3806-0. DOI: 10.1109/ICSME.2016.36. [Online]. Available: https://ieeexplore.ieee.org/document/7816536/.

[29] srcML website, *About srcml*, https://www.srcml.org/about.html, What is srcML? 2026.

[30] Babelfish Project, "Babelfish: Universal abstract syntax tree for multi-language analysis," 2019. [Online]. Available: https://bblf.sh/.

[31] J. Curtis, "On language-agnostic abstract-syntax trees: Student research abstract," in *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*, Virtual Event: ACM, Apr. 25, 2022, pp. 1619–1625, ISBN: 978-1-4503-8713-2. DOI: 10.1145/3477314.3506962. [Online]. Available: https://dl.acm.org/doi/10.1145/3477314.3506962.

[32] M. Schiewe, J. Curtis, V. Bushong, and T. Cerny, "Advancing static code analysis with language-agnostic component identification," *IEEE Access*, vol. 10, pp. 30 743–30 761, 2022, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3160485. [Online]. Available: https://ieeexplore.ieee.org/document/9737496/.

[33] A. Zwaan and H. van Antwerpen, "Scope graphs: The story so far," *OASIcs, Volume 109, EVCS 2023*, vol. 109, R. Lämmel, P. D. Mosses, and F. Steimann, Eds., 32:1–32:13, 2023, Artwork Size: 13 pages, 768667 bytes ISBN: 9783959772679 Medium: application/pdf, ISSN: 2190-6807. DOI: 10.4230/OASICS.EVCS.2023.32. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.EVCS.2023.32.

[34] D. A. Creager and H. van Antwerpen, "Stack graphs: Name resolution at scale," *OASIcs, Volume 109, EVCS 2023*, vol. 109, R. Lämmel, P. D. Mosses, and F. Steimann, Eds., 8:1–8:12, 2023, Artwork Size: 12 pages, 757501 bytes ISBN: 9783959772679 Medium: application/pdf, ISSN: 2190-6807. DOI: 10.4230/OASICS.EVCS.2023.8. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.EVCS.2023.8.

[35] M. Franz, C. T. Lopes, G. Huck, Y. Dong, O. Sumer, and G. D. Bader, "Cytoscape.js: A graph theory library for visualisation and analysis," *Bioinformatics*, vol. 32, no. 2, pp. 309–311, Jan. 15, 2016, ISSN: 1367-4811, 1367-4803. DOI: 10.1093/bioinformatics/btv557. [Online]. Available: https://academic.oup.com/bioinformatics/article/32/2/309/1744007.

[36] S. Klabnik and C. Nichols, *The Rust Programming Language*, 2nd. San Francisco, CA: No Starch Press, 2023, ISBN: 978-1-71850-310-6.

[37] S. Dielh, *Software visualization: visualizing the structure, behaviour, and evolution of software*. Berlin: Springer, 2007, ISBN: 978-3-540-46504-1.

[38] R. Harris and Svelte Contributors. "Svelte 5: Modern reactive component architecture." (2024), [Online]. Available: https://svelte.dev.

[39] H. Balci and U. Dogrusoz, "fCoSE: A fast compound graph layout algorithm with constraint support," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 12, pp. 4582–4593, Dec. 1, 2022, ISSN: 1077-2626, 1941-0506, 2160-9306. DOI: 10.1109/TVCG.2021.3095303. [Online]. Available: https://ieeexplore.ieee.org/document/9477202/.

[40] T. R. Magalhães and J. E. Montandon, "Understanding type hints in python libraries and frameworks: Early insights," in *Proceedings of the 2026 34th IEEE/ACM International Conference on Program Comprehension*, Rio de Janeiro , Brazil: ACM, Apr. 12, 2026, pp. 443–447, ISBN: 979-8-4007-2482-4. DOI: 10.1145/3794763.3794792. [Online]. Available: https://dl.acm.org/doi/10.1145/3794763.3794792.

Appendix A

Mathematical Notation and Glossary

To ensure formal rigor and clarity of exposition, the subsequent chapters utilize a mathematical notation derived from type theory and functional semantics. This chapter serves as a quick reference for the symbology and formal concepts introduced in the formalization of the system.

A.1 Notational Conventions

To ensure formal rigor, consistency, and clarity of exposition across chapters, the formal model adheres to a strict typographic taxonomy comprising five homogeneous symbol families, summarized in Table A.1.

Conceptual Family	Typographic Style	Primary Symbols	Semantics and Scope
Analysis Pipeline	Lowercase Greek	$\phi, \varepsilon, \rho, \gamma$	Operational transformations and pipeline composition ($\phi = \gamma \circ \rho \circ \varepsilon$).
IR Architecture	Uppercase Calligraphic	$\mathcal{M}, \mathcal{S}, \mathcal{F}, \mathcal{A}, \mathcal{X}, \mathcal{B}, \mathcal{C}, \mathcal{Q}$	Structural data model components, containers, and query algebra domain.
Resolution Contexts	Uppercase Greek & Calligraphic	$\Gamma, \Delta, \Sigma \quad \mathcal{E}, \mathcal{P}, \mathcal{K}, \mathcal{M}$	Scoping environments ($\Gamma = \langle \mathcal{E}, \mathcal{P}, \mathcal{K}, \mathcal{M} \rangle$), symbol stack ($\Delta$), and lexical scopes ($\Sigma$).
Semantic Types	Lowercase Greek	$\iota, \pi, \tau, \sigma, \delta, \theta$	Atomic type references (τ), identifiers (ι), qualified paths (π), signatures (σ), imports (δ), and type parameters (θ).
Structural Members	Lowercase Latin	f, p, v, q, k	Concrete member instances: fields (f), parameters (p), variables (v), queries (q), and type kinds (k).

Table A.1: Typographic and notational taxonomy adopted across the formal model.

The model relies on the following standard algebraic and set theory conventions:

- **Tuples (Cartesian Product)** $\langle x_1, x_2, \ldots, x_n \rangle$: Represent composite structures (corresponding to `structs` in the source code). Each element of the tuple defines an essential property of the entity.
- **Disjoint Union (Sum Type)** $A \mid B \mid C$: Indicates that a value belongs to exactly one variant among those listed (corresponding to tagged unions or `enums` in the code).

- **Vectors and Sequences** $\vec{X}$: Denotes a finite, ordered sequence of elements of type X (corresponding to `Vec<X>`).
- **Optionality** $X^?$: Indicates a value that may be present (of type X) or absent (corresponding to `Option<X>`).
- **Function Signatures** $f : A \to B$: Denotes a mapping from domain A to codomain B. Multi-argument functions use Cartesian products ($A \times B \to C$).
- **Boolean Domain** $\mathbb{B}$: The set of truth values $\{\text{true}, \text{false}\}$.

A.2 Glossary of Types and Domains

The following definitions catalog the symbols that constitute the formal Intermediate Representation (IR) and the analysis pipeline.

A.2.1 Base Types, Paths, and Identifiers

- $\mathcal{I}$ (**Identifier**): The set of valid strings used as names for variables, functions, types, and modules.
- Π (**Path Domain**) and π (**Qualified Name**): The domain of non-empty sequences of identifiers $\Pi = \mathcal{I}^+ = \{\langle \iota_1, \ldots, \iota_n \rangle \mid \iota_j \in \mathcal{I}, n \geq 1\}$, where each $\pi \in \Pi$ uniquely identifies a component's absolute path.
- τ (**Type Reference**): An abstract reference to a type, represented by a sum type modeling its discovery lifecycle.
- θ (**Type Parameter**): A formal generic parameter $\langle \iota, \vec{\tau}_{bounds} \rangle$ declaring identifier ι and constraint bounds $\vec{\tau}_{bounds}$.
- δ (**Import Directive**): A dependency declaration $\langle \pi_{path}, \iota^?_{alias}, \mathbb{B}_{wildcard} \rangle$ specifying an imported path, an optional local alias, and a wildcard flag.

A.2.2 Structural Members and Signatures

- v (**Variable**): A local variable declaration within a block, structurally equivalent to a field ($v \equiv f$).
- f (**Field**): A member data field defined by the tuple $\langle \iota, \tau, \vec{\tau}_{ann} \rangle$.
- p (**Parameter**): A formal parameter defined by $\langle \iota^?, \tau, \mathbb{B} \rangle$, comprising an optional name, a type reference, and a variadicity indicator.
- σ (**Signature**): The functional signature tuple $\langle \vec{p}, \tau_{ret} \rangle$, decoupling inputs and output from the implementing body.
- $\mathcal{B}$ (**Block**): A lexical execution block $\langle \vec{f}_{local}, \vec{\tau}_{calls}, \vec{\tau}_{inst}, \vec{\tau}_{accesses}, \vec{\tau}_{casts}, \vec{\mathcal{B}}_{sub} \rangle$.
- $\mathcal{F}$ (**Function**): A universal functional component $\langle \pi, \sigma, \vec{\theta}, \mathcal{B}^?, \mathbb{B}_{ctor}, \vec{\tau}_{ann} \rangle$, binding a qualified name to a signature, an ordered vector of type parameters, an optional body block, a constructor flag, and annotations.

A.2.3 Architectural Entities

- $\mathcal{S}$ (**Structured Type**): The abstraction for classes, structs, interfaces, traits, and enums, defined by $\langle \pi, k, \vec{\theta}, \vec{f}, \vec{\mathcal{F}}, \vec{\tau}_{sup}, \vec{\mathcal{S}}_{nest}, \vec{\tau}_{ann}, \vec{\delta}_{imp} \rangle$ where $k \in \{\text{Class}, \text{Struct}, \text{Interface}, \text{Trait}, \text{Enum}, \text{EnumVariant}\}$.
- $\mathcal{A}$ (**Alias**): A type alias mapping a qualified name to an existing target type reference: $\langle \pi, \tau_{target} \rangle$.
- $\mathcal{X}$ (**Implementation Extension**): An implementation block aggregating methods and aliases for a target type τ_{target}, flattened into $\mathcal{S}$ during Name Resolution.
- $\mathcal{C}$ (**Component**): The universal sum type of entities promoted to graph vertices: Modules ($\mathcal{M}$), Structured Types ($\mathcal{S}$), Type Aliases ($\mathcal{A}$), Functions ($\mathcal{F}$), Fields (f), Primitives, and External symbols.
- $\mathcal{M}$ (**Module**): The primary hierarchical namespace unit.
- $\mathcal{D}$ (**Intermediate Representation Domain**): The universe of valid component hierarchies rooted in module trees $\vec{\mathcal{M}}$.

A.2.4 Resolution Model and Scoping Contexts

- $\mathcal{Q}$ (**Query Algebra**): The domain of resolution expressions composed of primitives $\texttt{Find}(s)$, $\texttt{Extract}(q, s)$, and $\texttt{Call}(q)$, with $q \in \mathcal{Q}$.
- Σ (**Scope**): A single environment of lexical visibility containing a symbol table, supertype inheritance links, and imports.
- $\mathcal{E}$ (**Scope Tree**): The workspace-wide hierarchical tree of Σ scopes, enabling lexical scope climbing.
- Δ (**Symbol Stack**): The ephemeral stack of lexical frames $\Delta = [s_1, \ldots, s_n]$ where each frame $s_i : \mathcal{I} \to \mathcal{Q}$ maps local identifiers to queries during Query Building.
- $\mathcal{P}$ (**Primitive Registry**): The language-specific table of built-in primitive type keywords.
- $\mathcal{K}$ (**Language Configuration**): The configuration tuple governing language-specific semantic features (receiver keywords, transitive imports, type alias conventions).
- Γ (**Execution Context**): The evaluation context $\Gamma = \langle \mathcal{E}, \mathcal{P}, \mathcal{K}, \mathcal{M} \rangle$ used to resolve queries, including the super-scope memoization mapping $\mathcal{M}$.
- $\mathcal{G}$ (**Dependency Graph**): The directed graph $\mathcal{G} = (V, E)$ composed of component vertices V and typed dependency edges E.

A.2.5 Analysis Pipeline Phases

- ϕ (**Full Analysis Pipeline**): The end-to-end transformation $\phi : \mathcal{W} \to \mathcal{G}$, defined as $\phi = \gamma \circ \rho \circ \varepsilon$.
- ε (**Workspace Extraction**): The language-dependent phase $\varepsilon : \mathcal{W} \to \vec{\mathcal{M}}$ translating source code into unresolved IR modules.
- ρ (**Name Resolution**): The language-independent phase $\rho : \vec{\mathcal{M}} \to \vec{\mathcal{M}}$ that resolves symbolic type references into semantic pointers ($\rho = \rho_{\text{exec}} \circ \rho_{\text{build}}$).
  - ρ_{build} (**Query Building**): Traverses the IR with the Symbol Stack Δ, transforming unresolved paths into algebraic queries $q \in \mathcal{Q}$.
  - ρ_{exec} (**Query Execution**): Evaluates queries q against the Execution Context $\Gamma = \langle \mathcal{E}, \mathcal{P}, \mathcal{K}, \mathcal{M} \rangle$.
- γ (**Graph Construction**): The final phase $\gamma : \vec{\mathcal{M}} \to \mathcal{G}$ flattening the resolved IR into vertices V and typed edges E.

Appendix B

Source Code and Reproducibility

The complete source code of the *OmniDeps* framework, including the heuristic extraction engine, the resolution algebra, the embedded web visualizer, and the experimental benchmark suite developed for this thesis, is open-source and publicly available.

B.1 Repository Availability

The project repository can be accessed at the following URL:

`https://github.com/FerrarioChristian/omnideps`

To ensure long-term archival and full replication of the empirical evaluation, a complete reproducibility package containing the source code of *OmniDeps* and *Graph-Synchron*, the source snapshots of the 156 analyzed repositories, the extracted dependency archives (both raw and preprocessed), and the comparative evaluation reports is publicly archived on Zenodo at:

`https://doi.org/10.5281/zenodo.23315197`

B.2 Installation for End Users

OmniDeps is distributed as a single standalone binary. The web visualizer (SvelteKit) is pre-compiled and embedded directly inside the Rust executable. To use the tool, it is not necessary to install Node.js, npm, or Rust. Users can simply:

1. Navigate to the **Releases** page on the GitHub repository.
2. Download the executable for their Operating System (Windows `.exe`, macOS, or Linux).
3. Run it directly from the terminal.

B.3 Command Line Interface

The framework provides several subcommands to orchestrate the analysis and visualization process.

Analyze Code

To extract the dependency graph from a target project, use the `analyze` command:

```
omnideps analyze /path/to/project --output graph.json
```

This command parses the codebase, executes the extraction heuristics, runs the name resolution engine, and outputs the final dependency graph directly in standard Cytoscape.js JSON format.

Custom Configuration

To initialize a default configuration file (`omnideps.json`) that can be customized to override the default extraction rules and resolution strategies, run:

```
omnideps config init
```

The generated configuration file can then be provided to the analyzer using the `-config` flag (e.g., `omnideps analyze /path -config omnideps.json`).

Web Visualizer

To interactively explore the generated graphs, the embedded web interface can be launched via:

```
omnideps serve
```

This starts an ultra-lightweight local HTTP server (defaulting to port 3000) and automatically opens the default web browser to the interactive UI.

Benchmarking Suite

The project includes custom benchmarks to accurately measure false positives and false negatives on the AST abstraction across the supported languages. To run a specific benchmark:

```
omnideps benchmark run tests/benchmarks/benchmark-rust -o /output_dir
```

To execute the entire suite and generate the aggregated CSV report:

```
omnideps benchmark all -o /output_dir
```

B.4 Building from Source

For development and reproducibility purposes, the project can be built from source. This requires both **Node.js** and the **Rust Toolchain** (`cargo` and `rustc`).

First, build the SvelteKit frontend, which generates static files in the `visualizer-svelte/build` directory:

```
cd visualizer-svelte
npm install
npm run build
cd ..
```

Next, compile the Rust executable. The `rust-embed` macro will automatically package the compiled Svelte web assets inside the binary during compilation:

```
cargo build --release
```

The final executable will be located in `target/release/omnideps`.

B.5 Testing

The framework includes a comprehensive suite of unit and integration tests designed to validate the internal extraction and resolution logic. To execute the test suite, run:

```
cargo test
```

B.6 Benchmarking Pipeline

To orchestrate multi-tool executions and evaluate graph similarity across the dataset, the companion benchmarking framework *Graph-Synchron* is open-source and available at:

`https://github.com/FerrarioChristian/graph-synchron`

Automated Benchmarking

The `measurer.py` script coordinates the execution of static analysis tools across target repositories, recording runtimes and storing extracted graphs into a shared evaluation archive:

```
python3 measurer.py -l Java -t omnideps depends
```

A specific project can be targeted by specifying the repository name (e.g., `-p junit4`).

Similarity Evaluation and Pairwise Diffing

To compute the harmonized Jaccard similarity, superiority, and inferiority metrics across a language corpus, `main.py` provides the `score` subcommand:

```
python3 main.py score -f omnideps -s depends -l Java
```

To inspect set-theoretic structural differences and categorical edge agreement rates between two extractors, use the `diff pairwise` subcommand:

```
python3 main.py diff pairwise -f omnideps -s depends -l Java
```

Both subcommands can also be restricted to an individual project using the `-p <project>` flag.